



Ayudantía 5

A*, Heurísticas, Minimax y Presentación Tarea 3

Por Vicente Cordero y Felipe Espinoza

5 de mayo, 2025



Contenidos

1. Repaso Búsqueda
2. Heurísticas
3. A^*
4. Búsqueda Adversaria
5. Apoyo Tarea 3



Repaso Búsqueda



Búsqueda Genérica

- **S:** Conjunto de nodos o estados.
- **A:** Conjunto de acciones o transiciones entre nodos.
- **s_init:** Nodo inicial desde donde se empieza la búsqueda.
- **G:** Conjunto de nodos objetivo o estados meta.



Búsqueda Genérica

Open : contiene todos los nodos **generados** pero no explorados

Closed : contiene los nodos **explorados**

Sucesores : **sucesores** o hijos de un nodo



Búsqueda Genérica

Input: Un problema de búsqueda (S, A, S_{init}, G)

Output: Un nodo objetivo

```
1  Open es un contenedor vacío
2  Closed es un conjunto vacío
3  Inserta  $S_{init}$  a Open
4   $parent(S_{init}) = null$ 
5  while Open  $\neq \emptyset$ 
6       $u \leftarrow \text{Extraer}(\text{Open})$ 
7      Inserta  $u$  en Closed
8      for each  $v \in Succ(u) \setminus (Open \cup Closed)$ 
9           $parent(v) = u$ 
10         if  $v \in G$  return  $v$ 
11     Inserta  $v$  a Open
```

La diferencia con el resto de los algoritmos es la estructura de datos que se usa para *Open*



Heurísticas



Algoritmos vistos hasta ahora

**Búsqueda
Genérica**

DFS

IDDFS

BFS

¿Que tienen en común?



Algoritmos vistos hasta ahora

- Encuentran caminos más cortos en un grafo **ponderado**, es decir, **encuentran soluciones óptimas**
- ¿Qué sucede cuando el grafo de búsqueda es **MUY** grande?

**Alto consumo
de memoria**

**Exploración
innecesaria**

**Gran tiempo de
ejecución**

- Queremos que nuestra búsqueda obtenga el camino al nodo objetivo, **evitando revisar nodos innecesarios**. ¿Cómo lo logramos?



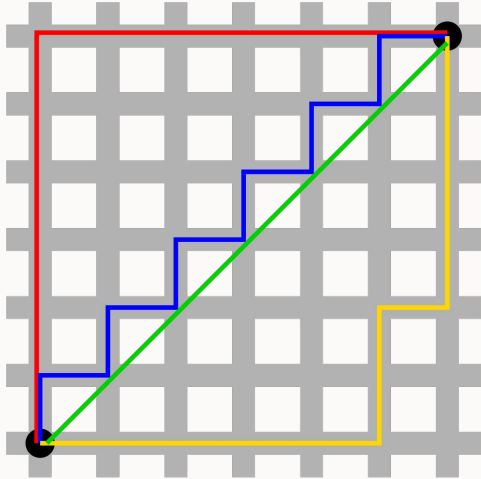
Función Heurística

- Función que **estima** la cercanía de un nodo **s** a un nodo objetivo **G**.
- Para crearla necesitamos saber **dónde se ubica el nodo objetivo** o cómo conseguirlo.
- Con ella podemos discriminar qué nodo **conviene** explorar, **priorizando** nodos que podrían llevar a la solución de forma **más rápida**.

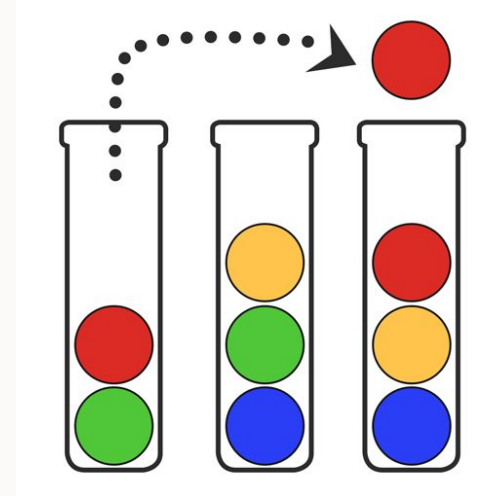
$$h(s)$$



Ejemplos de Heurísticas



- ● ● Distancia Manhattan
- Distancia Euclidiana



Cantidad de bolitas en un frasco que no tienen un mismo color



Ejemplos de Heurísticas

1	2	3
4		6
7	8	5

Cantidad de piezas en
la posición correcta

○	×	○
×		×
	○	

Cantidad de figuras del
jugador que están
alineadas



Admisibilidad

Definición

Una heurística ***h*** se dice **admissible** si para todo estado ***s*** se cumple que:

$$h(s) \leq h^*(s)$$

Donde $h^*(s)$ es el costo de un camino óptimo desde el estado ***s*** a un estado objetivo

Nuestra función es **admissible si nunca sobreestima** respecto a un **camino óptimo**



Admisibilidad

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es admisible?

H1

$H_1 = 6$				G
S				
$H_1 = 8$				

H2

$H_2 = 8$				G
S				
$H_2 = 6$				



Admisibilidad

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es admisible?

H1 es admisible!

$H_1 = 6$				G
S				
$H_1 = 8$				

H2 no es admisible
ya que $H_2 = 8 > h^* = 6$

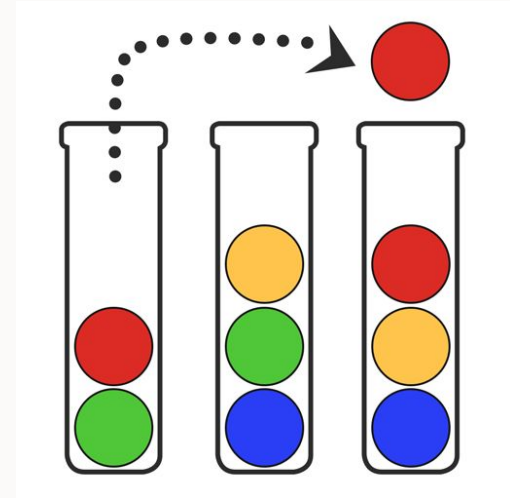
$H_2 = 8$				G
S				
$H_2 = 6$				



Admisibilidad - ejemplo

El objetivo es que cada tubo tenga bolitas de un solo color.

Una heurística admisible podría ser contar la cantidad de bolitas que no tienen el mismo color en un tubo. Notar que **al menos** este número de bolitas **se deben desplazar** para lograr el estado objetivo. Por lo que **nunca sobreestima** el trabajo que se debe realizar para llegar al objetivo.





Consistencia

Definición

Una heurística ***h*** se dice **consistente** si y sólo si:

- $h(s) = 0$, para todo $s \in G$
- $h(s) \leq c(s, s') + h(s')$ para todo vecino s' de s

O de manera equivalente $h(s) - h(s') \leq c(s, s')$

Nuestra función es consistente si nunca sobreestima en los estados objetivos ni el **costo es mayor que la variación de la heurística** entre dos estados vecinos



Consistencia

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es consistente?

H1

$H_1 = 10$	$H_1 = 8$	$H_1 = 6$	$H_1 = 4$	$H_1 = 2$
$H_1 = 12$		$H_1 = 8$		$H_1 = 0$
S		$H_1 = 10$		$H_1 = 2$
$H_1 = 16$	$H_1 = 14$	$H_1 = 12$		$H_1 = 4$
$H_1 = 18$		$H_1 = 14$		$H_1 = 6$

G

H2

$H_2 = 5$	$H_2 = 4$	$H_2 = 3$	$H_2 = 2$	$H_2 = 1$
$H_2 = 4$		$H_2 = 2$		$H_2 = 0$
S		$H_2 = 3$		$H_2 = 1$
$H_2 = 6$	$H_2 = 5$	$H_2 = 4$		$H_2 = 2$
$H_2 = 7$		$H_2 = 5$		$H_2 = 3$

G



Consistencia

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es consistente?

H_1 no es consistente

$H_1 = 10$	$H_1 = 8$	$H_1 = 6$	$H_1 = 4$	$H_1 = 2$
$H_1 = 12$		$H_1 = 8$		$H_1 = 0$
S		$H_1 = 10$		$H_1 = 2$
$H_1 = 16$	$H_1 = 14$	$H_1 = 12$		$H_1 = 4$
$H_1 = 18$		$H_1 = 14$		$H_1 = 6$

G

H_2 si lo es

$H_2 = 5$	$H_2 = 4$	$H_2 = 3$	$H_2 = 2$	$H_2 = 1$
$H_2 = 4$		$H_2 = 2$		$H_2 = 0$
S		$H_2 = 3$		$H_2 = 1$
$H_2 = 6$	$H_2 = 5$	$H_2 = 4$		$H_2 = 2$
$H_2 = 7$		$H_2 = 5$		$H_2 = 3$

G



Consistencia y admisibilidad

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es consistente y/o admisible?

$H1$

$H_1 = 5$	$H_1 = 4$	$H_1 = 3$	$H_1 = 2$	$H_1 = 1$
$H_1 = 3$		$H_1 = 2$		$H_1 = 0$
$S : H_1 = 5$		$H_1 = 3$		$H_1 = 1$
$H_1 = 6$	$H_1 = 5$	$H_1 = 4$		$H_1 = 2$
$H_1 = 7$		$H_1 = 5$		$H_1 = 3$

G

$H2$

$H_2 = 5$	$H_2 = 4$	$H_2 = 3$	$H_2 = 2$	$H_2 = 1$
$H_2 = 4$		$H_2 = 2$		$H_2 = 0$
$S : H_2 = 5$		$H_2 = 3$		$H_2 = 1$
$H_2 = 6$	$H_2 = 5$	$H_2 = 4$		$H_2 = 2$
$H_2 = 7$		$H_2 = 5$		$H_2 = 3$

G



Consistencia y admisibilidad

Definamos el costo de moverse entre casillas igual a 1. ¿Cuál heurística es consistente y/o admisible?

H_1 es admisible pero no consistente

H_2 si lo es

$H_1 = 5$	$H_1 = 4$	$H_1 = 3$	$H_1 = 2$	$H_1 = 1$
$H_1 = 3$		$H_1 = 2$		$H_1 = 0$
$S : H_1 = 5$		$H_1 = 3$		$H_1 = 1$
$H_1 = 6$	$H_1 = 5$	$H_1 = 4$		$H_1 = 2$
$H_1 = 7$		$H_1 = 5$		$H_1 = 3$

G

$H_2 = 5$	$H_2 = 4$	$H_2 = 3$	$H_2 = 2$	$H_2 = 1$
$H_2 = 4$		$H_2 = 2$		$H_2 = 0$
$S : H_2 = 5$		$H_2 = 3$		$H_2 = 1$
$H_2 = 6$	$H_2 = 5$	$H_2 = 4$		$H_2 = 2$
$H_2 = 7$		$H_2 = 5$		$H_2 = 3$

G



Relación consistencia y admisibilidad

Definición

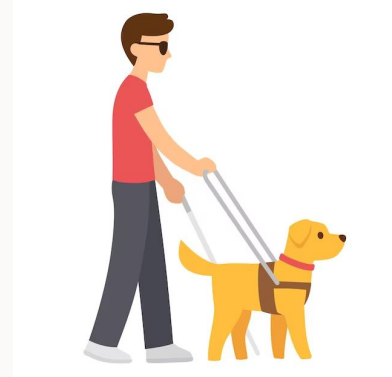
Si una heurística ***h*** es consistente, entonces ***h*** es admisible

¡Esta relación no es invertible! Es decir, si una heurística es admisible, no necesariamente va a ser consistente.



Heurísticas admisibles

- Podemos observar que una heurística admisible no es más que una estimación del costo (que **nunca sobreestima**) para llegar al estado objetivo.
- Una heurística es mejor en cuanto **mejor sea tal predicción**.
- Esta nos ayuda a discernir cuál es el mejor camino que se puede tomar dada la información del **estado actual** y a encontrar óptimos...





A* (AStar)

A*



A* es un algoritmo utilizado en problemas de búsqueda, su característica distintiva es que utiliza heurística, utilizando la siguiente función de evaluación $f(n) = g(n) + h(n)$

- La rapidez y complejidad computacional está estrechamente relacionada a la **heurística seleccionada**.
- Siempre entrega soluciones óptimas si se usa con una **heurística admisible**.
- Dijkstra es A* con heurística cero.

Dijkstra

$$f(n) = g(n)$$

A*

$$f(n) = g(n) + h(n)$$



A*

$$f(n) = g(n) + h(n)$$

$g(n)$: Costo de un camino desde s_0 hasta un nodo n .

$g(n)$ **NO ES FUNCIÓN!**



Algoritmo de A*

Algoritmo A*

Input: Un problema de búsqueda (S, A, s_0, G)

Output: Un nodo objetivo

- 1 **for each** $s \in S$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
- 8 Insertar v

La extracción es del u con **menor valor** $f(s) = g(s) + h(s)$

Se revisa si se llegó al nodo objetivo cuando se expande (notar que **no existirá un camino de menor costo si** es que es el **nodo objetivo**, ¿por qué?)

Procedimiento de insertar v



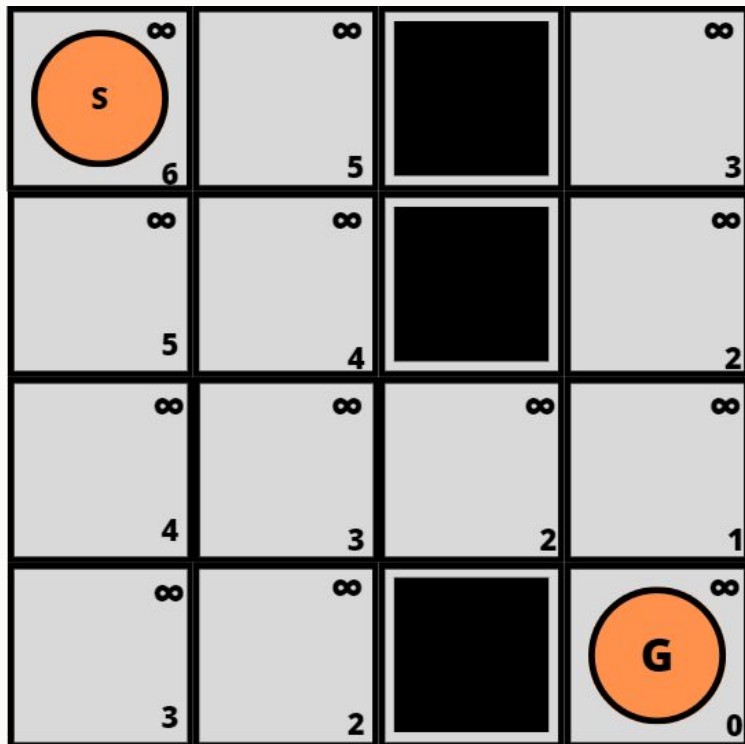
Procedimiento insertar v

Insertar v en Open

- 1 $cost_v = g(u) + c(u, v)$ // el costo de llegar a v por u
- 2 **if** $cost_v \geq g(v)$ **return** // seguimos solo si $cost_v < g(v)$
- 3 $parent(v) \leftarrow u$
- 4 $g(v) \leftarrow cost_v$
- 5 $f(v) \leftarrow g(v) + h(v)$
- 6 **if** $v \in Open$ **then** Reordenar $Open$ // depende de la impl.
- 7 **else** Insertar v en $Open$



EJEMPLO



$$f(s) = g(s) + h(s)$$

$g(s)$: largo del camino

$h(s)$: distancia de manhattan



1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$

2 $Open \leftarrow \{s_0\}$

3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$

4 while $Open \neq \emptyset$

5 Extrae un u desde $Open$ con menor valor- f

6 if u es objetivo return u

7 for each $v \in Succ(u)$ do

1 $cost_v = g(u) + c(u, v)$

2 if $cost_v \geq g(v)$ return

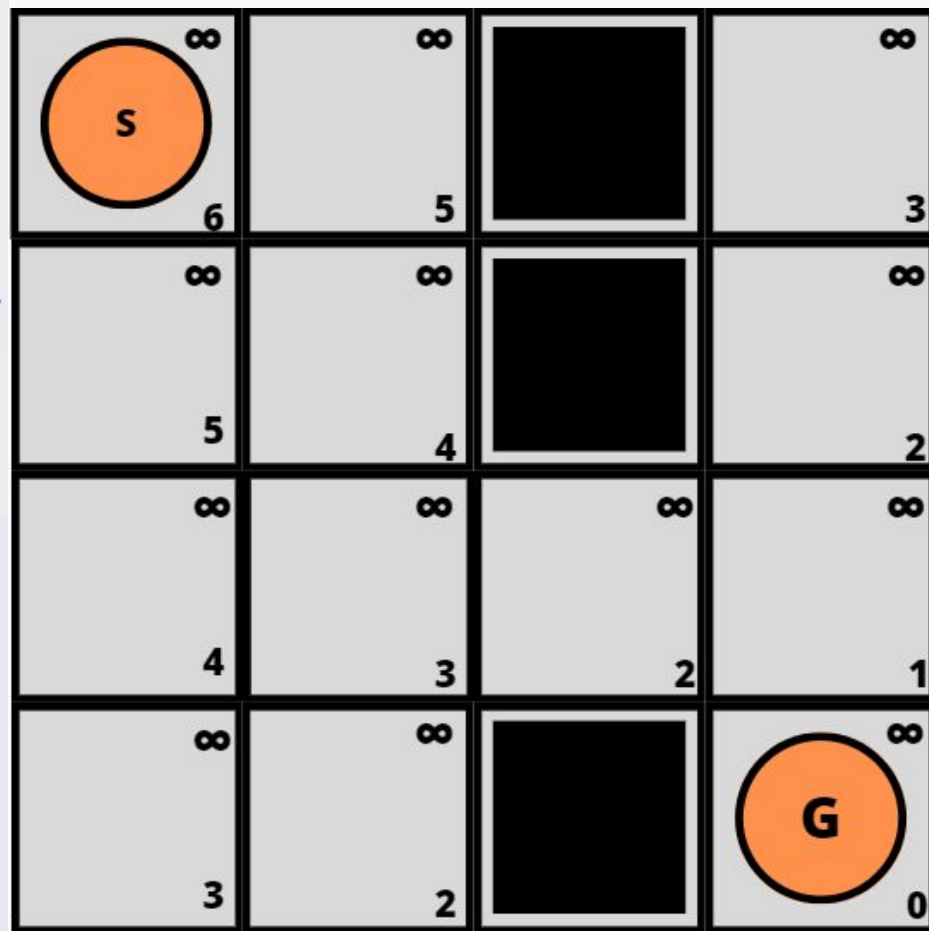
3 $parent(v) \leftarrow u$

4 $g(v) \leftarrow cost_v$

5 $f(v) \leftarrow g(v) + h(v)$

6 if $v \in Open$ then Reordenar $Open$

7 else Insertar v en $Open$



1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$

2 $Open \leftarrow \{s_0\}$

3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$

4 while $Open \neq \emptyset$

5 Extrae un u desde $Open$ con menor valor- f

6 if u es objetivo return u

7 for each $v \in Succ(u)$ do

1 $cost_v = g(u) + c(u, v)$

2 if $cost_v \geq g(v)$ return

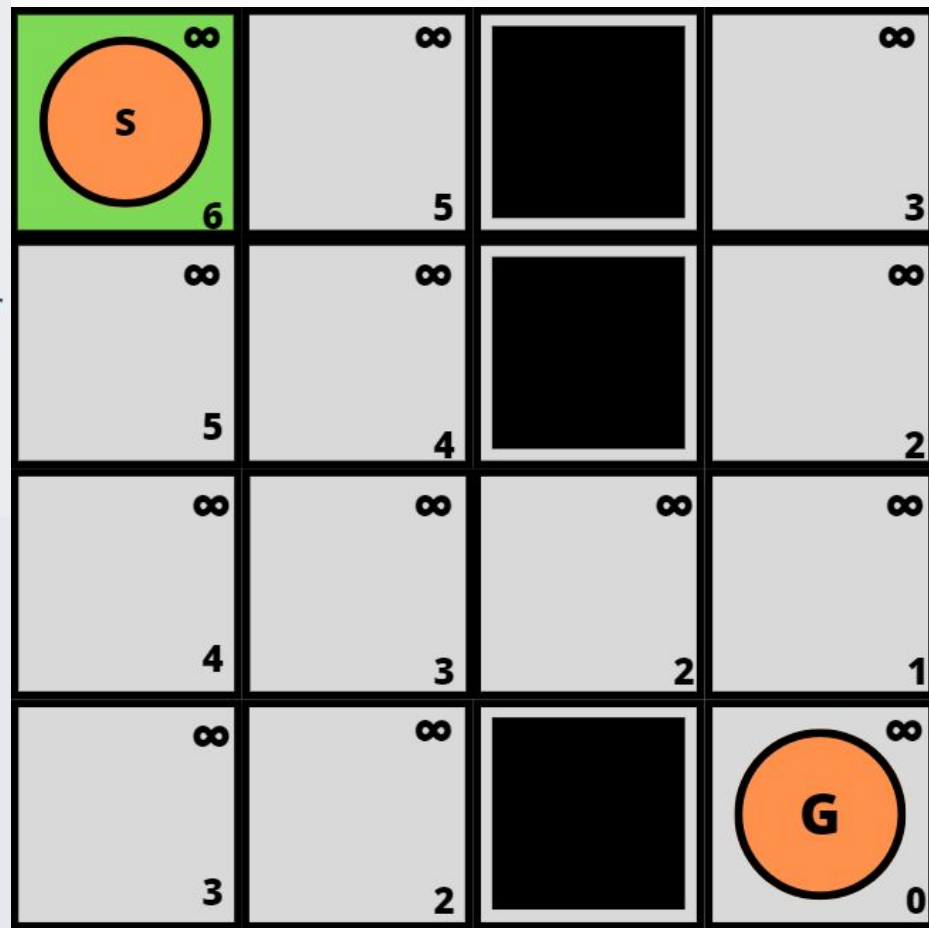
3 $parent(v) \leftarrow u$

4 $g(v) \leftarrow cost_v$

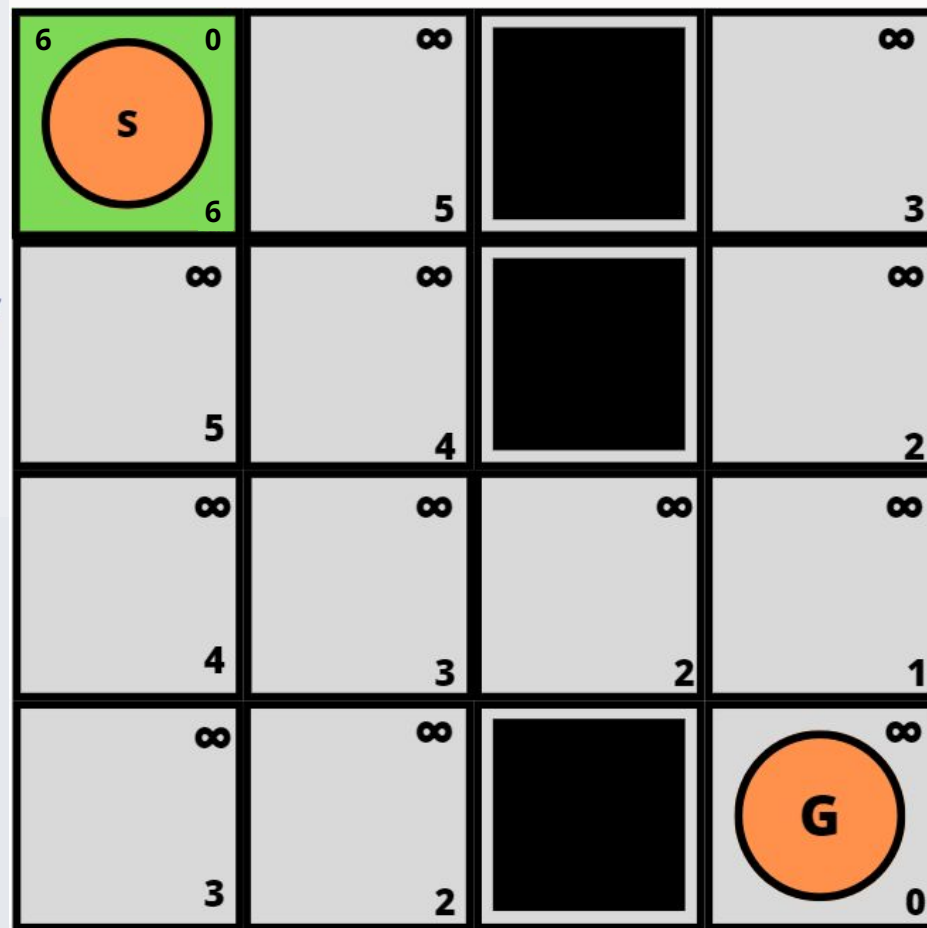
5 $f(v) \leftarrow g(v) + h(v)$

6 if $v \in Open$ then Reordenar $Open$

7 else Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0; f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$

2 $Open \leftarrow \{s_0\}$

3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$

4 while $Open \neq \emptyset$

5 Extrae un u desde $Open$ con menor valor- f

6 if u es objetivo return u

7 for each $v \in Succ(u)$ do

1 $cost_v = g(u) + c(u, v)$

2 if $cost_v \geq g(v)$ return

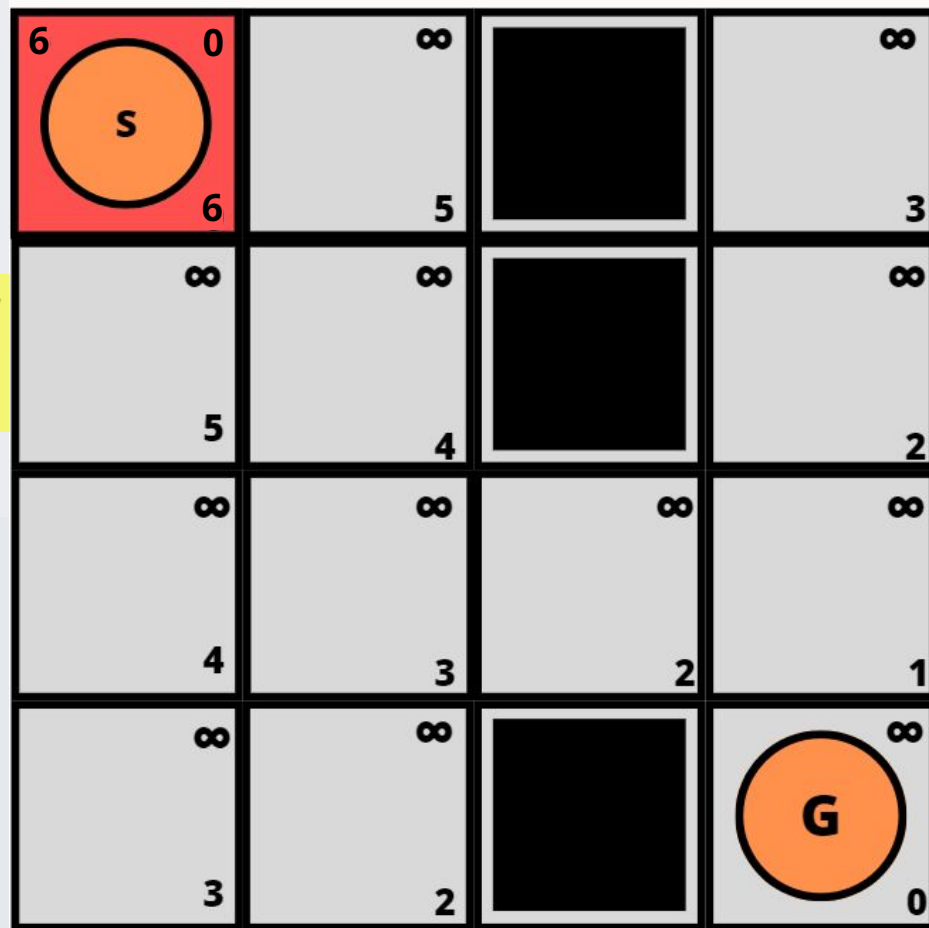
3 $parent(v) \leftarrow u$

4 $g(v) \leftarrow cost_v$

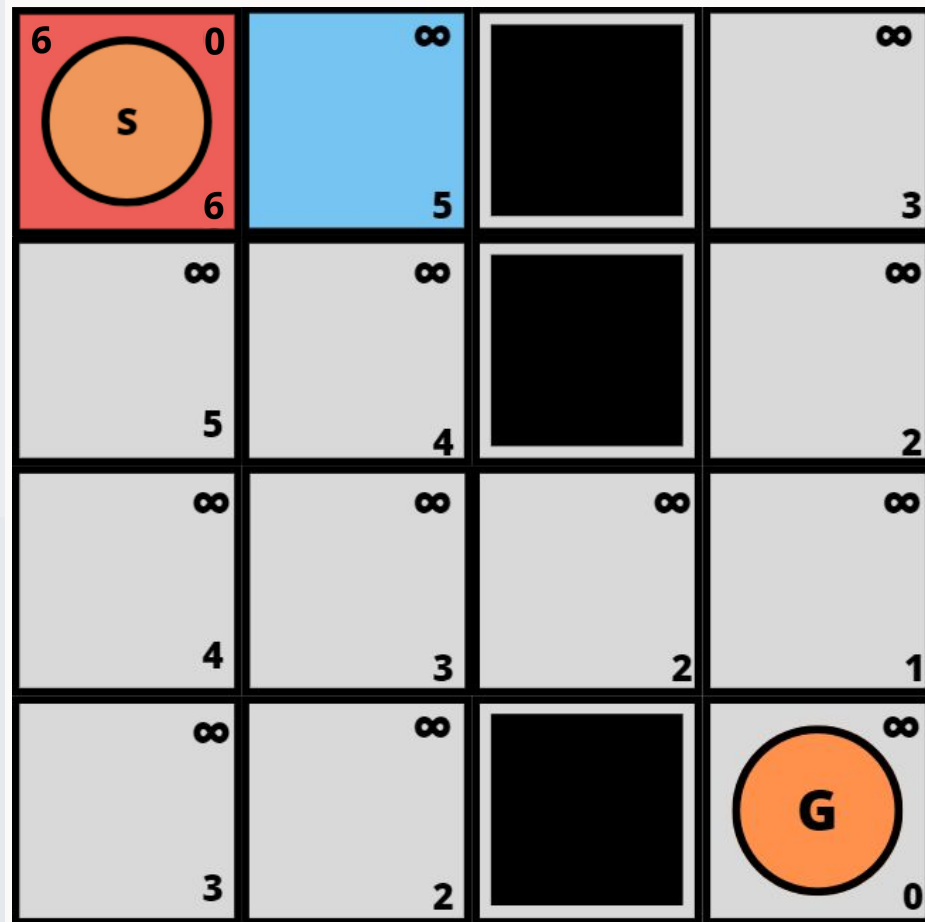
5 $f(v) \leftarrow g(v) + h(v)$

6 if $v \in Open$ then Reordenar $Open$

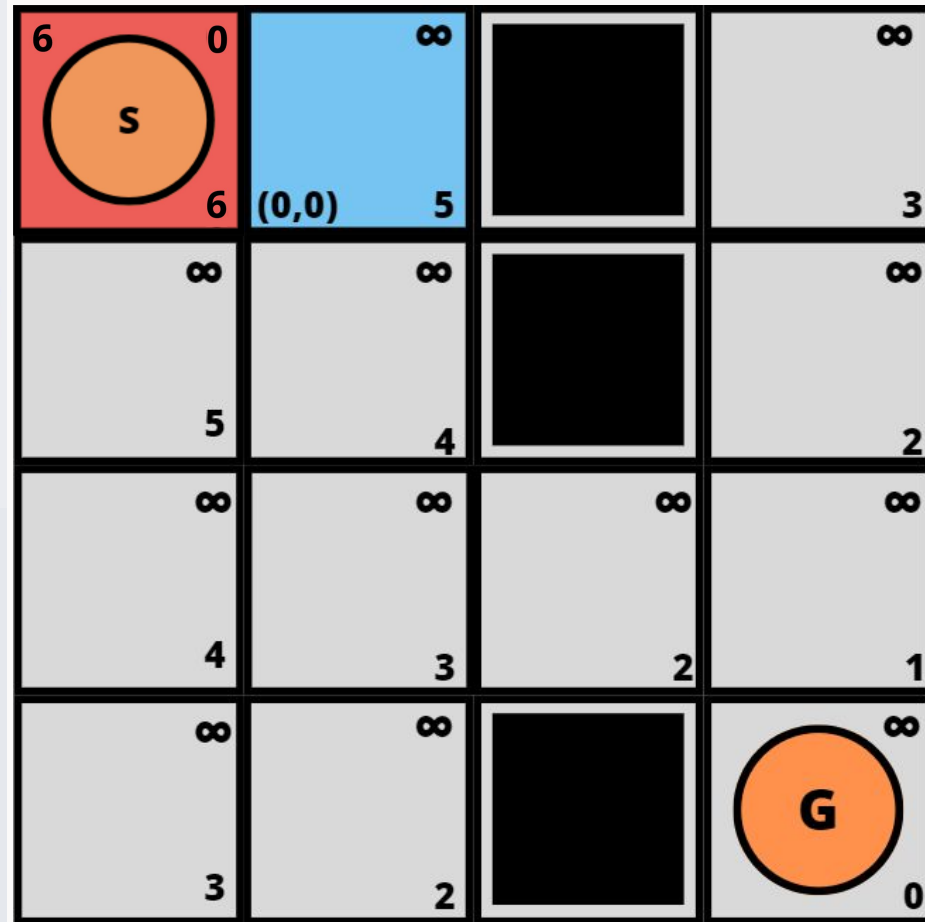
7 else Insertar v en $Open$



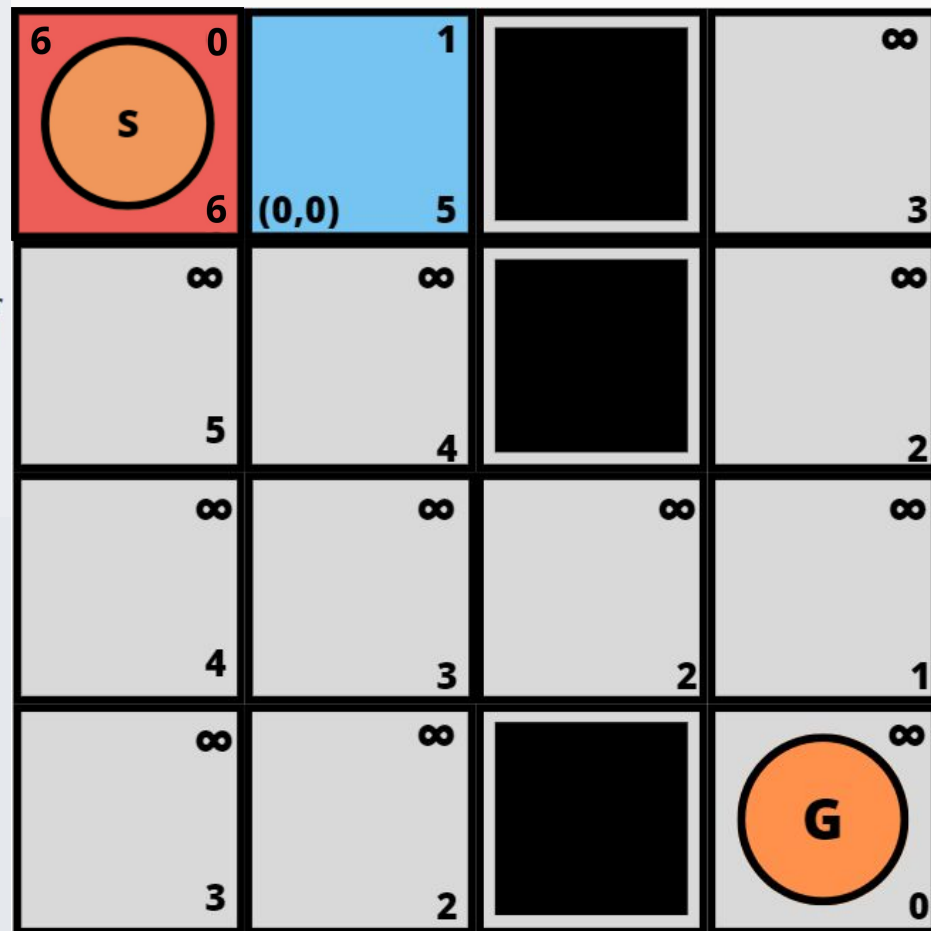
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



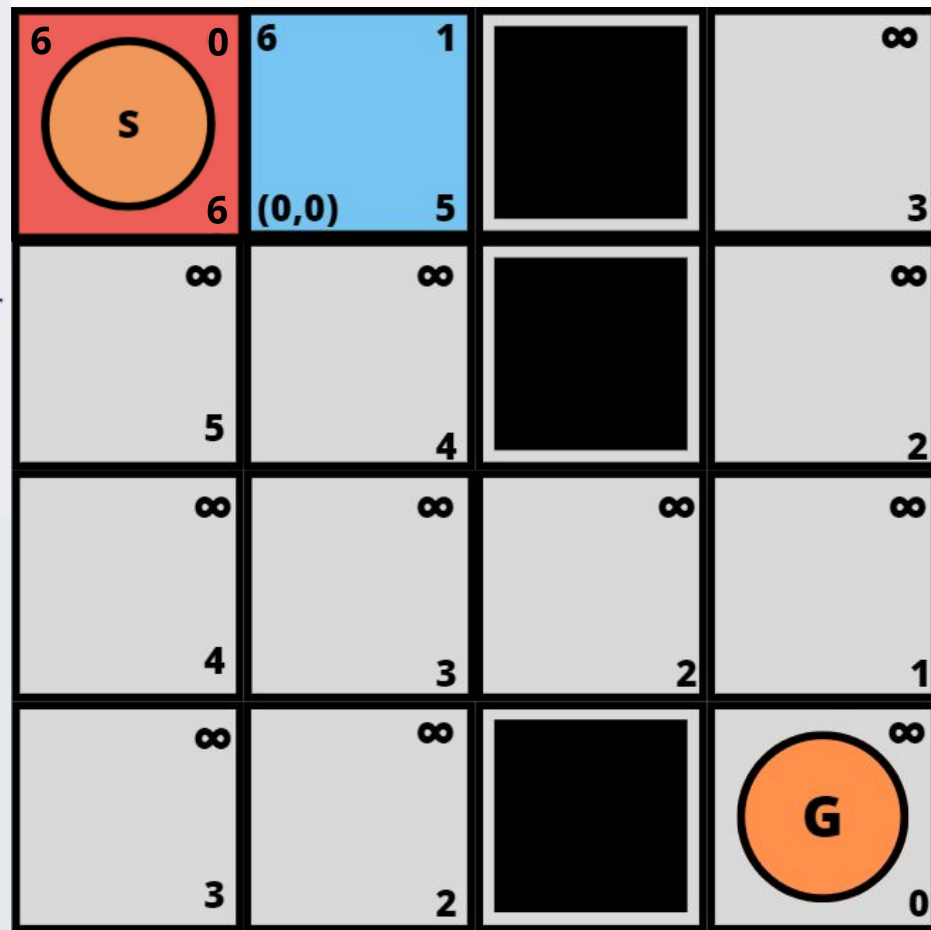
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



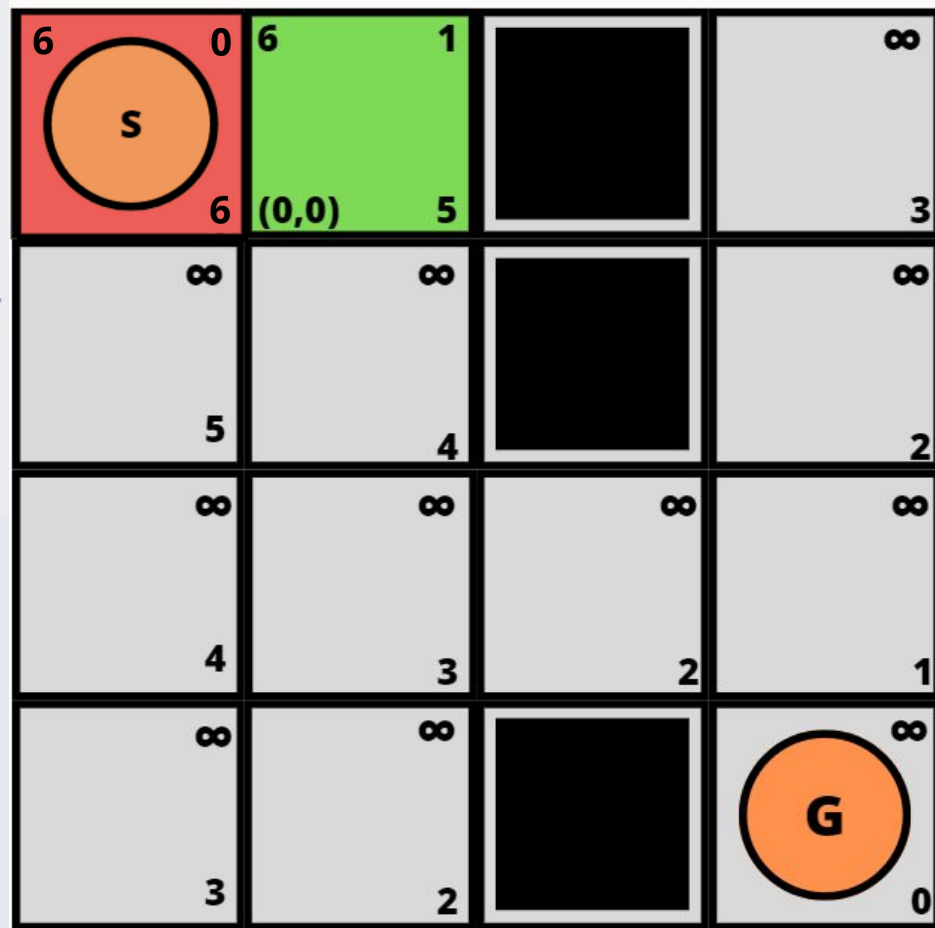
- 1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 while $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 if u es objetivo return u
- 7 for each $v \in Succ(u)$ do
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 if $cost_v \geq g(v)$ return
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 if $v \in Open$ then Reordenar $Open$
 - 7 else Insertar v en $Open$



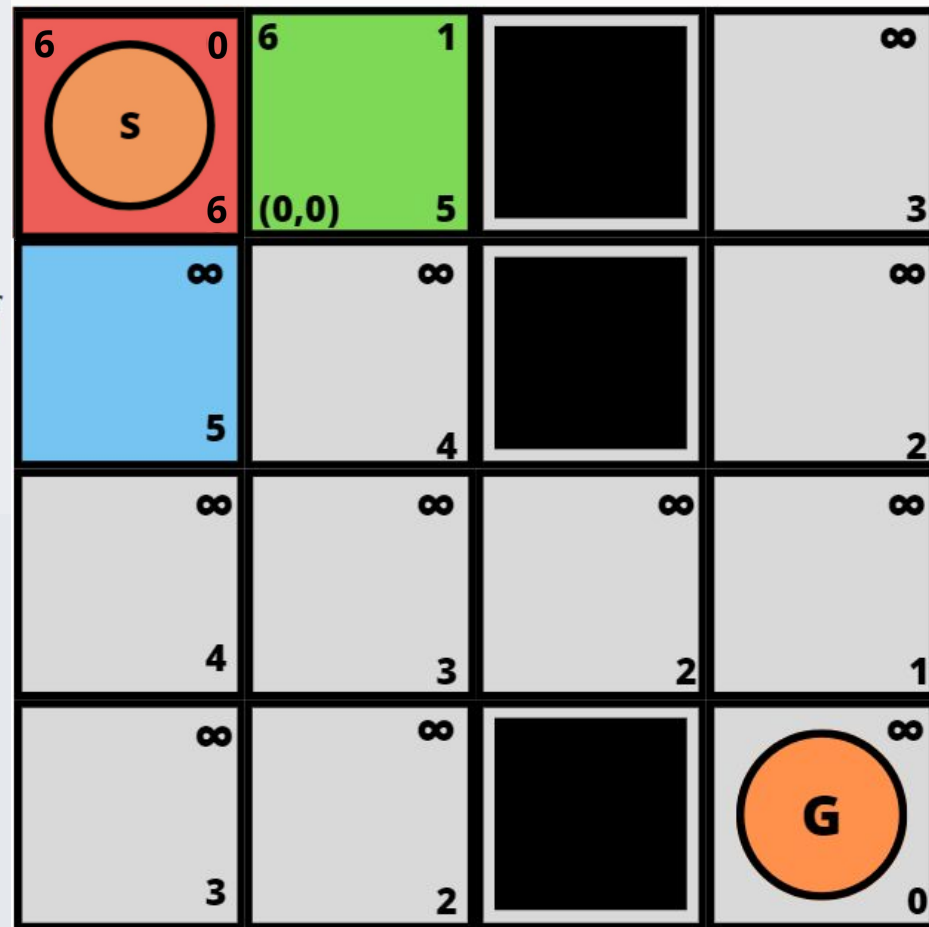
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



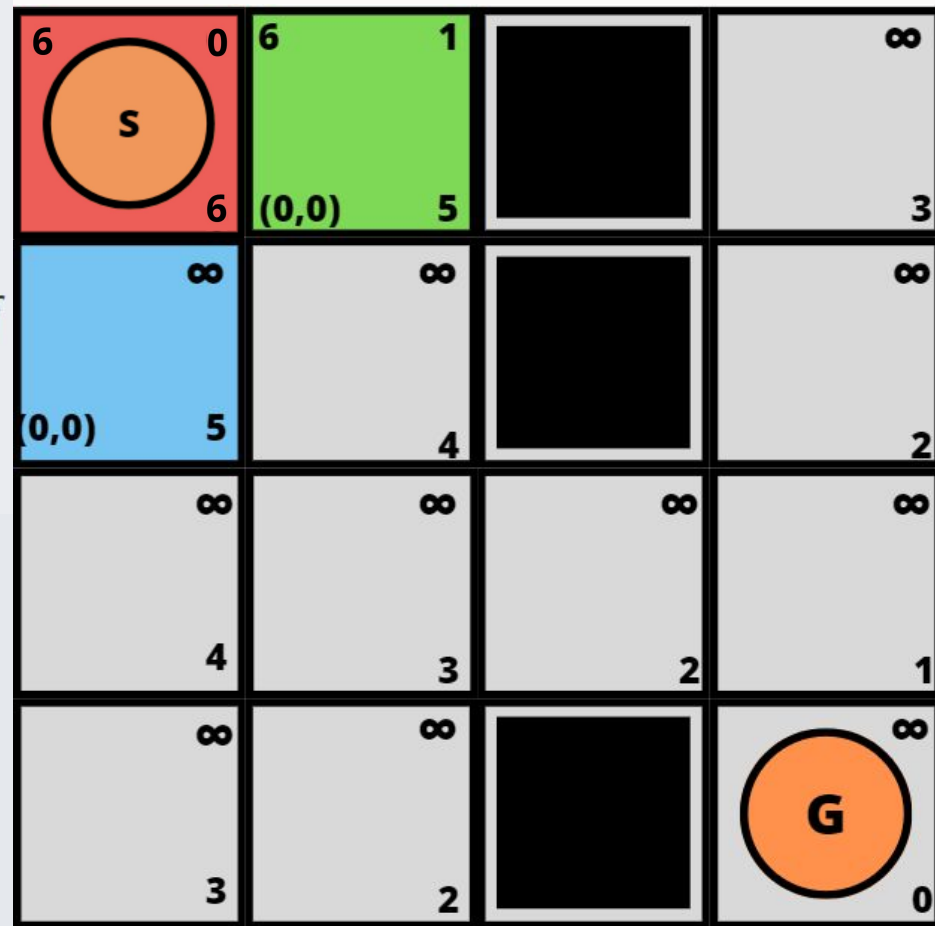
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



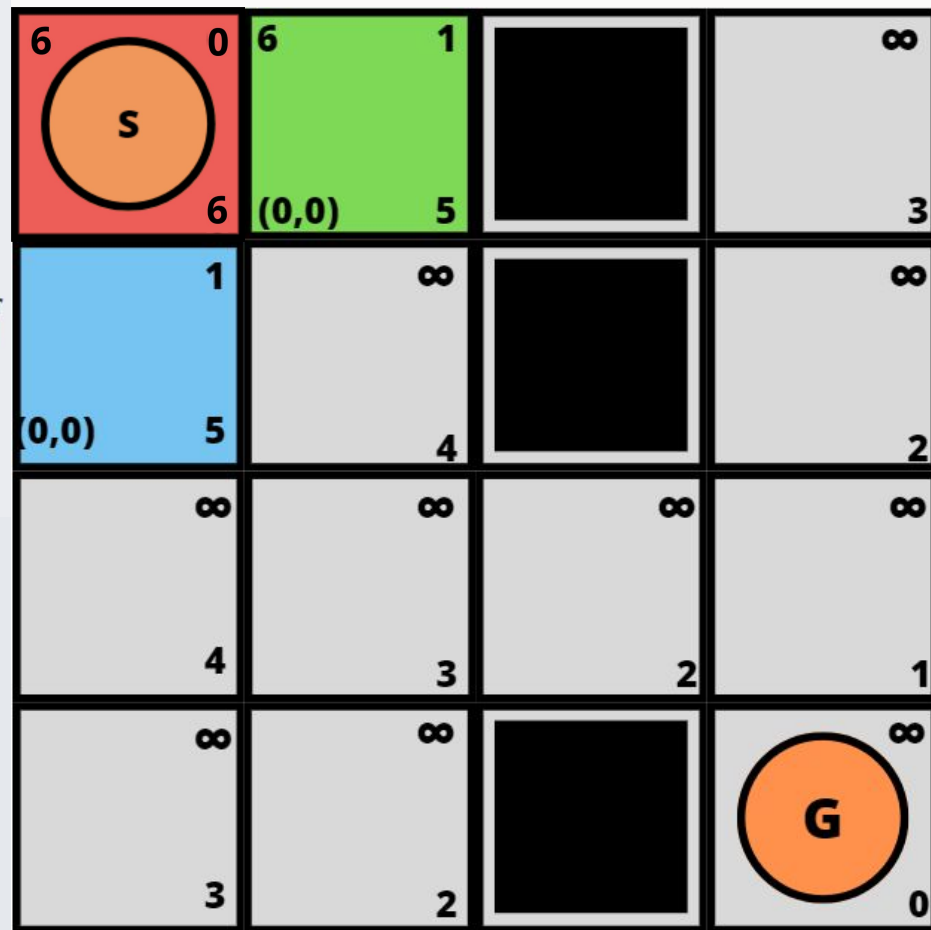
- 1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 while $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 if u es objetivo return u
- 7 for each $v \in Succ(u)$ do
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 if $cost_v \geq g(v)$ return
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 if $v \in Open$ then Reordenar $Open$
 - 7 else Insertar v en $Open$



- 1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 while $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 if u es objetivo return u
- 7 for each $v \in Succ(u)$ do
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 if $cost_v \geq g(v)$ return
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 if $v \in Open$ then Reordenar $Open$
 - 7 else Insertar v en $Open$



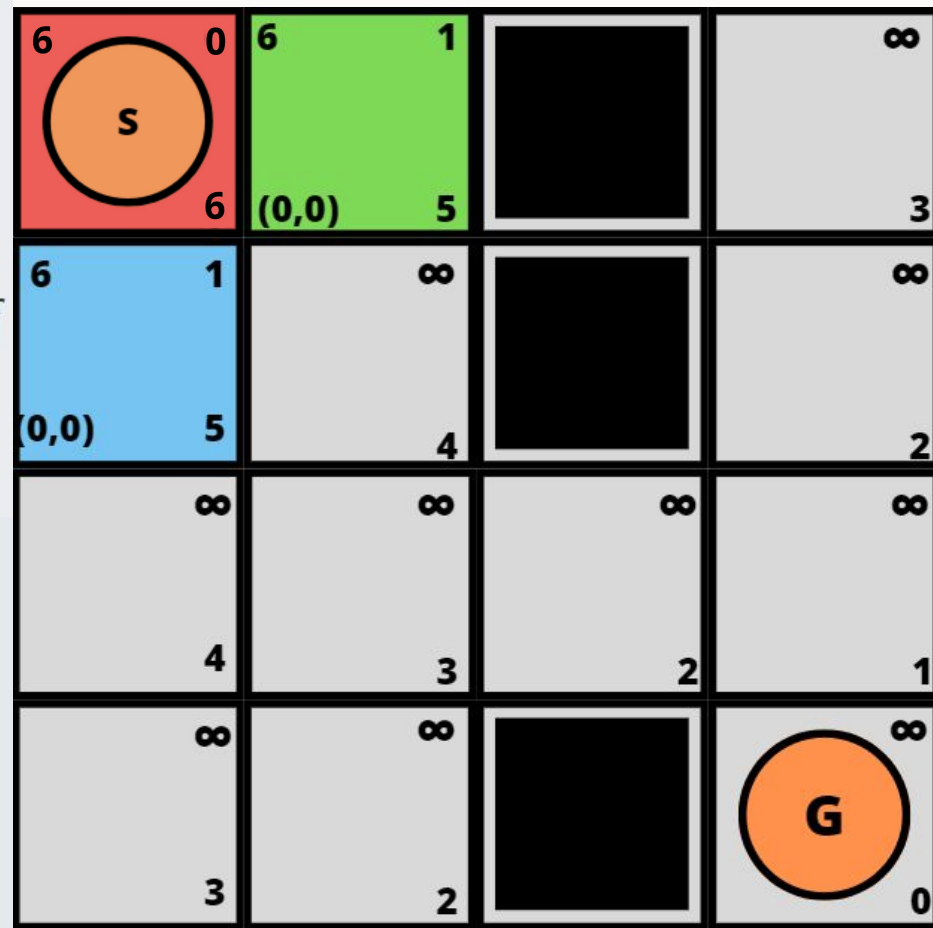
- 1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 while $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 if u es objetivo return u
- 7 for each $v \in Succ(u)$ do
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 if $cost_v \geq g(v)$ return
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 if $v \in Open$ then Reordenar $Open$
 - 7 else Insertar v en $Open$



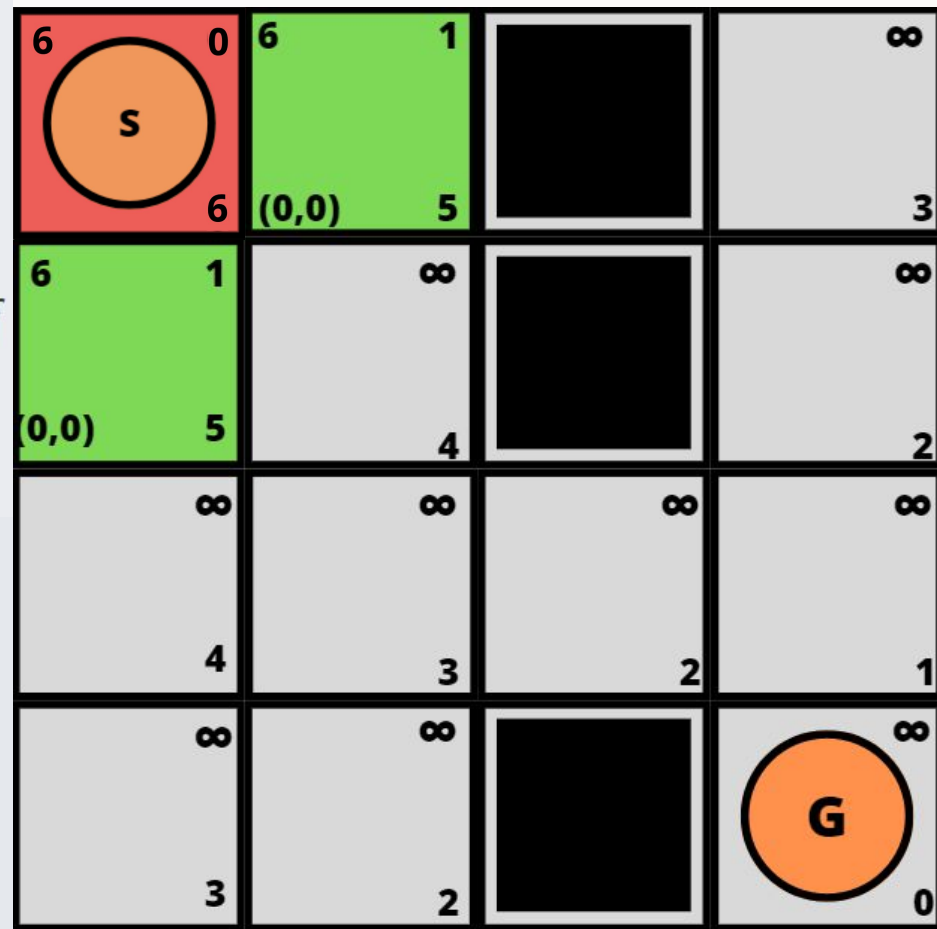
```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
1    $cost_v = g(u) + c(u, v)$ 
2   if  $cost_v \geq g(v)$  return
3    $parent(v) \leftarrow u$ 
4    $g(v) \leftarrow cost_v$ 
5    $f(v) \leftarrow g(v) + h(v)$ 
6   if  $v \in Open$  then Reordenar  $Open$ 
7   else Insertar  $v$  en  $Open$ 

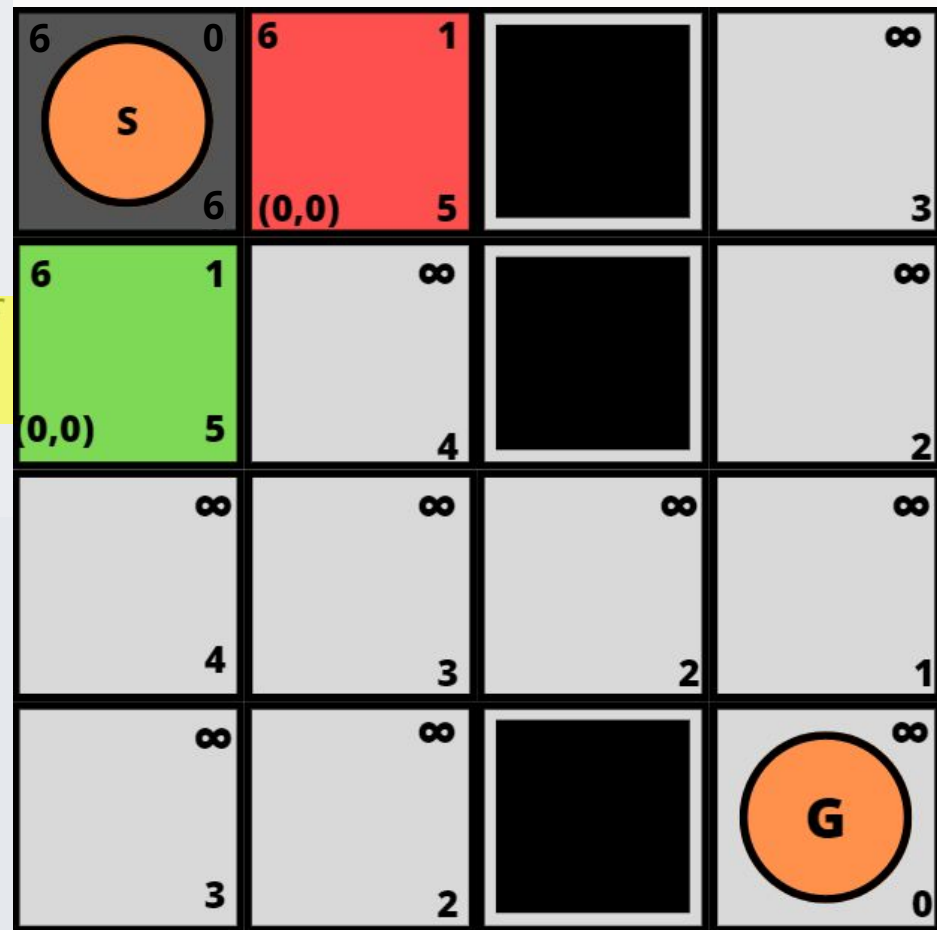
```



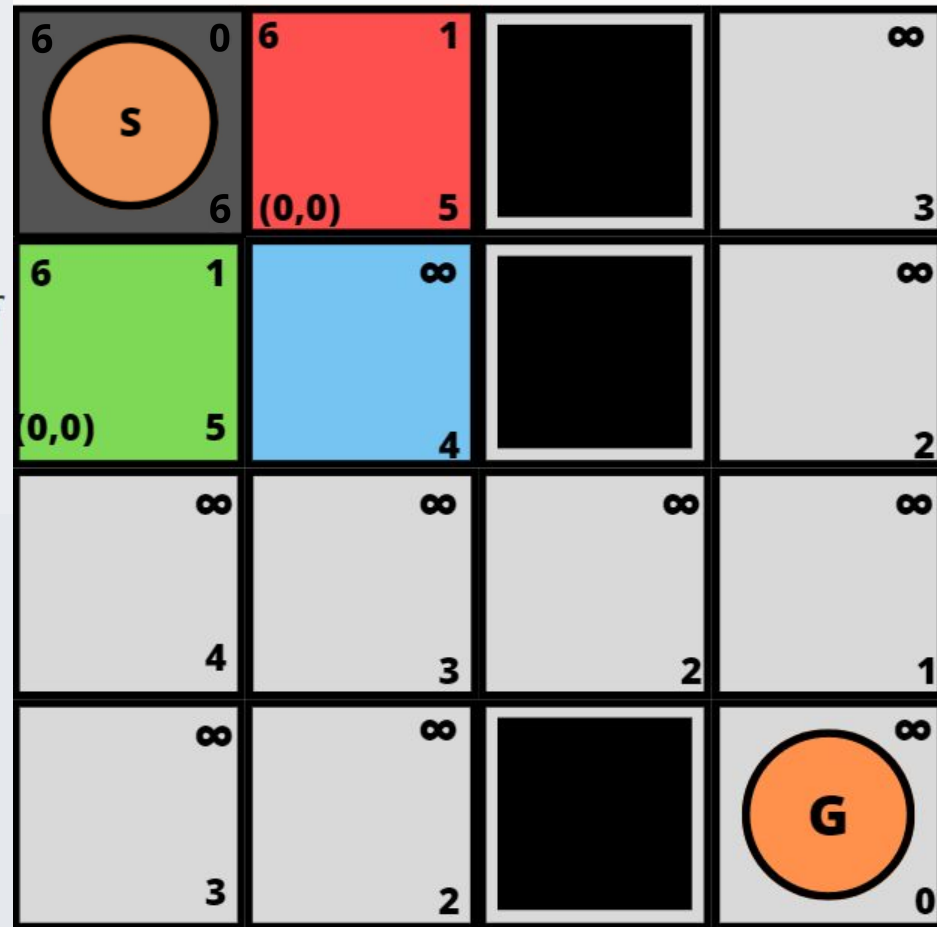
- 1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 while $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 if u es objetivo return u
- 7 for each $v \in Succ(u)$ do
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 if $cost_v \geq g(v)$ return
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 if $v \in Open$ then Reordenar $Open$
 - 7 else Insertar v en $Open$



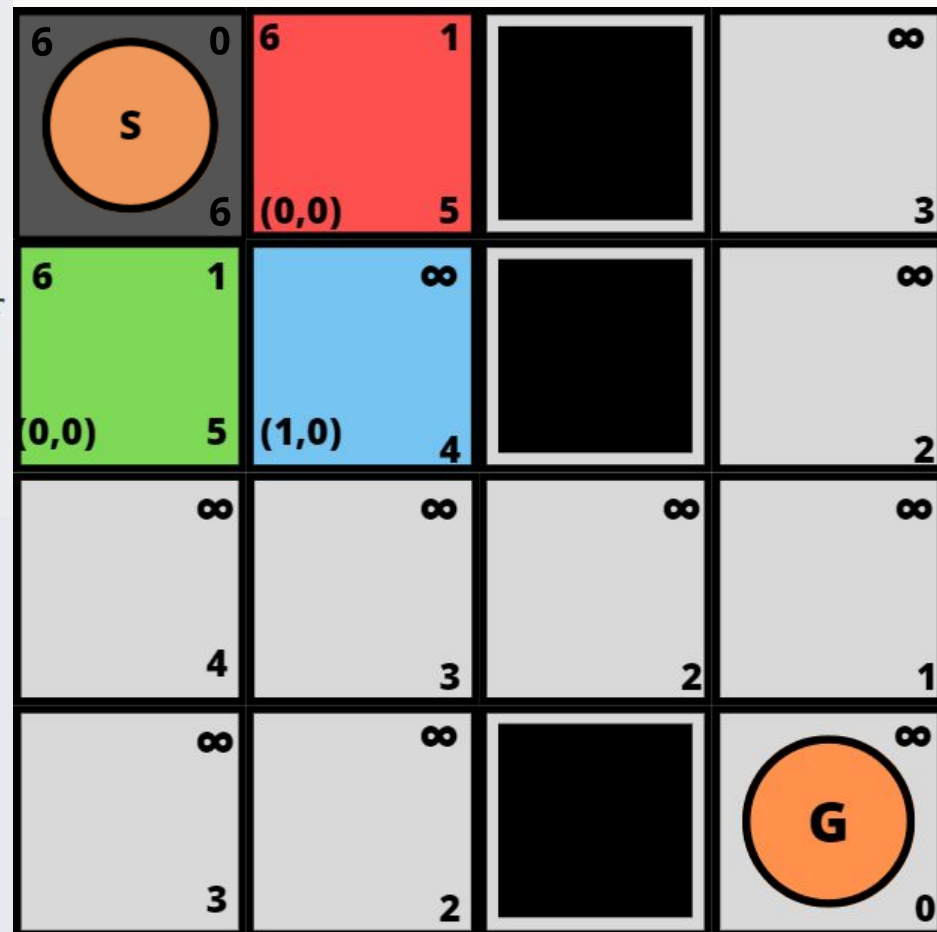
- 1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 while $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 if u es objetivo return u
- 7 for each $v \in Succ(u)$ do
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 if $cost_v \geq g(v)$ return
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 if $v \in Open$ then Reordenar $Open$
 - 7 else Insertar v en $Open$



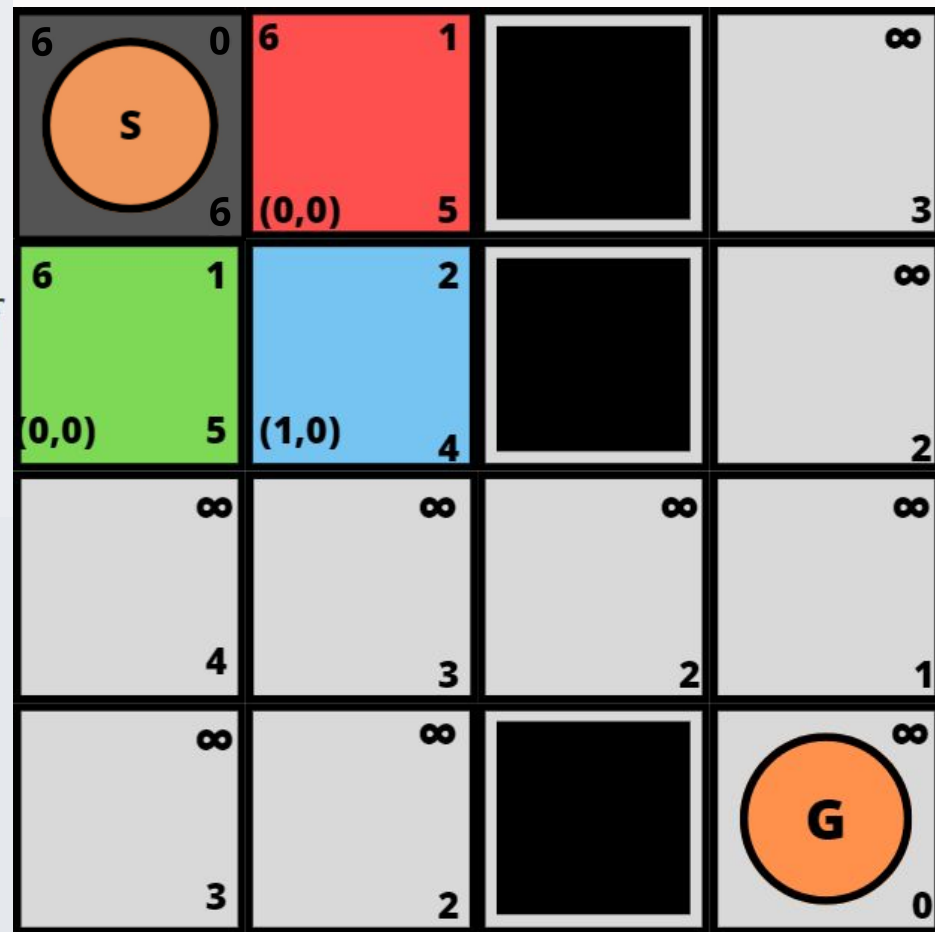
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



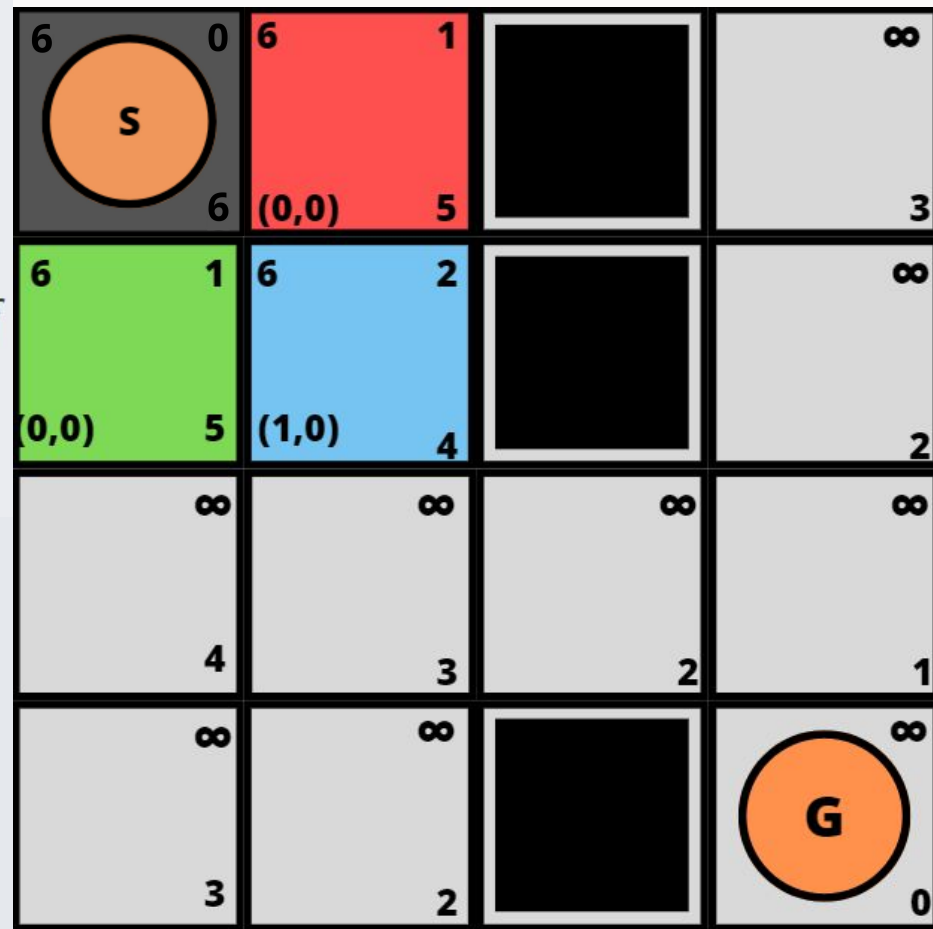
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



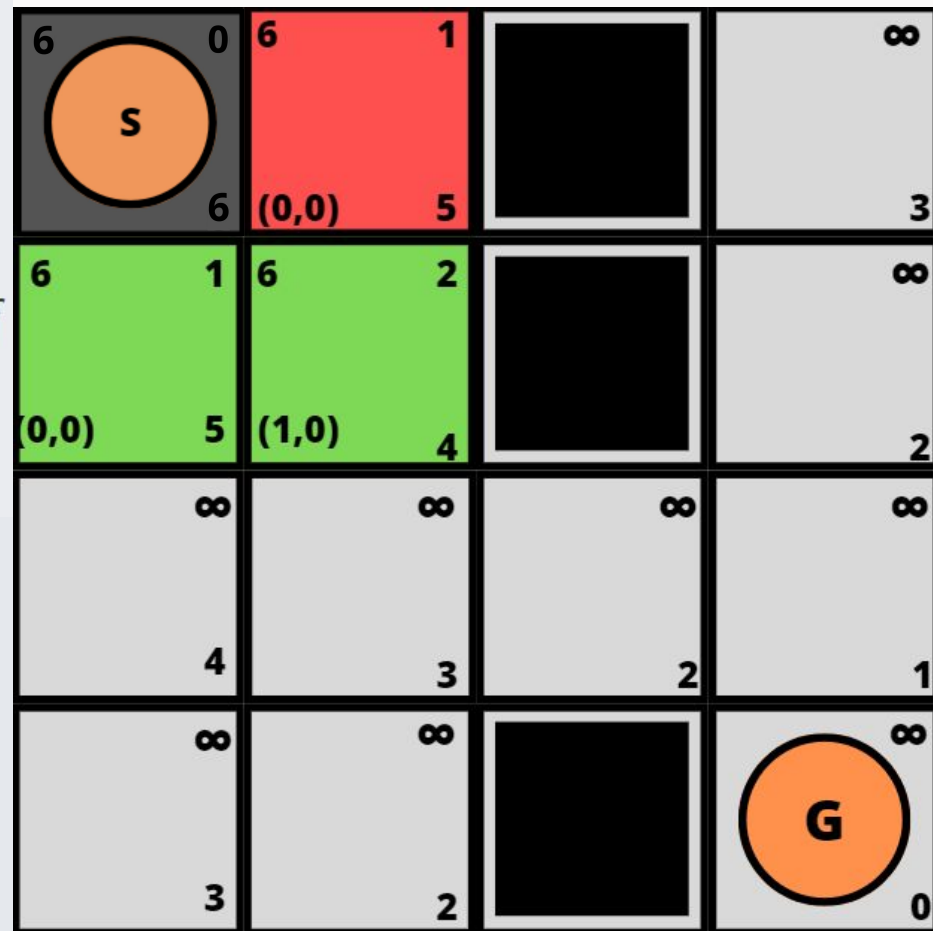
```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
1    $cost_v = g(u) + c(u, v)$ 
2   if  $cost_v \geq g(v)$  return
3    $parent(v) \leftarrow u$ 
4    $g(v) \leftarrow cost_v$ 
5    $f(v) \leftarrow g(v) + h(v)$ 
6   if  $v \in Open$  then Reordenar  $Open$ 
7   else Insertar  $v$  en  $Open$ 

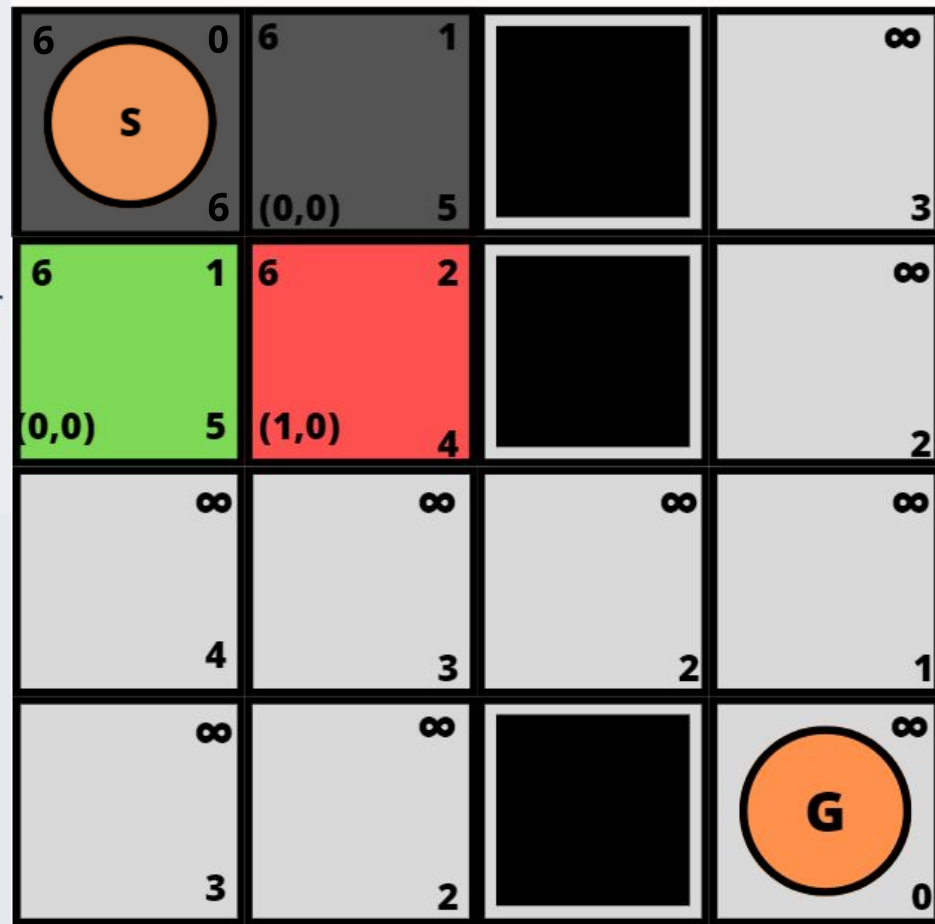
```



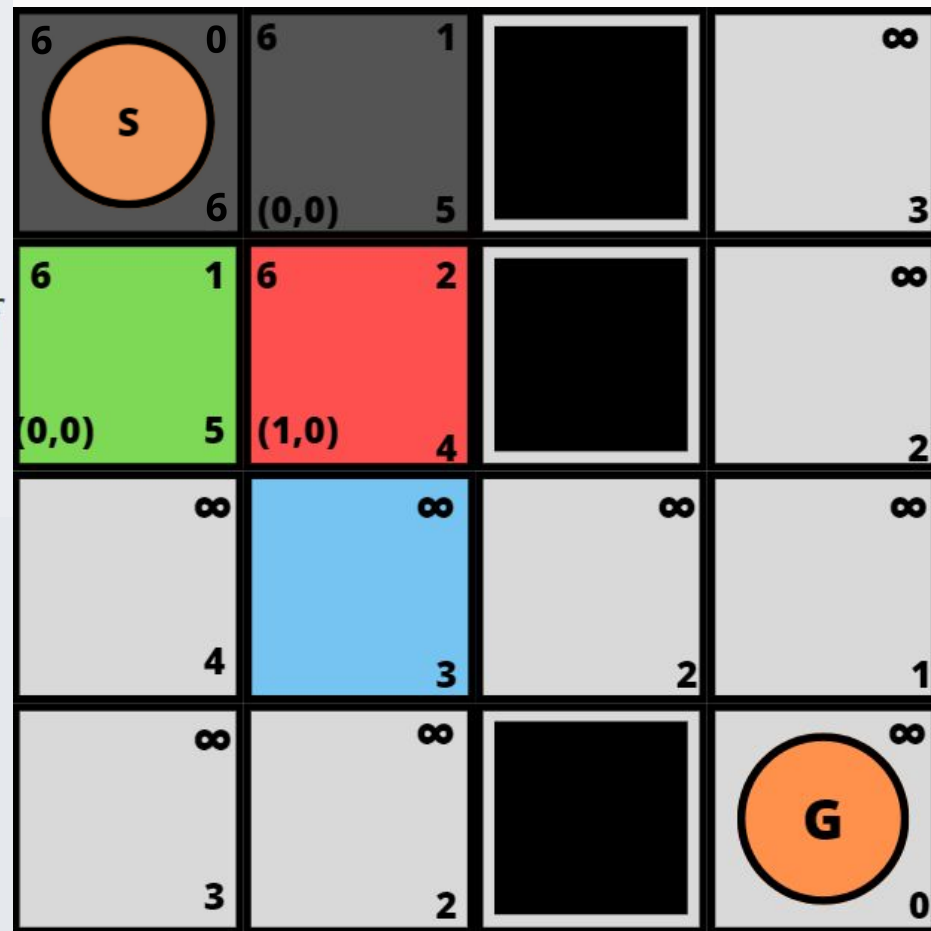
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



- 1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 while $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 if u es objetivo return u
- 7 for each $v \in Succ(u)$ do
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 if $cost_v \geq g(v)$ return
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 if $v \in Open$ then Reordenar $Open$
 - 7 else Insertar v en $Open$



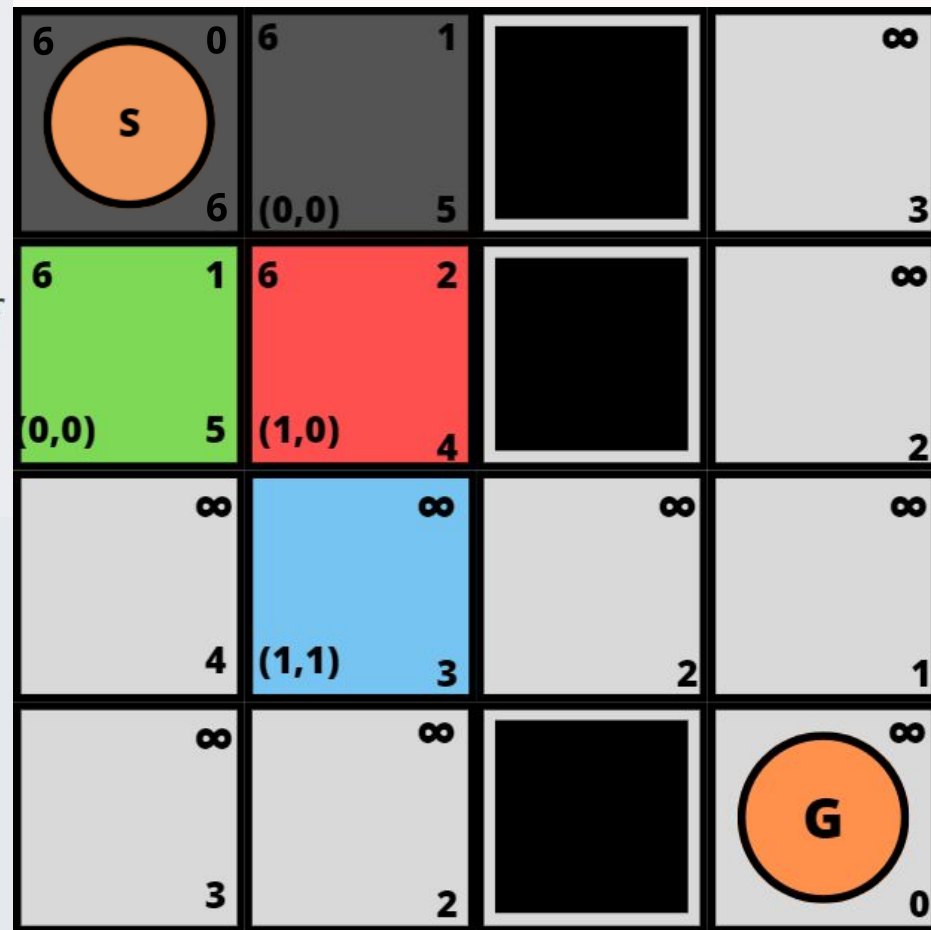
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
1    $cost_v = g(u) + c(u, v)$ 
2   if  $cost_v \geq g(v)$  return
3    $parent(v) \leftarrow u$ 
4    $g(v) \leftarrow cost_v$ 
5    $f(v) \leftarrow g(v) + h(v)$ 
6   if  $v \in Open$  then Reordenar  $Open$ 
7   else Insertar  $v$  en  $Open$ 

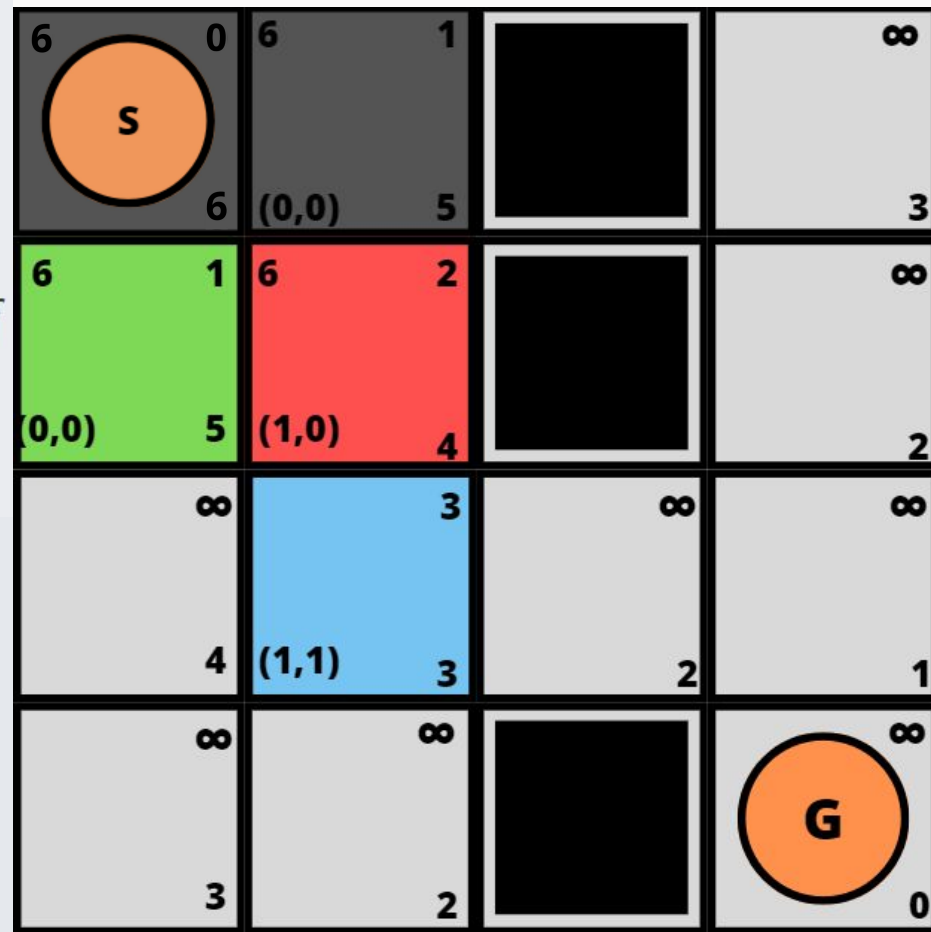
```



```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
1    $cost_v = g(u) + c(u, v)$ 
2   if  $cost_v \geq g(v)$  return
3    $parent(v) \leftarrow u$ 
4    $g(v) \leftarrow cost_v$ 
5    $f(v) \leftarrow g(v) + h(v)$ 
6   if  $v \in Open$  then Reordenar  $Open$ 
7   else Insertar  $v$  en  $Open$ 

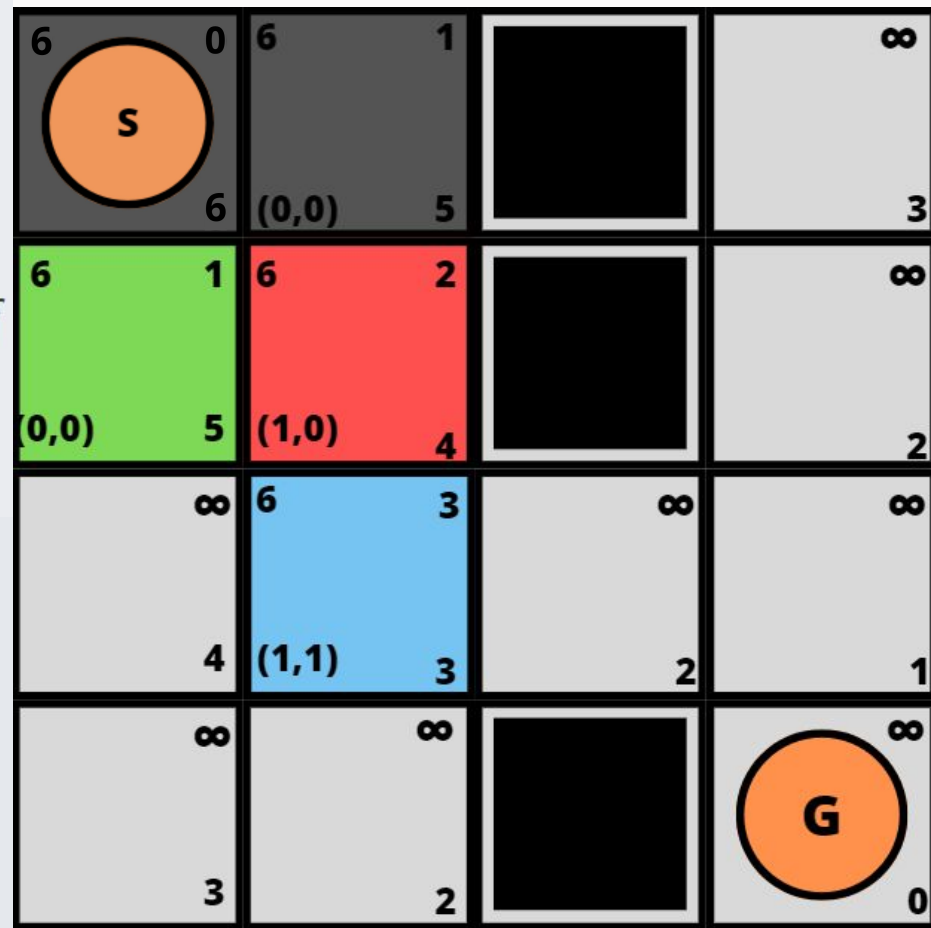
```



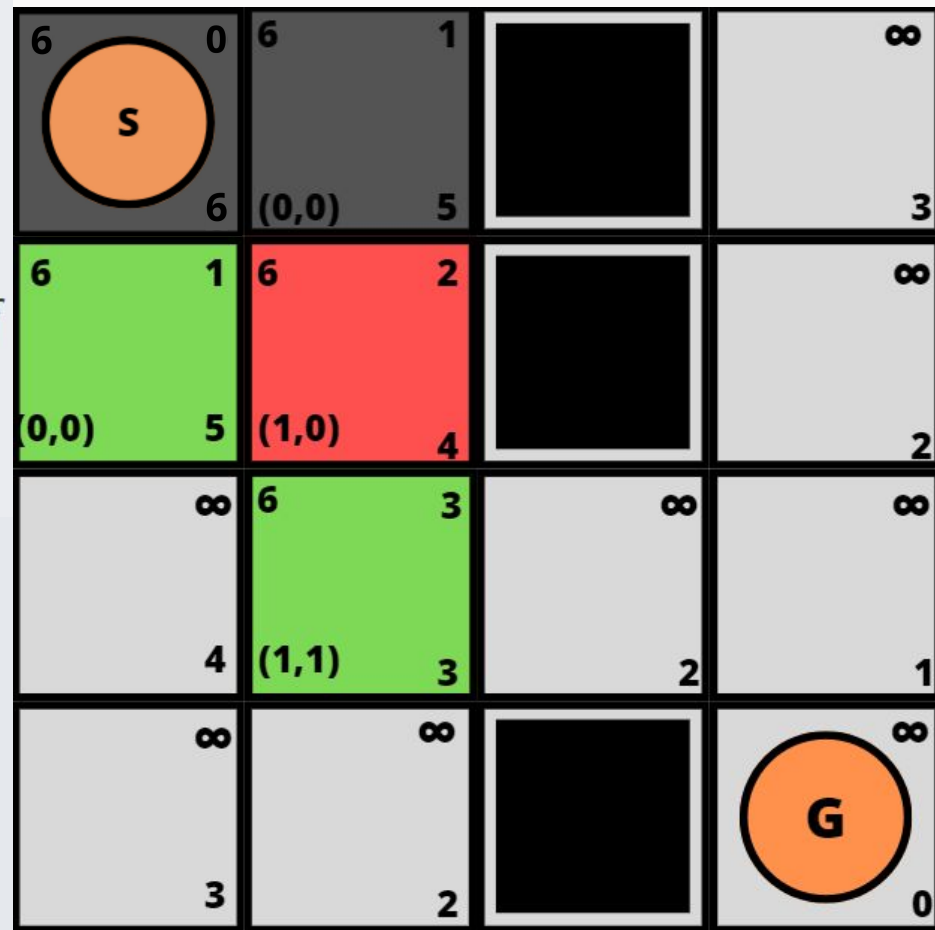

```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
1    $cost_v = g(u) + c(u, v)$ 
2   if  $cost_v \geq g(v)$  return
3    $parent(v) \leftarrow u$ 
4    $g(v) \leftarrow cost_v$ 
5    $f(v) \leftarrow g(v) + h(v)$ 
6   if  $v \in Open$  then Reordenar  $Open$ 
7   else Insertar  $v$  en  $Open$ 

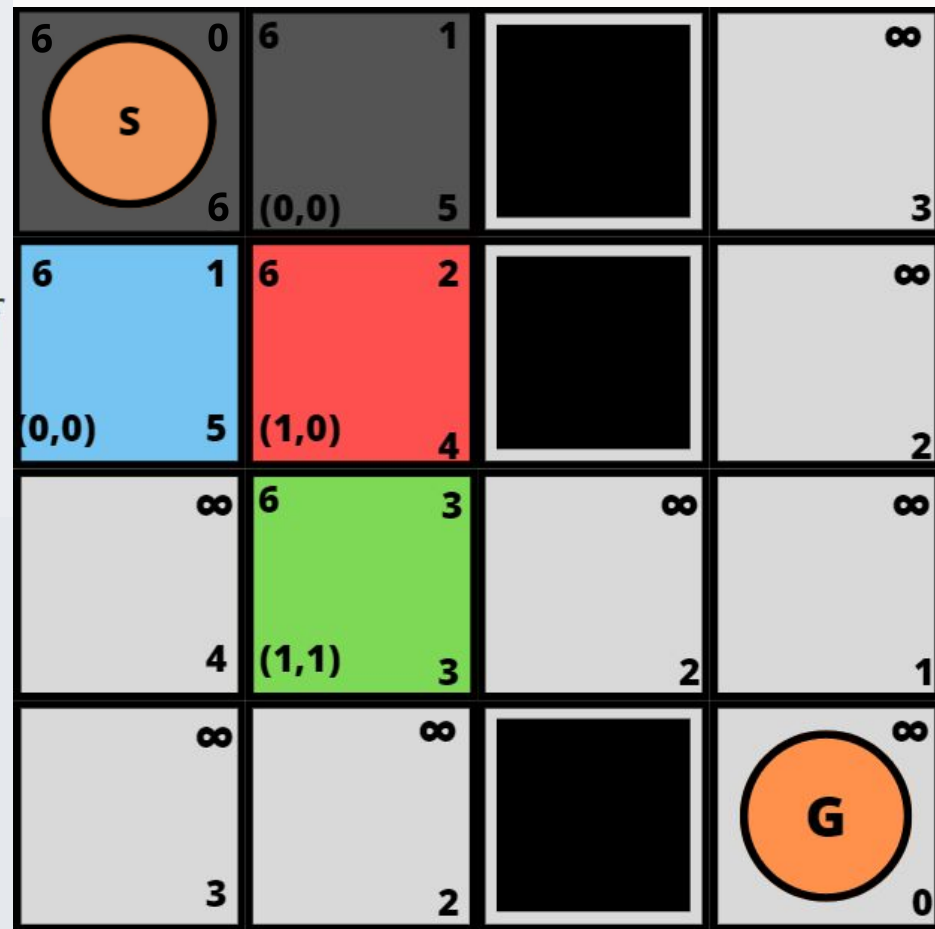
```



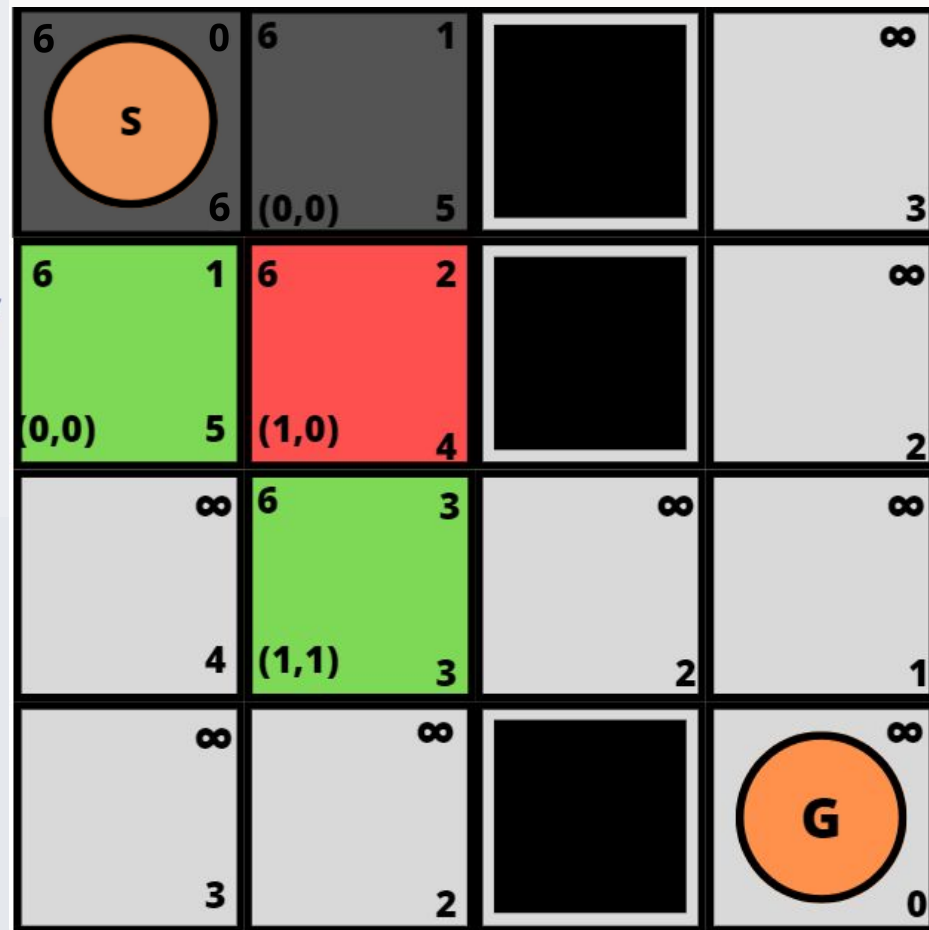
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



- 1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 while $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 if u es objetivo return u
- 7 for each $v \in Succ(u)$ do
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 if $cost_v \geq g(v)$ return
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 if $v \in Open$ then Reordenar $Open$
 - 7 else Insertar v en $Open$



```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 

```

2 $Open \leftarrow \{s_0\}$

3 $g(s_0) \leftarrow 0; f(s_0) \leftarrow h(s_0)$

```
4 while  $Open \neq \emptyset$ 
```

5 Extrae un u desde $Open$ con menor valor- f

```
6 if  $u$  es objetivo return  $u$ 
```

```

7 for each  $v \in Succ(u)$  do

```

$$\mathbf{1} \quad cost_v = g(u) + c(u, v)$$

2 if $cost_v \geq g(v)$ return

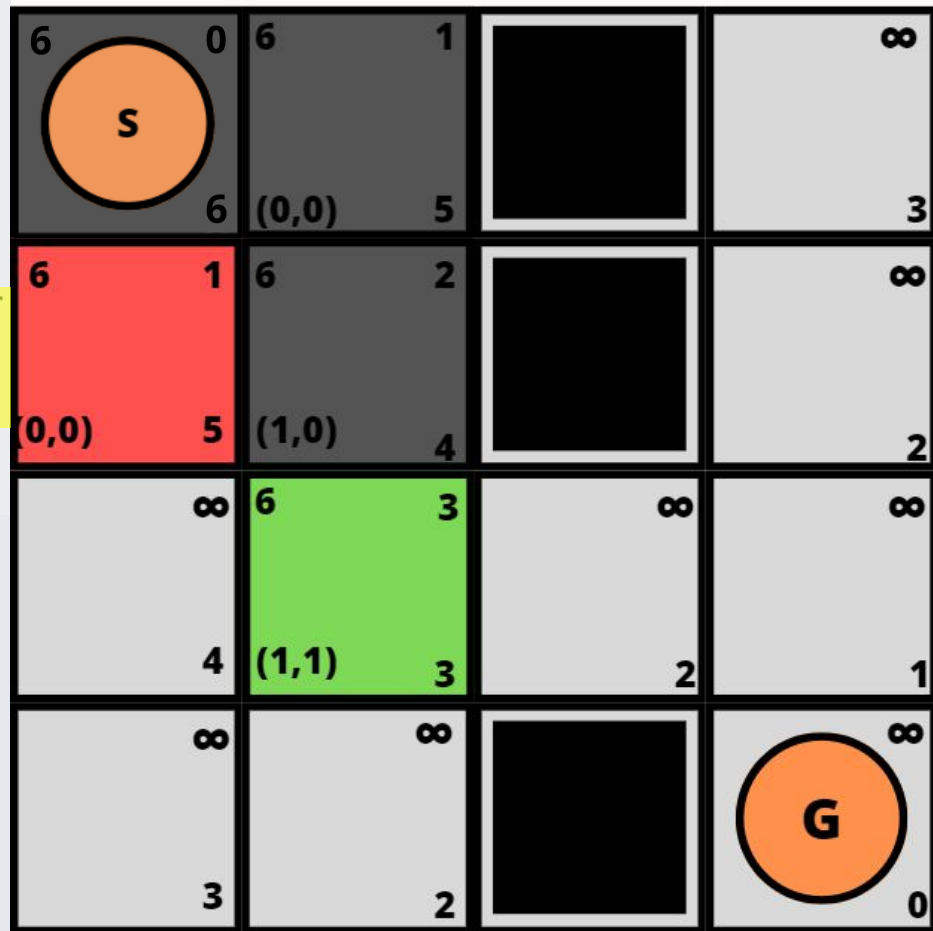
3 $parent(v) \leftarrow u$

4 $g(v) \leftarrow cost_v$

5 $f(v) \leftarrow g(v) + h(v)$

6 if $v \in Open$ then Reordenar $Open$

7 **else** Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**

- 1 $cost_v = g(u) + c(u, v)$

- 2 **if** $cost_v \geq g(v)$ **return**

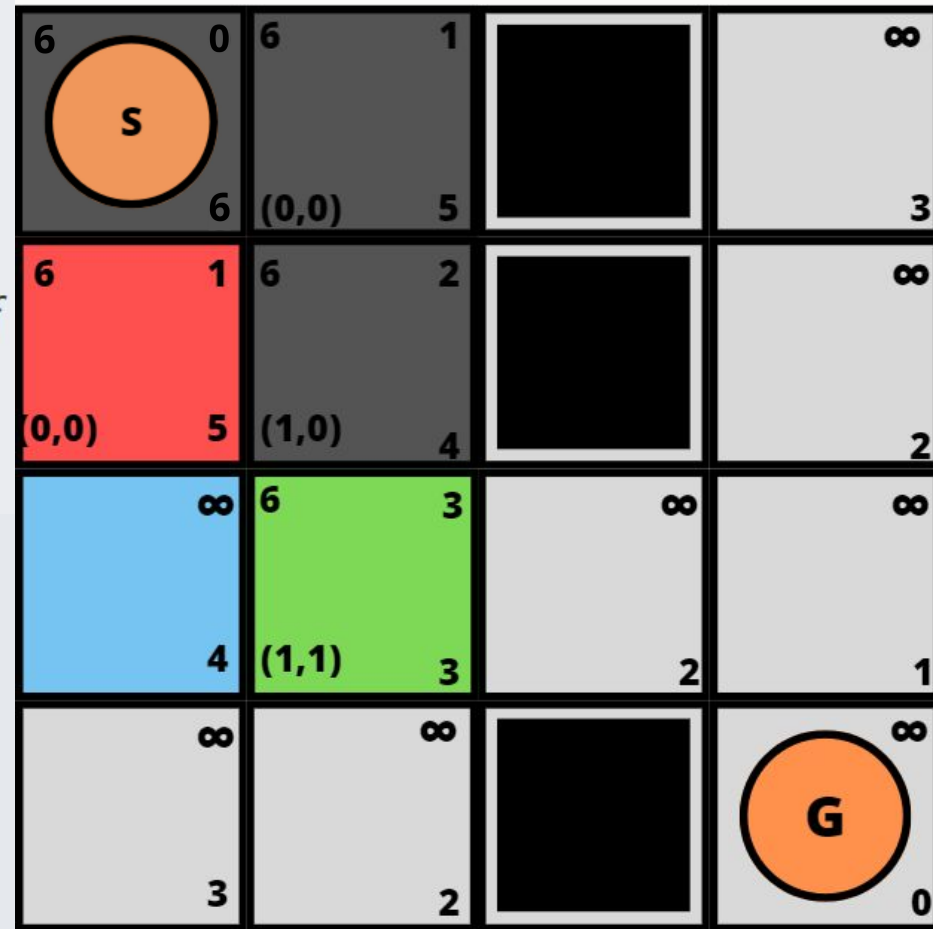
- 3 $parent(v) \leftarrow u$

- 4 $g(v) \leftarrow cost_v$

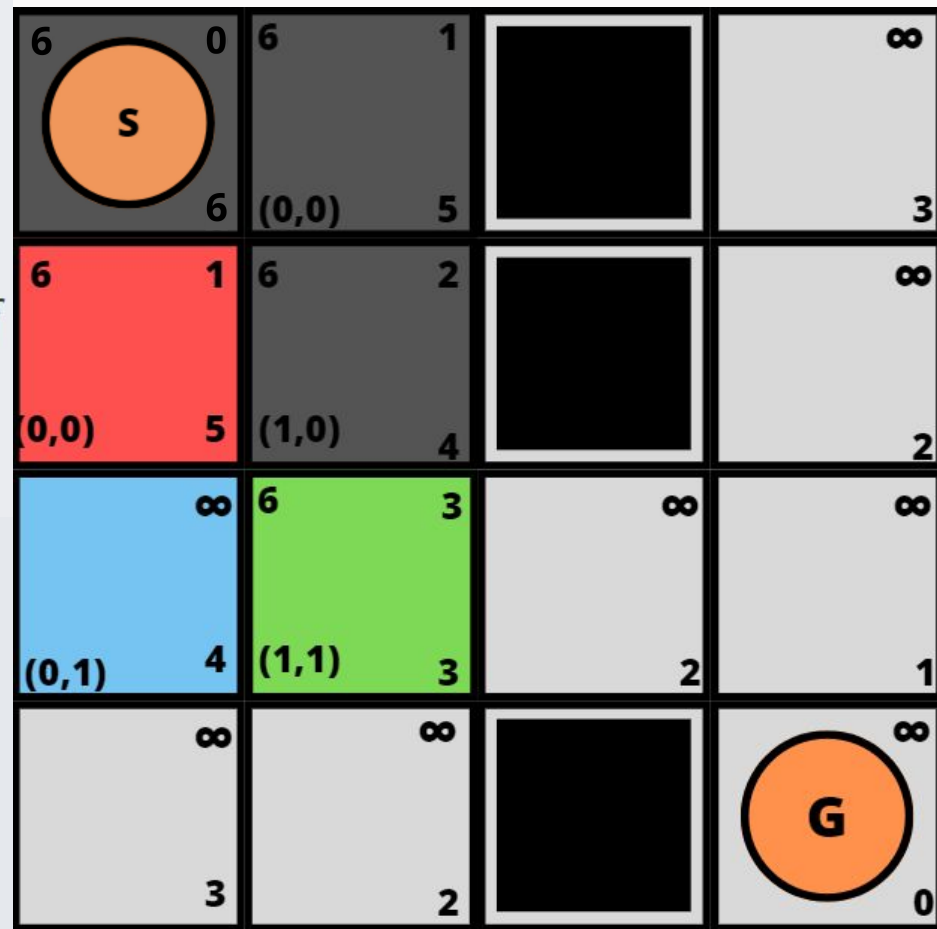
- 5 $f(v) \leftarrow g(v) + h(v)$

- 6 **if** $v \in Open$ **then** Reordenar $Open$

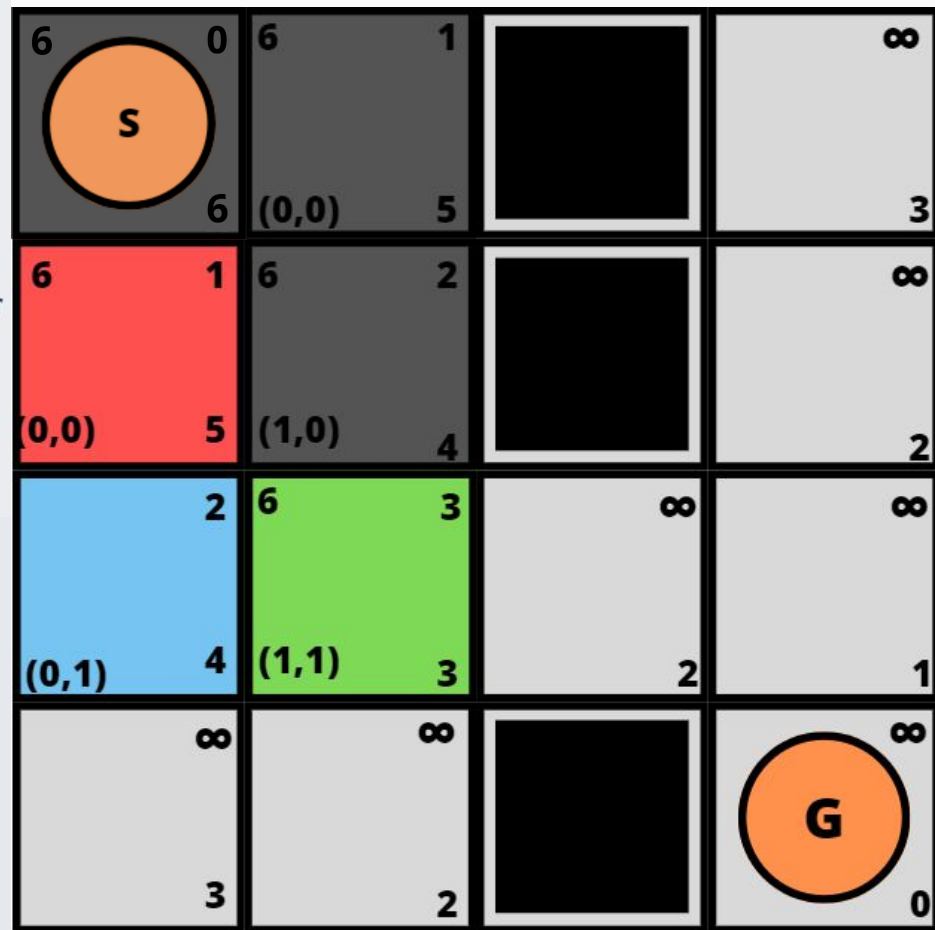
- 7 **else** Insertar v en $Open$



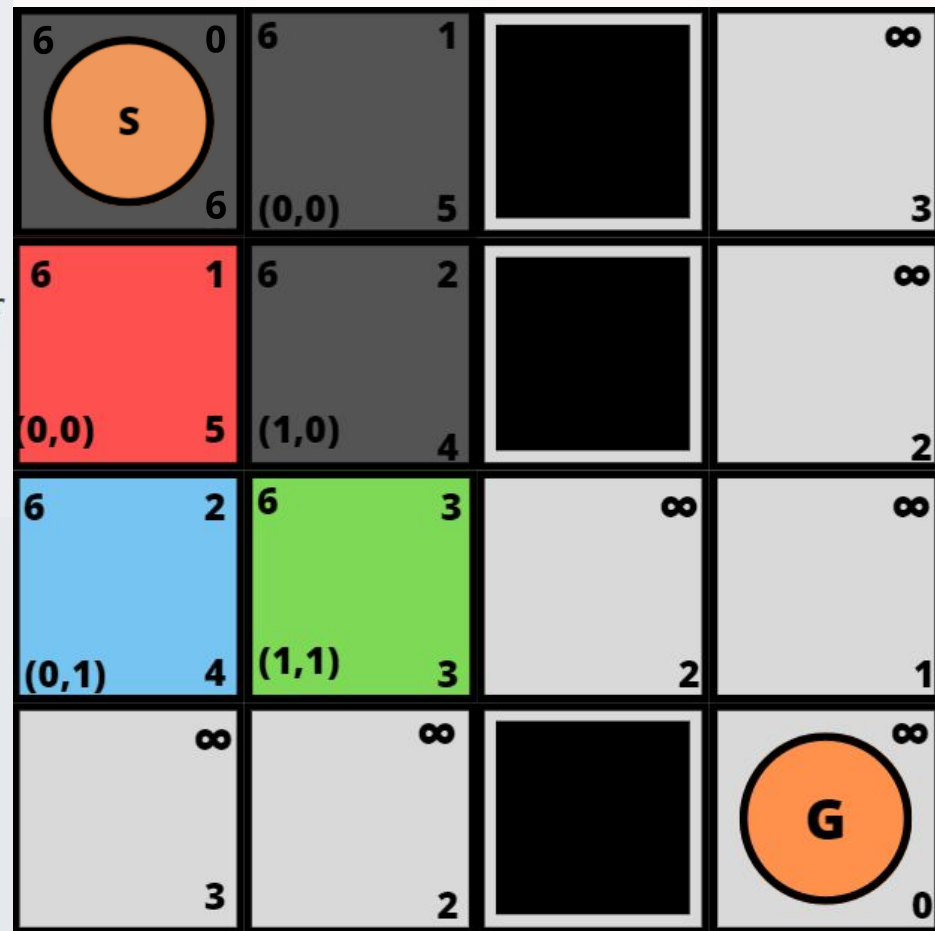
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



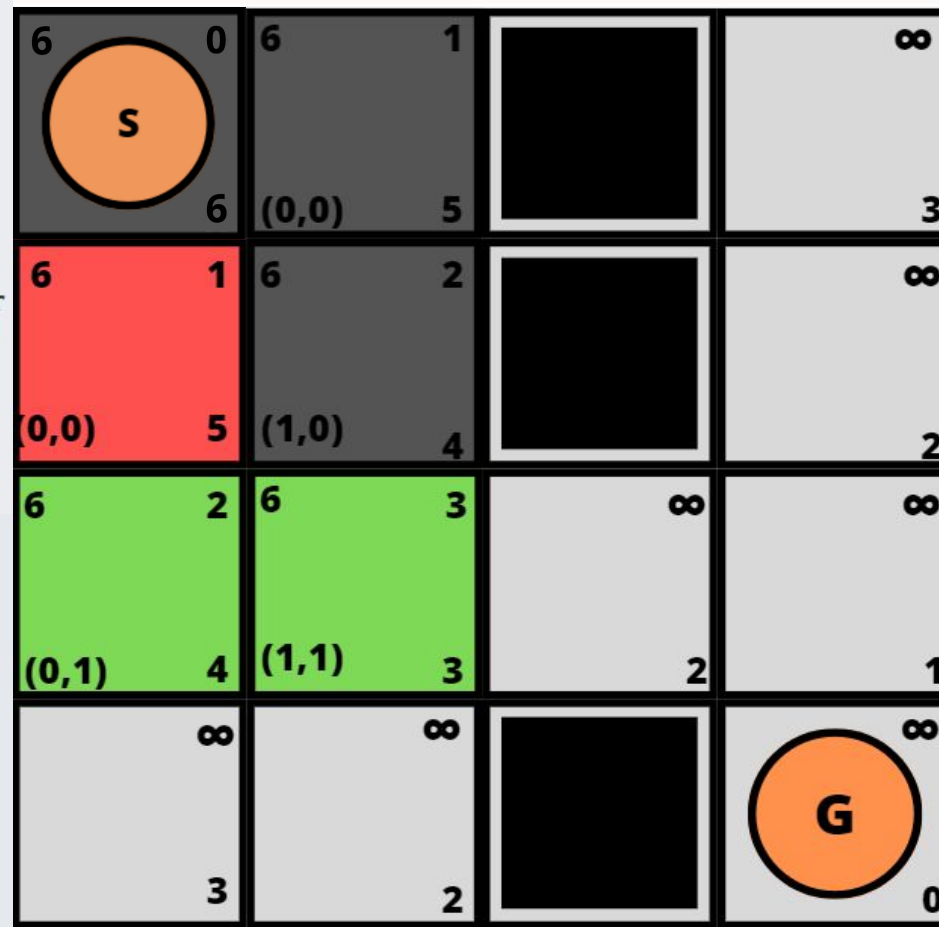
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
8     1  $cost_v = g(u) + c(u, v)$ 
9     2 if  $cost_v \geq g(v)$  return
10    3  $parent(v) \leftarrow u$ 
11    4  $g(v) \leftarrow cost_v$ 
12    5  $f(v) \leftarrow g(v) + h(v)$ 
13    6 if  $v \in Open$  then Reordenar  $Open$ 
14    7 else Insertar  $v$  en  $Open$ 

```



1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$

2 $Open \leftarrow \{s_0\}$

3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$

4 while $Open \neq \emptyset$

5 Extrae un u desde $Open$ con menor valor- f

6 if u es objetivo return u

7 for each $v \in Succ(u)$ do

1 $cost_v = g(u) + c(u, v)$

2 if $cost_v \geq g(v)$ return

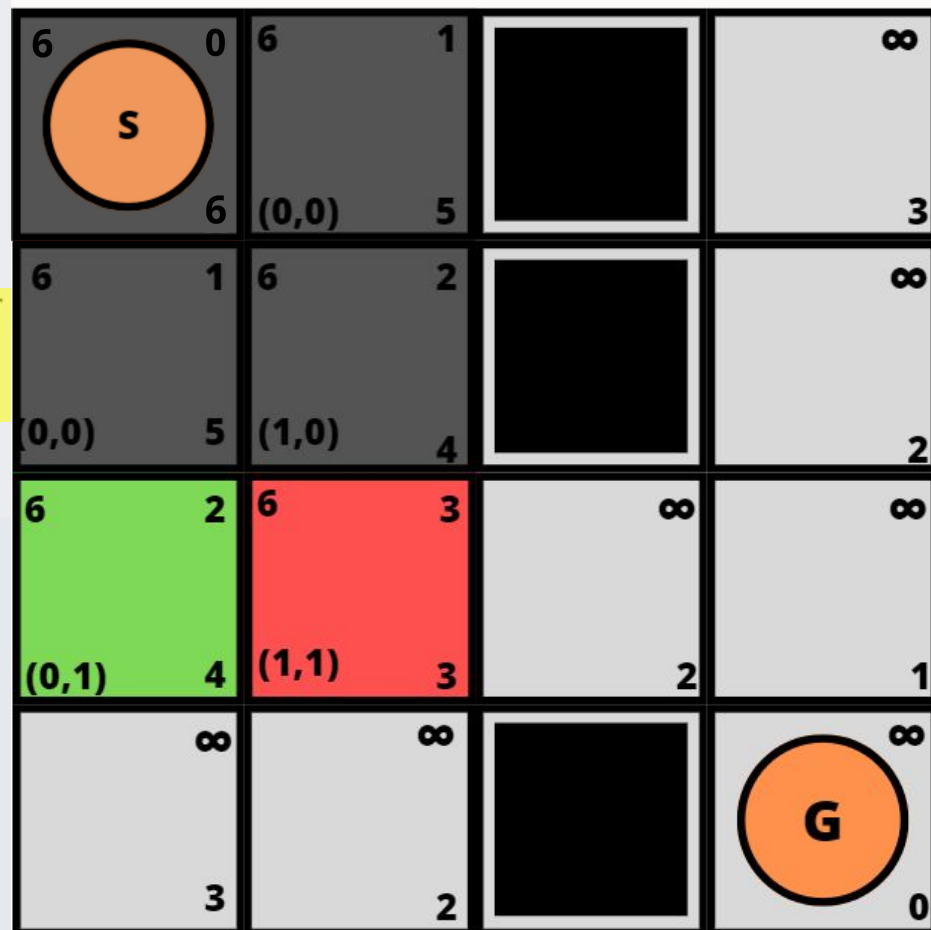
3 $parent(v) \leftarrow u$

4 $g(v) \leftarrow cost_v$

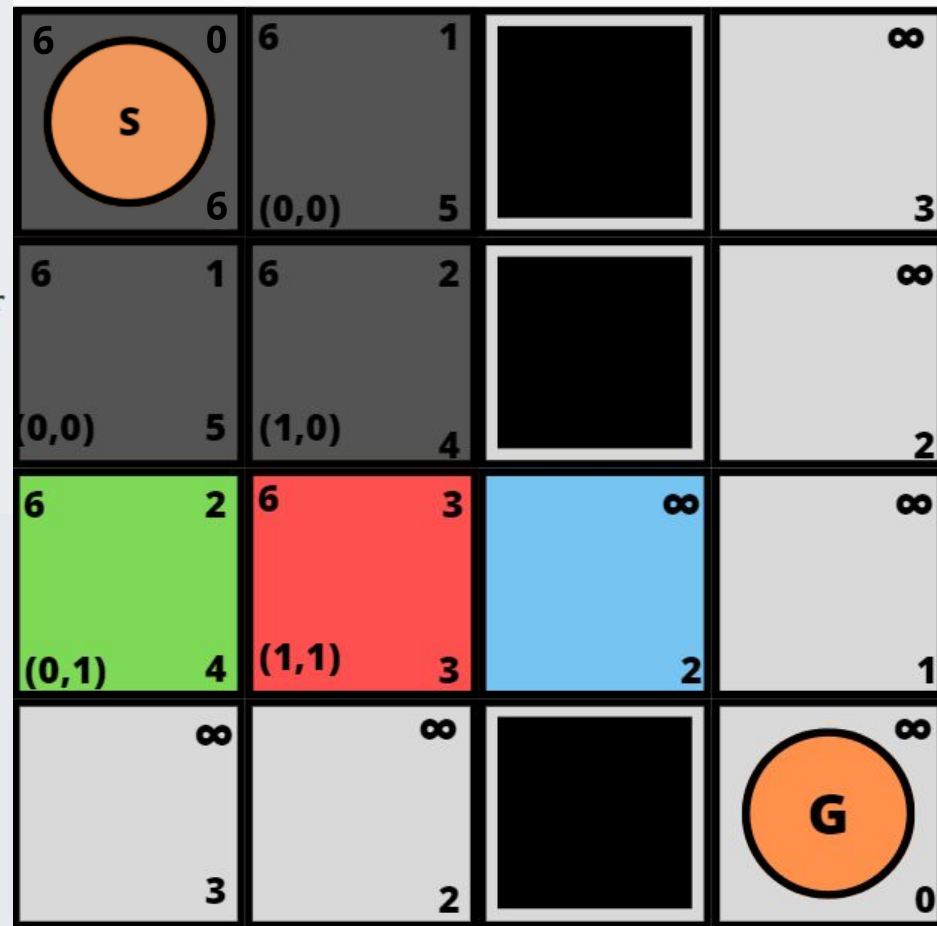
5 $f(v) \leftarrow g(v) + h(v)$

6 if $v \in Open$ then Reordenar $Open$

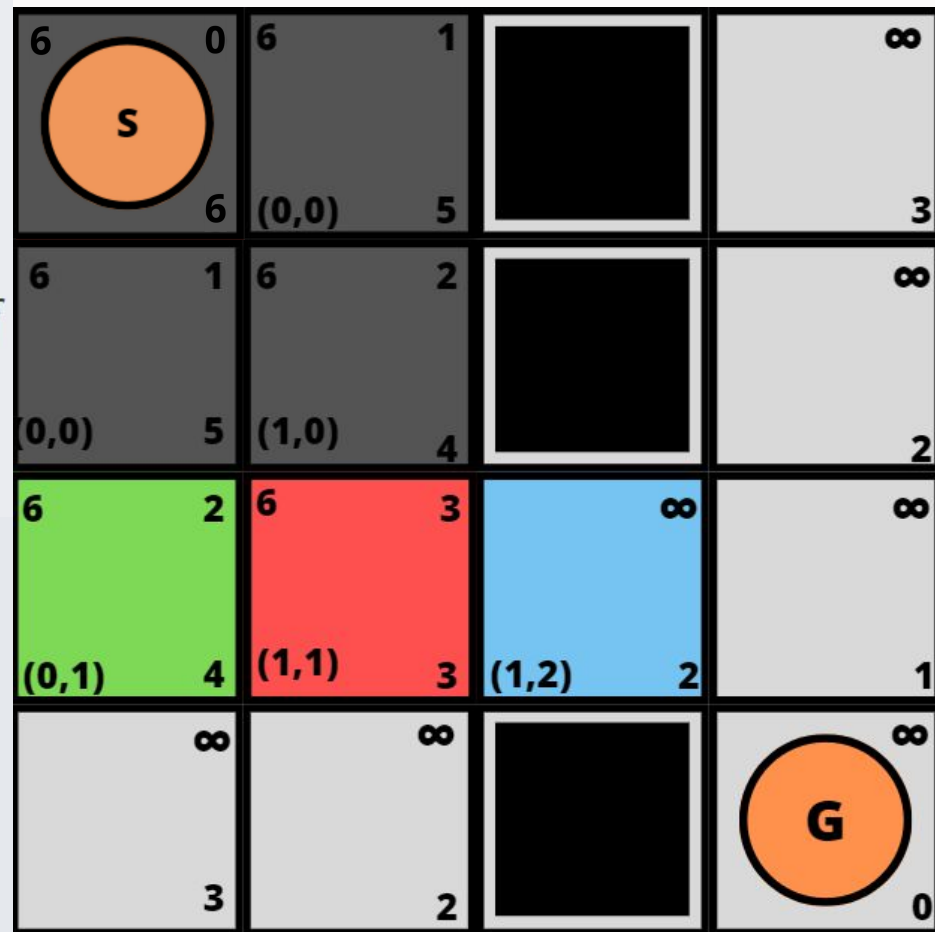
7 else Insertar v en $Open$



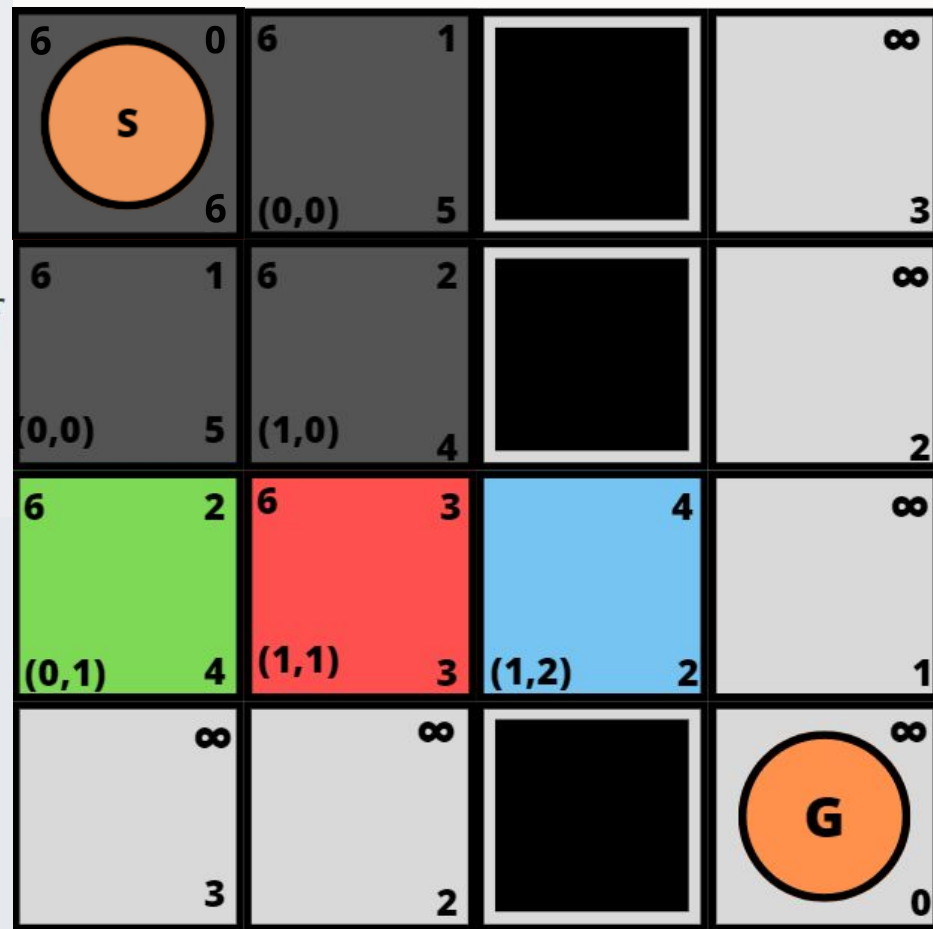
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



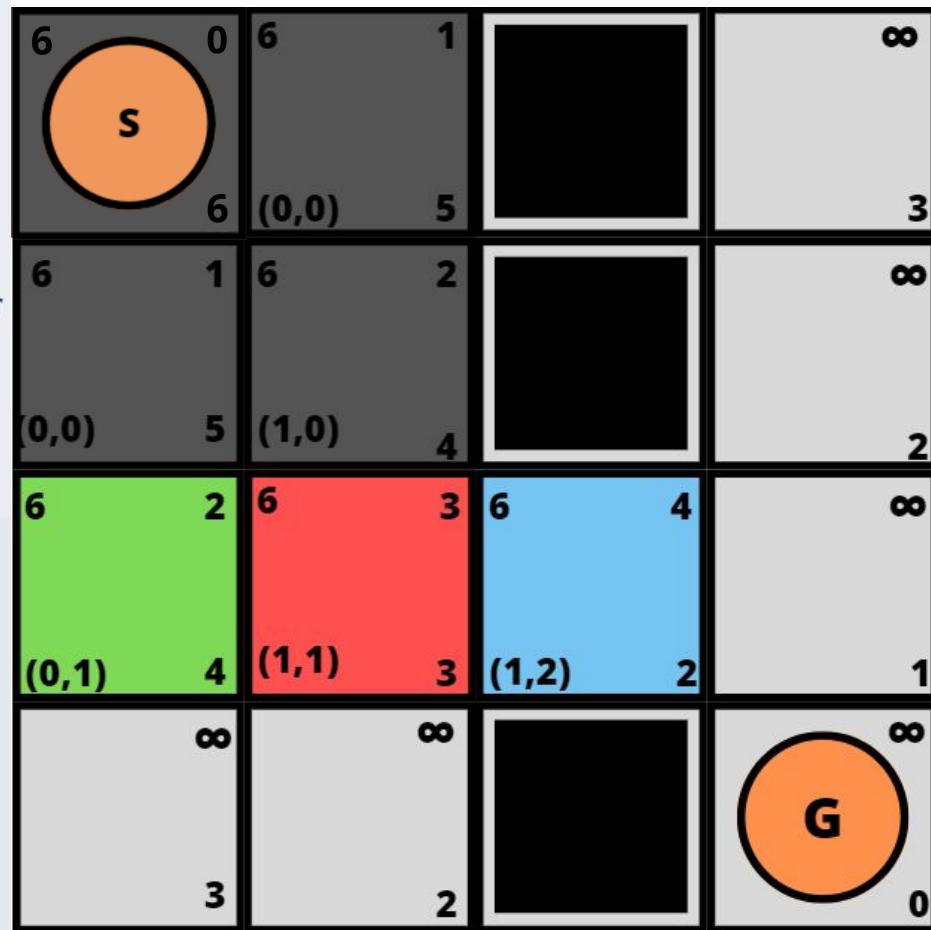
- 1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 while $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 if u es objetivo return u
- 7 for each $v \in Succ(u)$ do
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 if $cost_v \geq g(v)$ return
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 if $v \in Open$ then Reordenar $Open$
 - 7 else Insertar v en $Open$



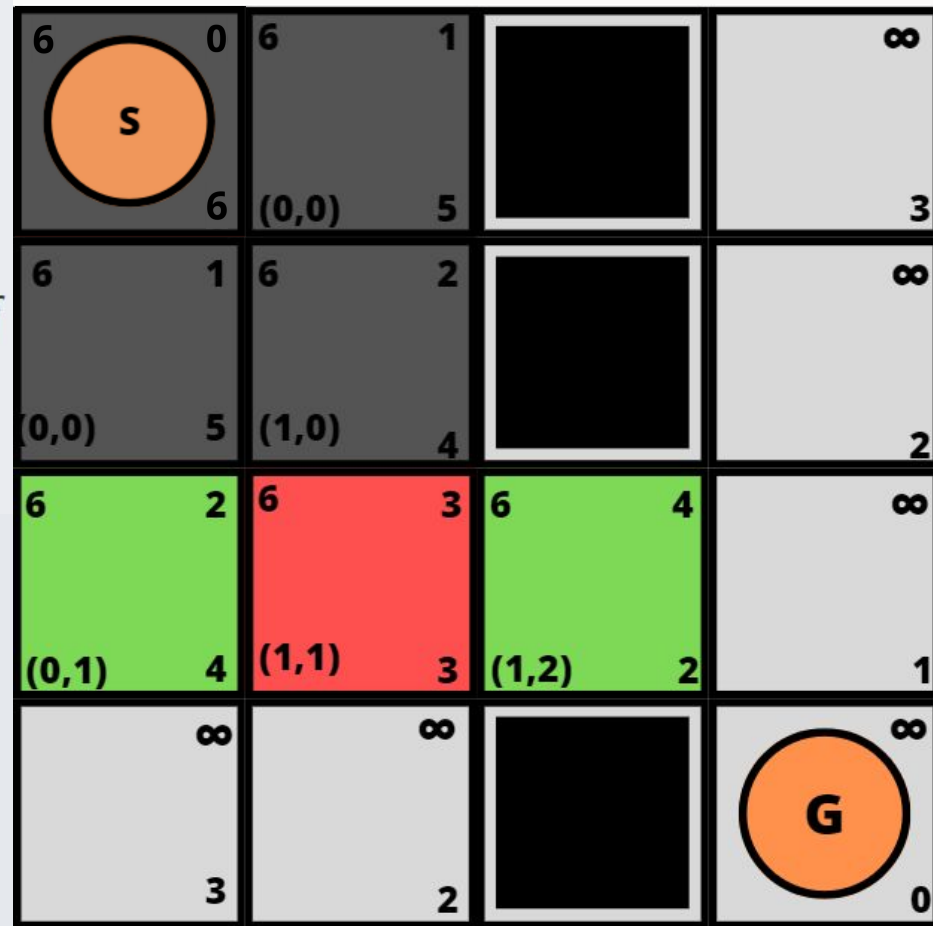
```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
1    $cost_v = g(u) + c(u, v)$ 
2   if  $cost_v \geq g(v)$  return
3    $parent(v) \leftarrow u$ 
4    $g(v) \leftarrow cost_v$ 
5    $f(v) \leftarrow g(v) + h(v)$ 
6   if  $v \in Open$  then Reordenar  $Open$ 
7   else Insertar  $v$  en  $Open$ 

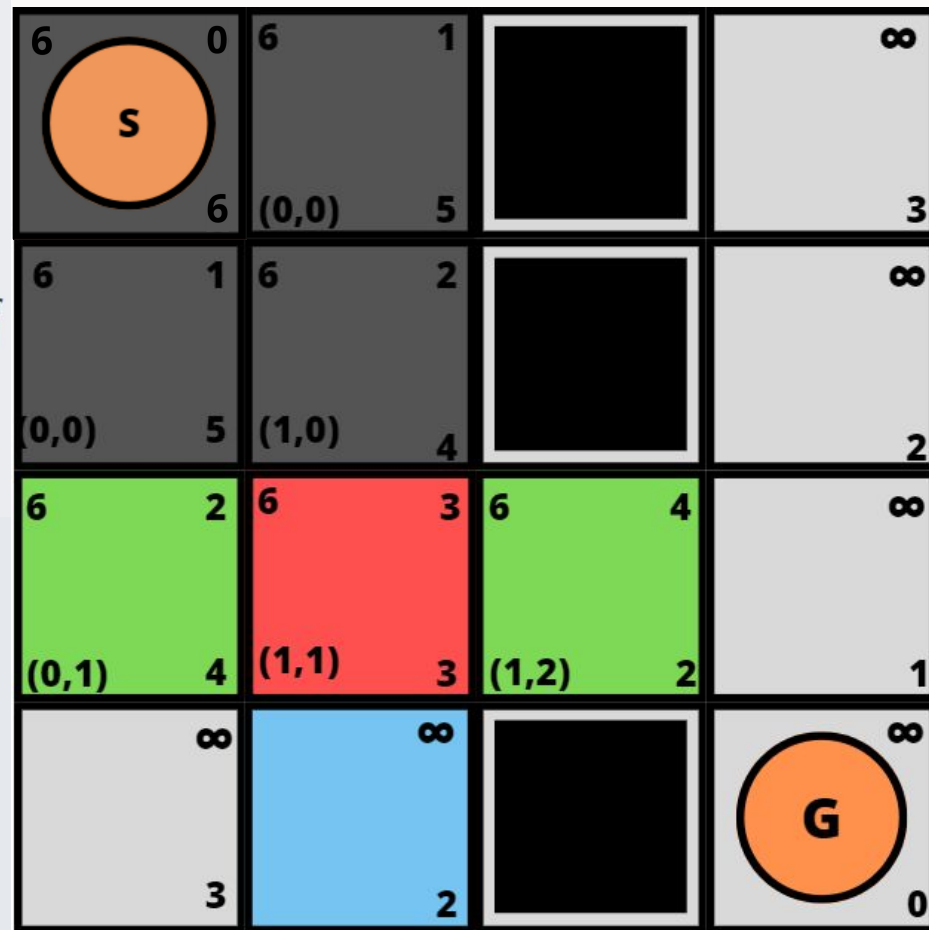
```



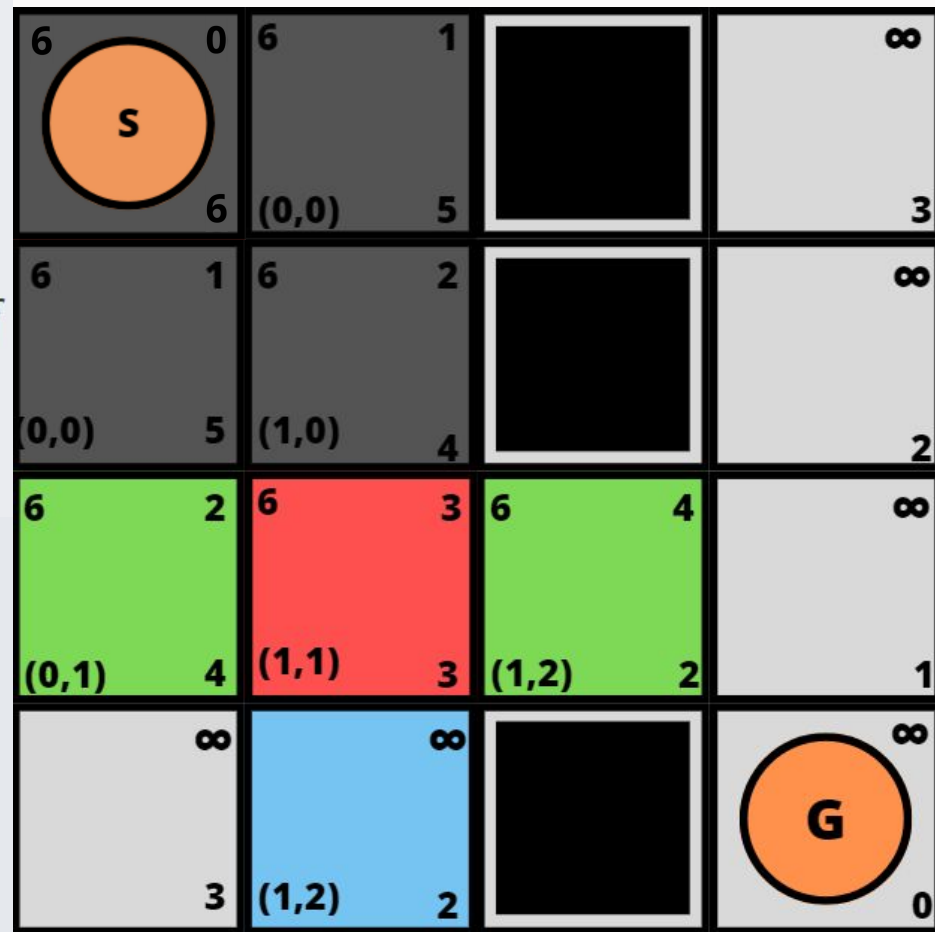
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



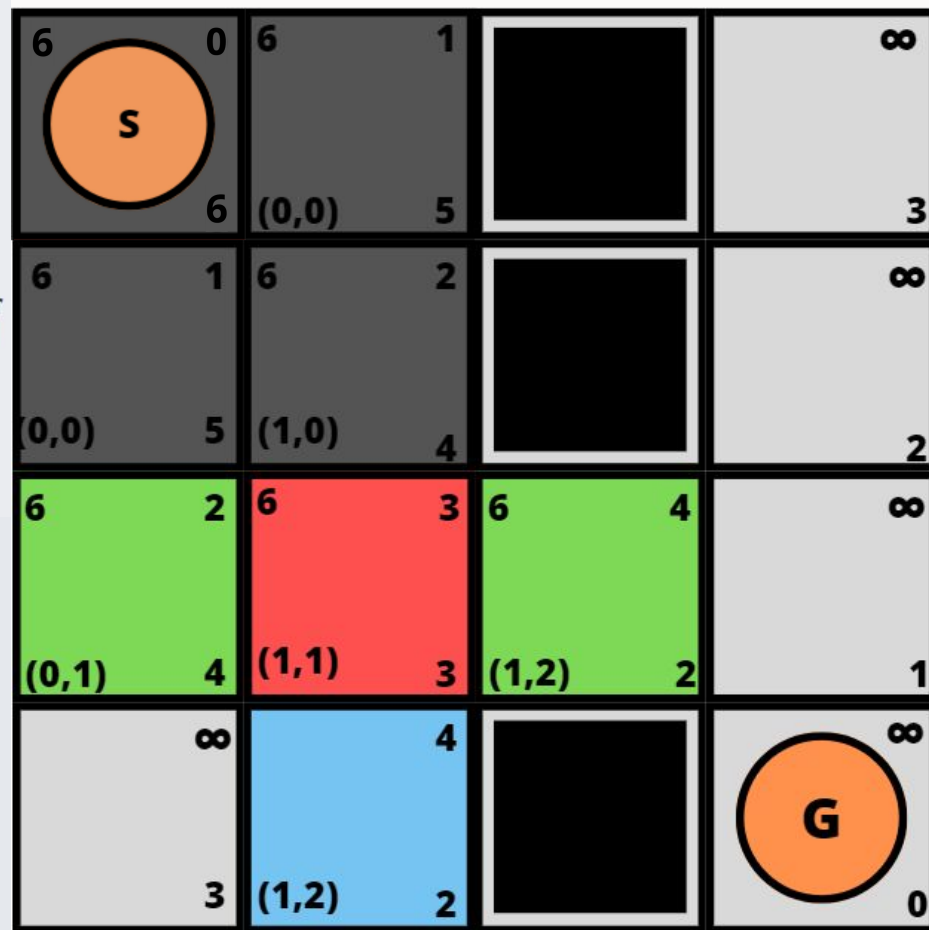
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
8     1  $cost_v = g(u) + c(u, v)$ 
9     2 if  $cost_v \geq g(v)$  return
10    3  $parent(v) \leftarrow u$ 
11    4  $g(v) \leftarrow cost_v$ 
12    5  $f(v) \leftarrow g(v) + h(v)$ 
13    6 if  $v \in Open$  then Reordenar  $Open$ 
14    7 else Insertar  $v$  en  $Open$ 

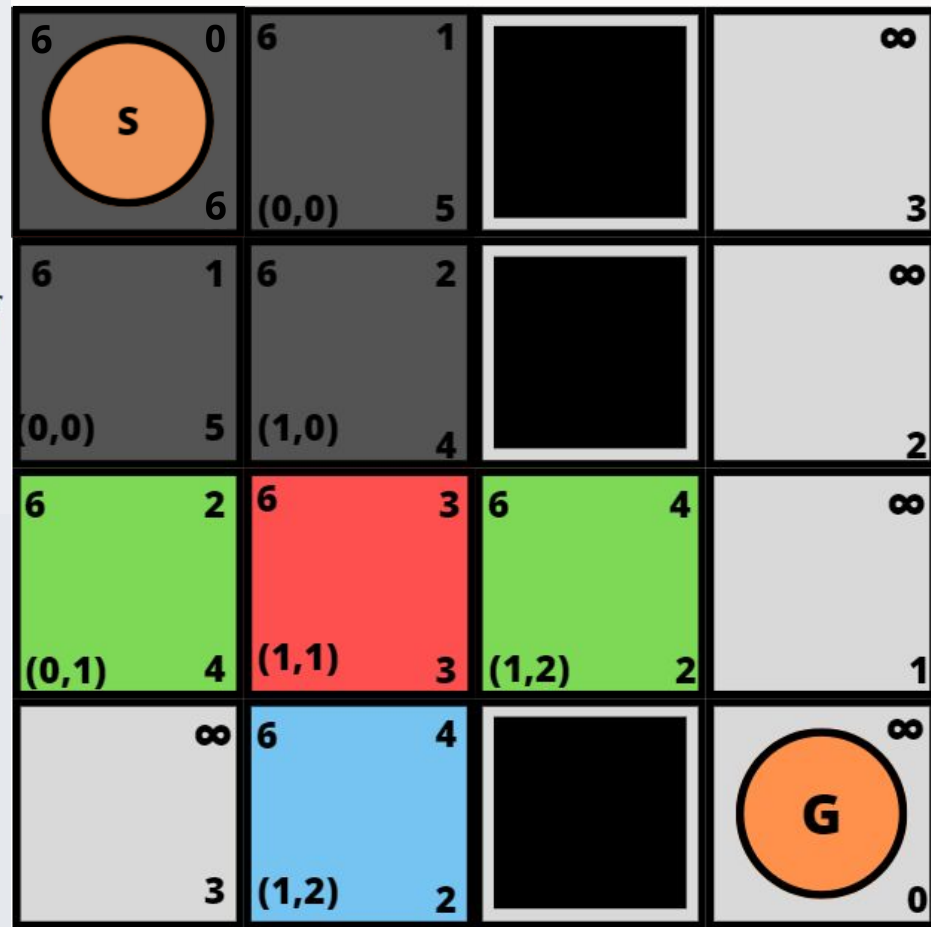
```



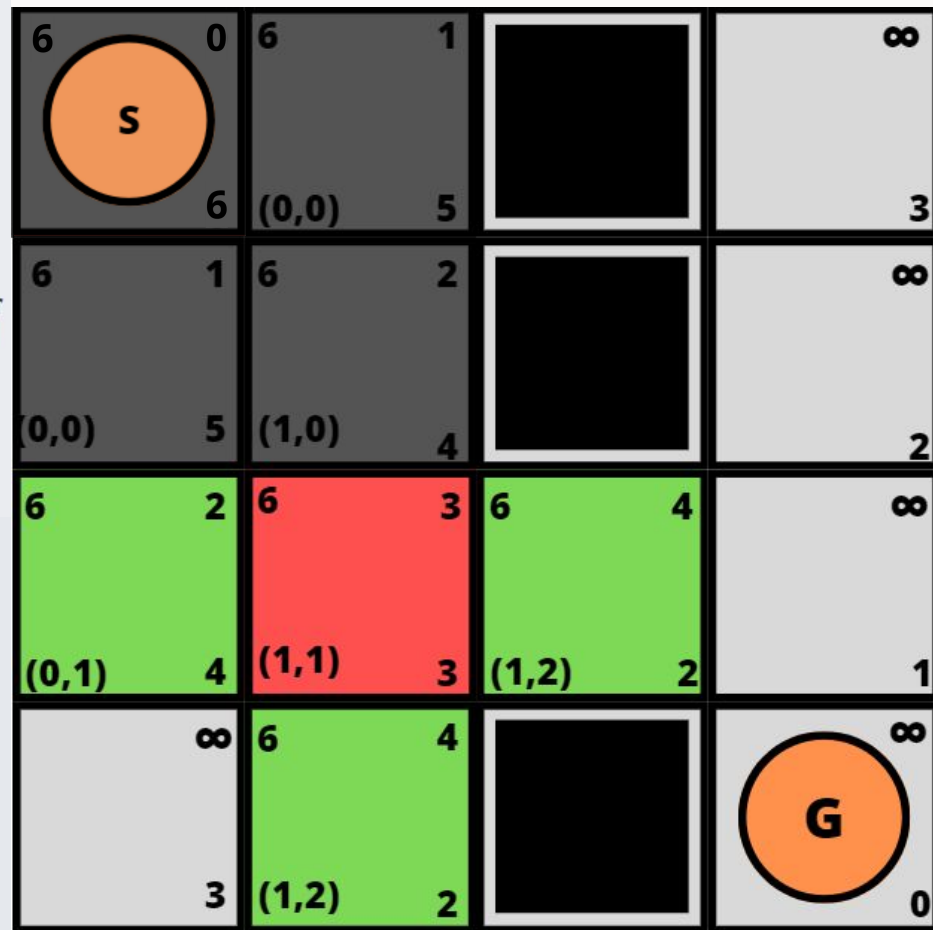

```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
8     1  $cost_v = g(u) + c(u, v)$ 
9     2 if  $cost_v \geq g(v)$  return
10    3  $parent(v) \leftarrow u$ 
11    4  $g(v) \leftarrow cost_v$ 
12    5  $f(v) \leftarrow g(v) + h(v)$ 
13    6 if  $v \in Open$  then Reordenar  $Open$ 
14    7 else Insertar  $v$  en  $Open$ 

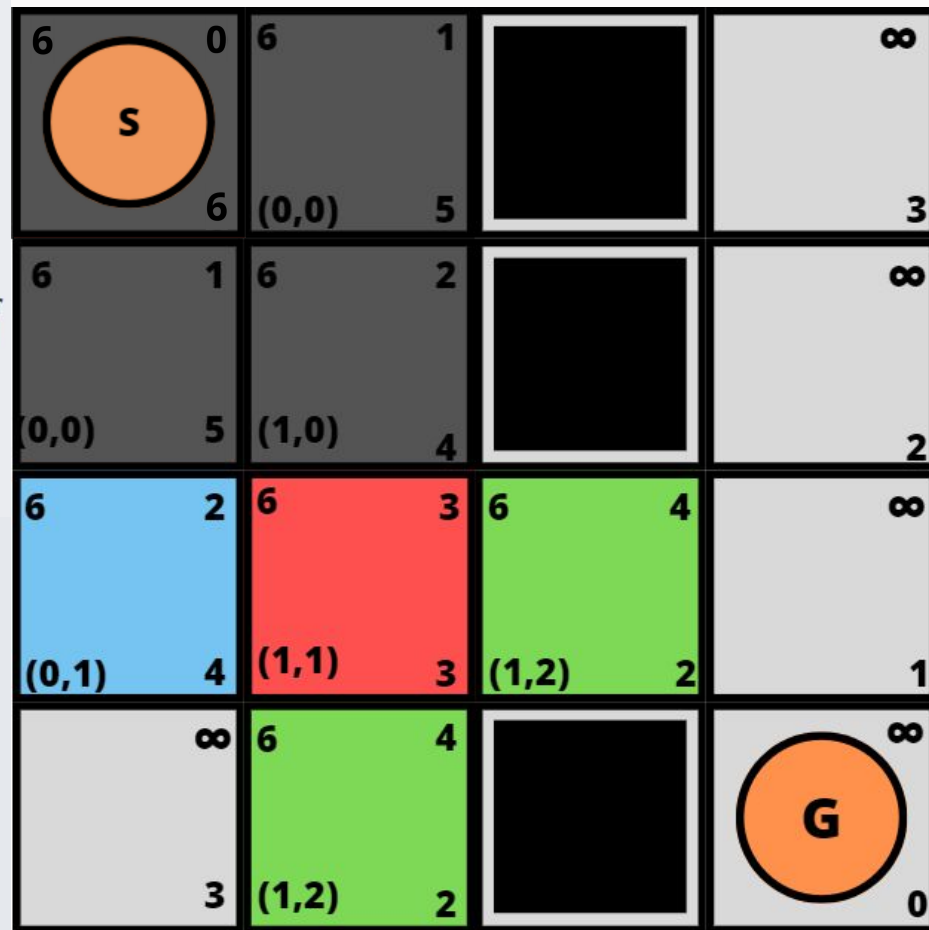
```



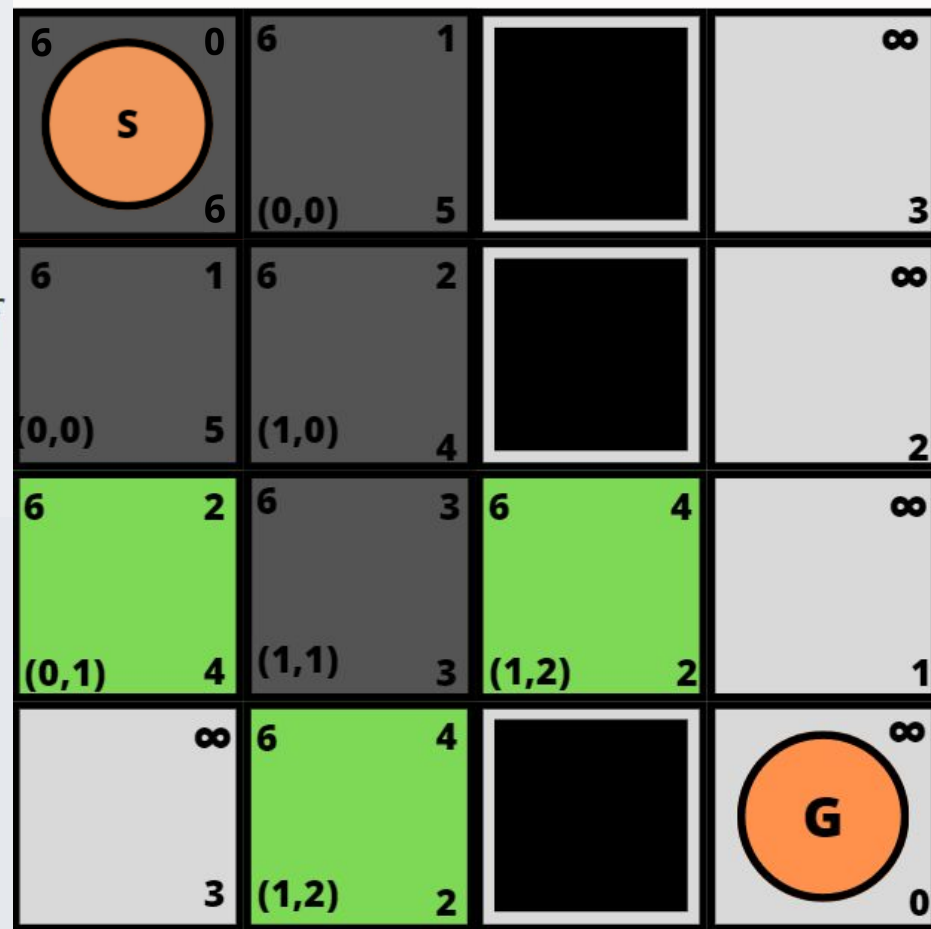
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$

2 $Open \leftarrow \{s_0\}$

3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$

4 while $Open \neq \emptyset$

5 Extrae un u desde $Open$ con menor valor- f

6 if u es objetivo return u

7 for each $v \in Succ(u)$ do

1 $cost_v = g(u) + c(u, v)$

2 if $cost_v \geq g(v)$ return

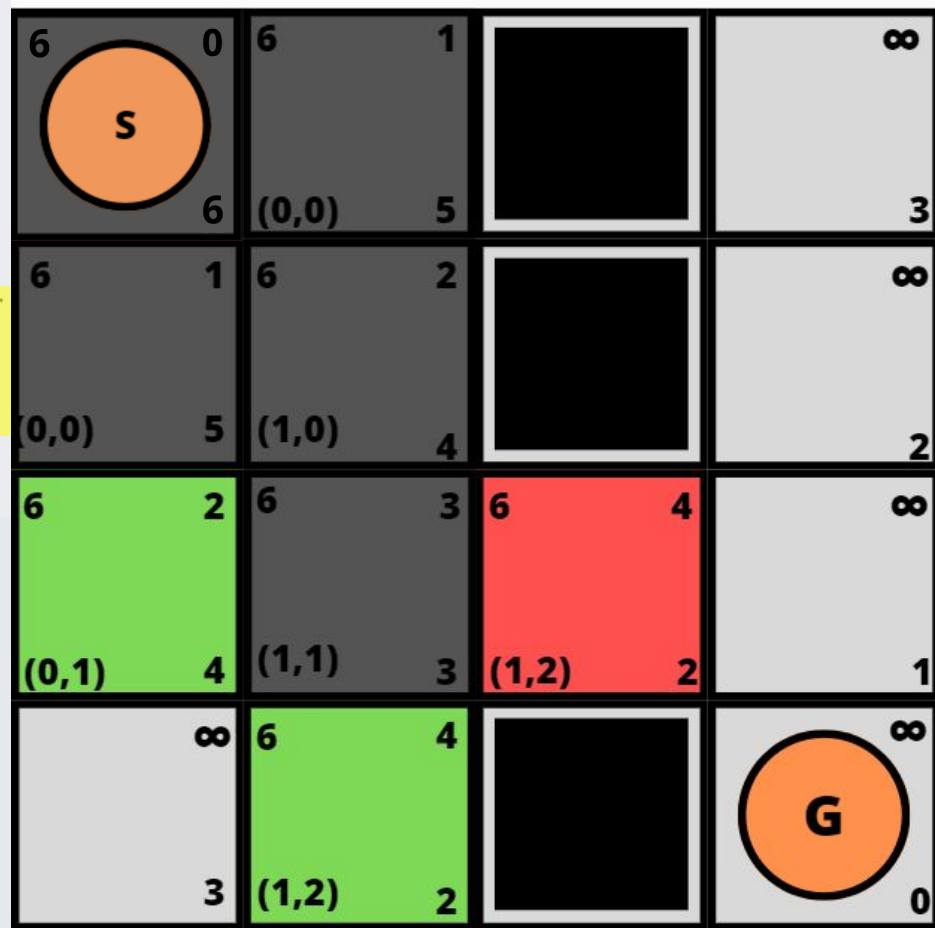
3 $parent(v) \leftarrow u$

4 $g(v) \leftarrow cost_v$

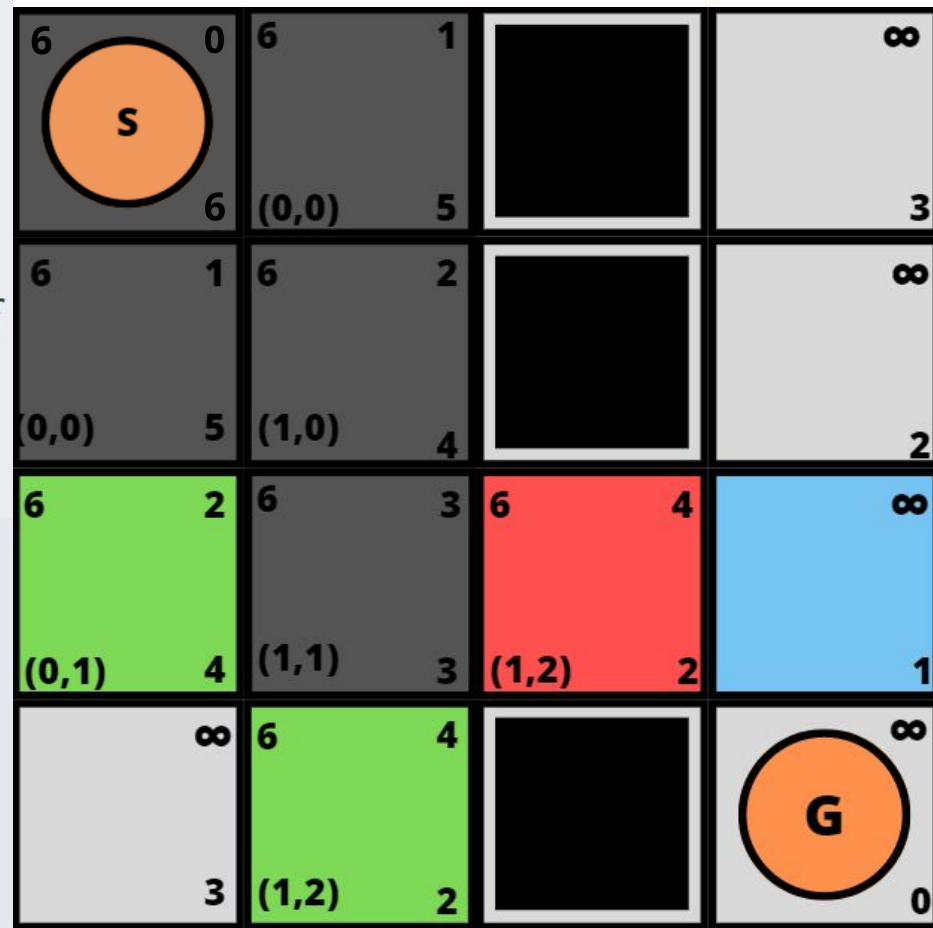
5 $f(v) \leftarrow g(v) + h(v)$

6 if $v \in Open$ then Reordenar $Open$

7 else Insertar v en $Open$



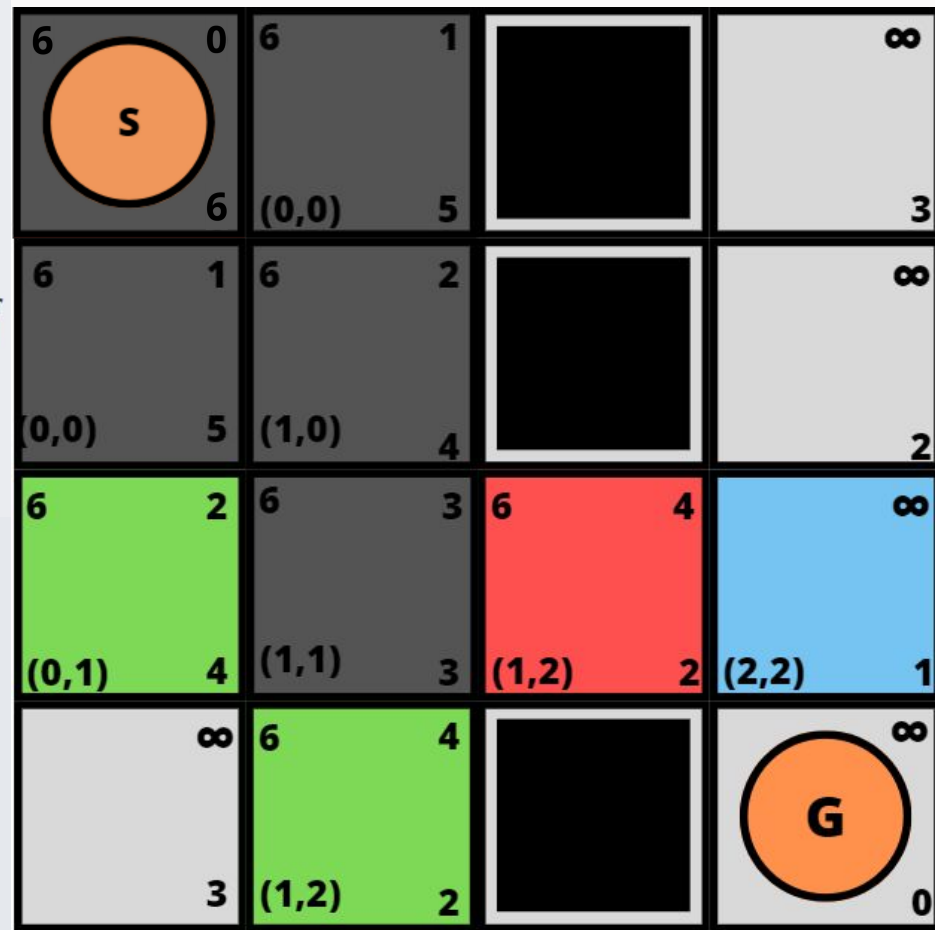
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



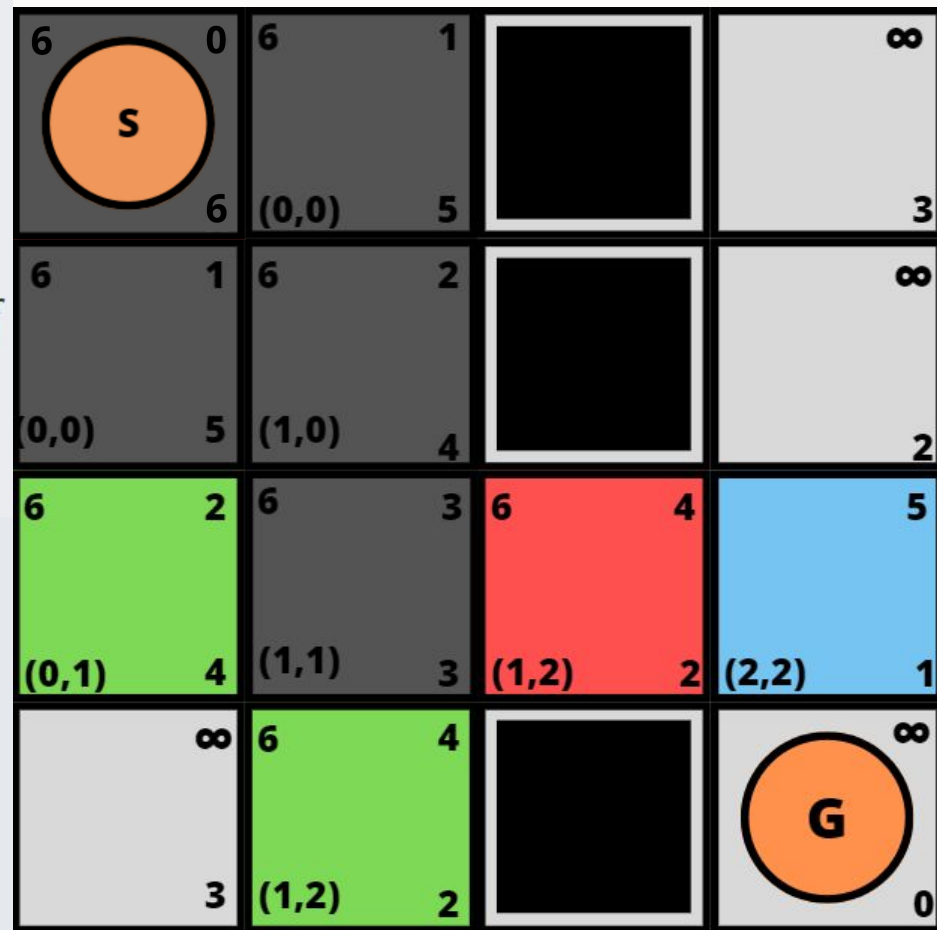
```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
8     1  $cost_v = g(u) + c(u, v)$ 
9     2 if  $cost_v \geq g(v)$  return
10    3  $parent(v) \leftarrow u$ 
11    4  $g(v) \leftarrow cost_v$ 
12    5  $f(v) \leftarrow g(v) + h(v)$ 
13    6 if  $v \in Open$  then Reordenar  $Open$ 
14    7 else Insertar  $v$  en  $Open$ 

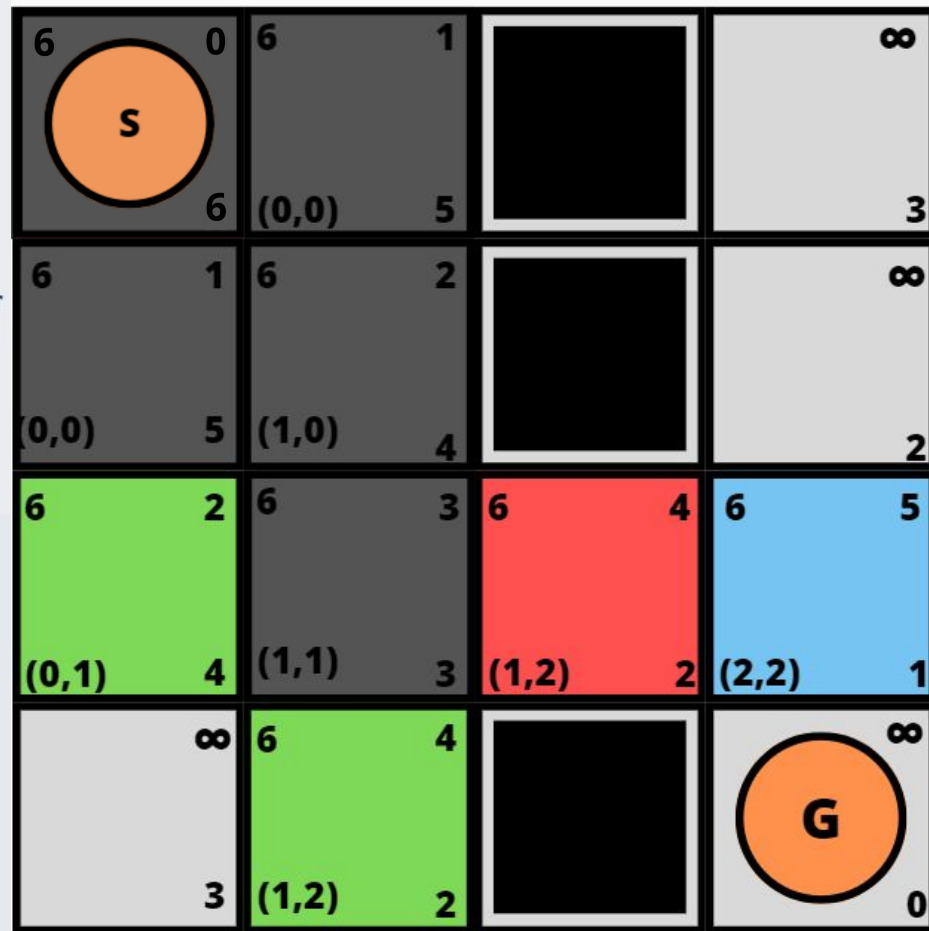
```



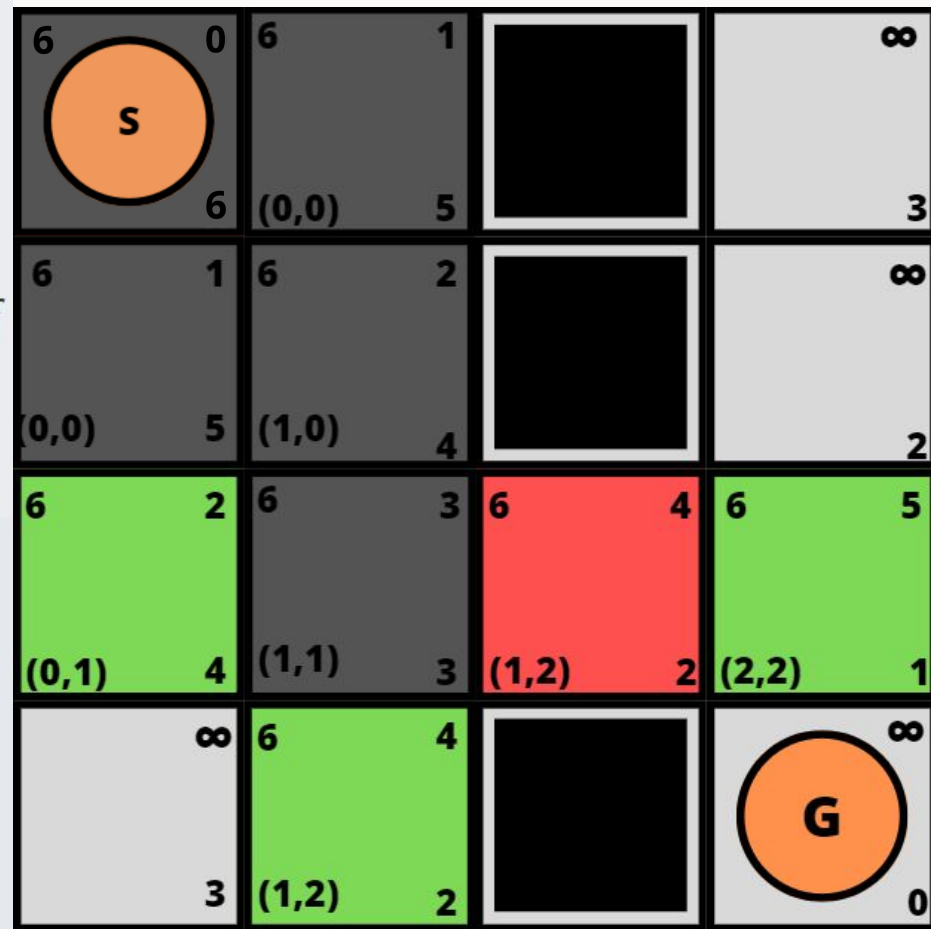
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$

2 $Open \leftarrow \{s_0\}$

3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$

4 while $Open \neq \emptyset$

5 Extrae un u desde $Open$ con menor valor- f

6 if u es objetivo return u

7 for each $v \in Succ(u)$ do

1 $cost_v = g(u) + c(u, v)$

2 if $cost_v \geq g(v)$ return

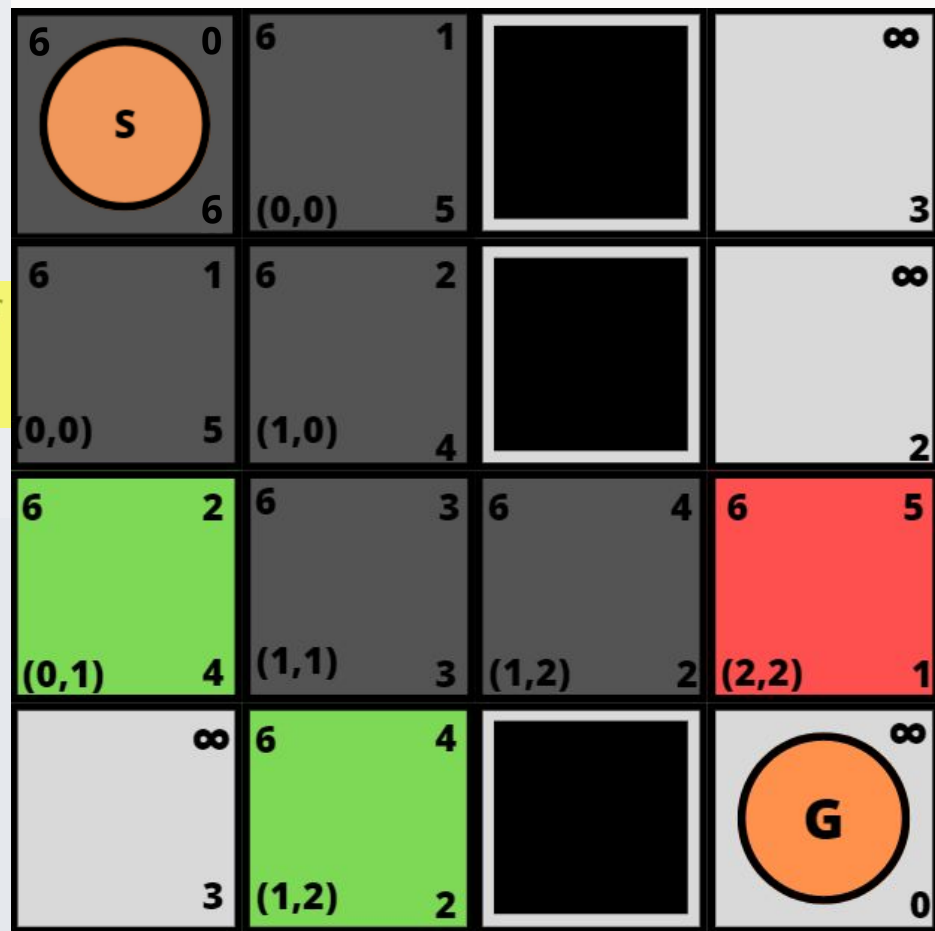
3 $parent(v) \leftarrow u$

4 $g(v) \leftarrow cost_v$

5 $f(v) \leftarrow g(v) + h(v)$

6 if $v \in Open$ then Reordenar $Open$

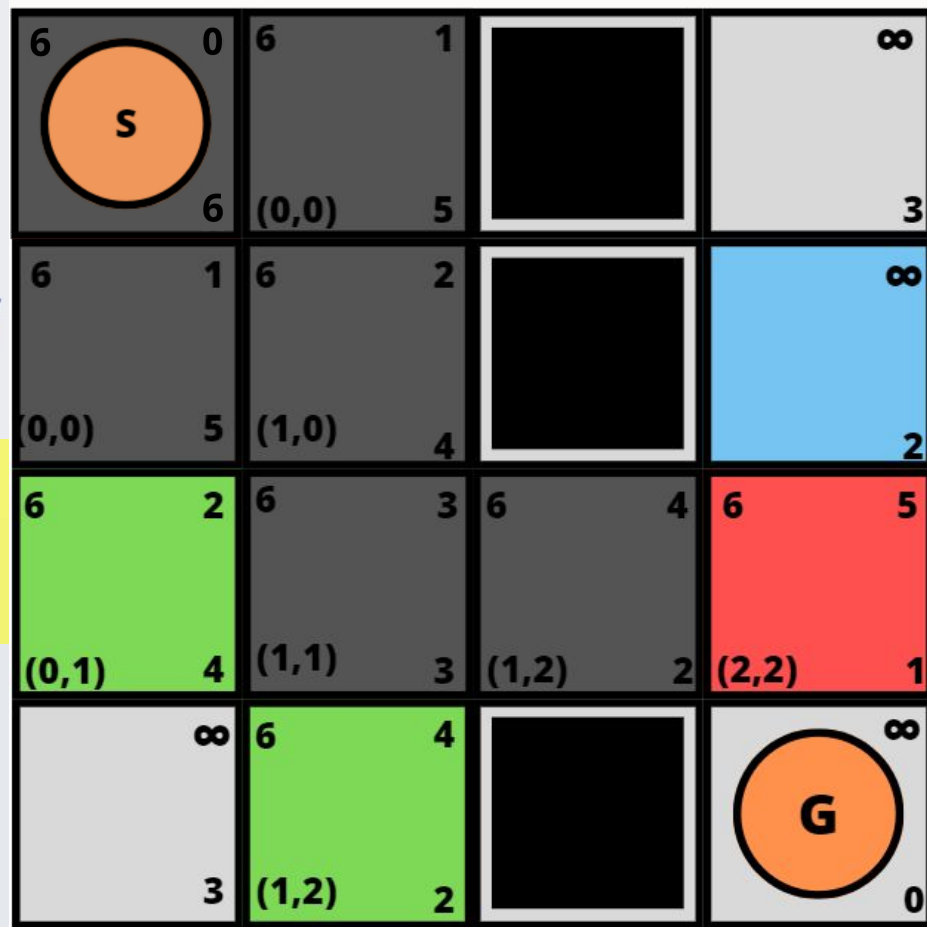
7 else Insertar v en $Open$



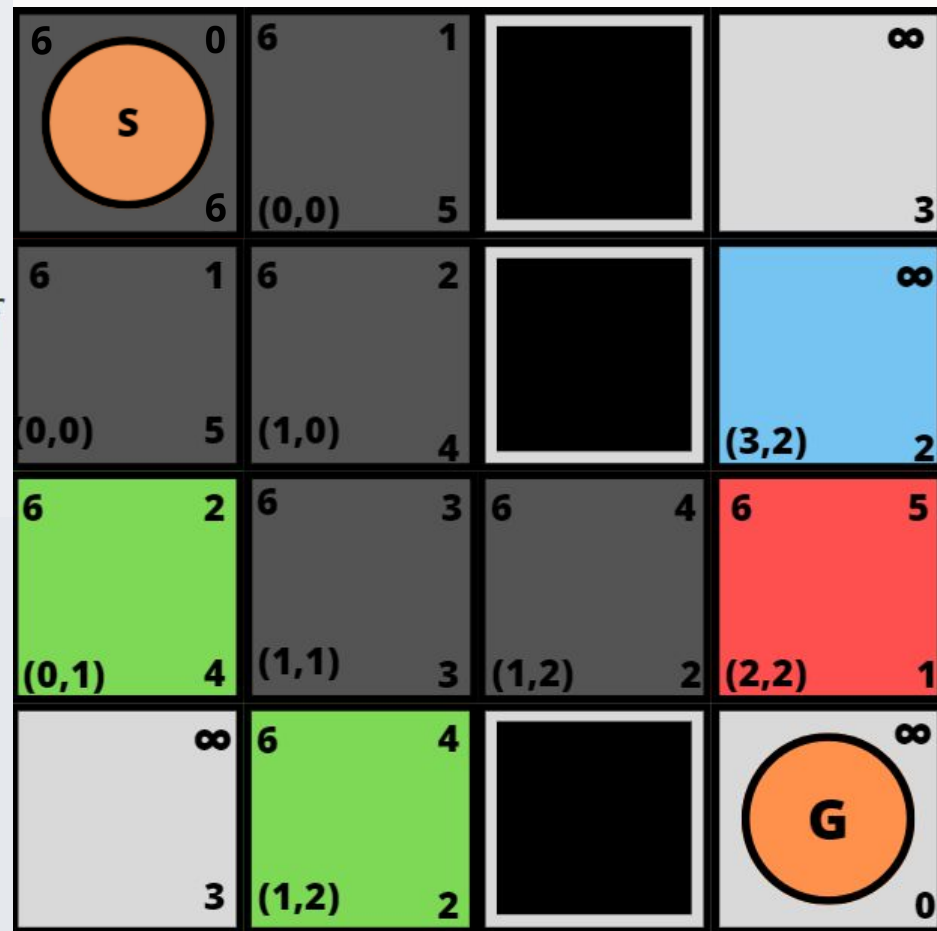
```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
8     1  $cost_v = g(u) + c(u, v)$ 
9     2 if  $cost_v \geq g(v)$  return
10    3  $parent(v) \leftarrow u$ 
11    4  $g(v) \leftarrow cost_v$ 
12    5  $f(v) \leftarrow g(v) + h(v)$ 
13    6 if  $v \in Open$  then Reordenar  $Open$ 
14    7 else Insertar  $v$  en  $Open$ 

```



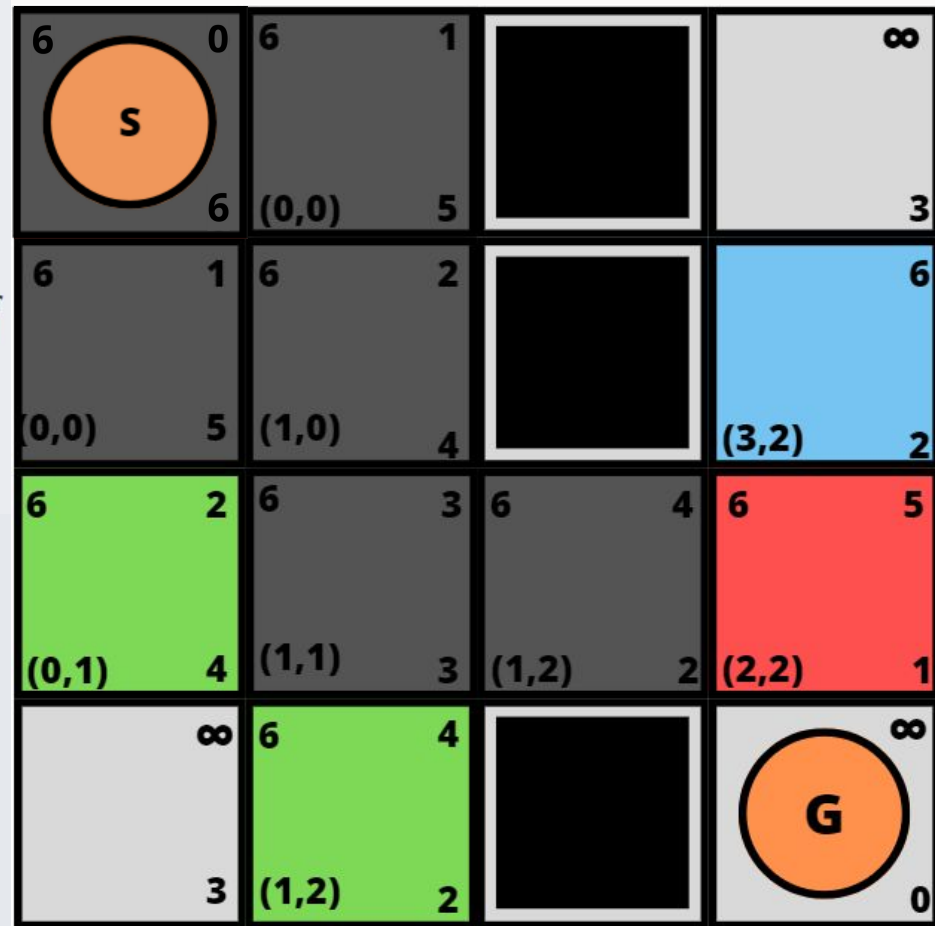
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
8     1  $cost_v = g(u) + c(u, v)$ 
9     2 if  $cost_v \geq g(v)$  return
10    3  $parent(v) \leftarrow u$ 
11    4  $g(v) \leftarrow cost_v$ 
12    5  $f(v) \leftarrow g(v) + h(v)$ 
13    6 if  $v \in Open$  then Reordenar  $Open$ 
14    7 else Insertar  $v$  en  $Open$ 

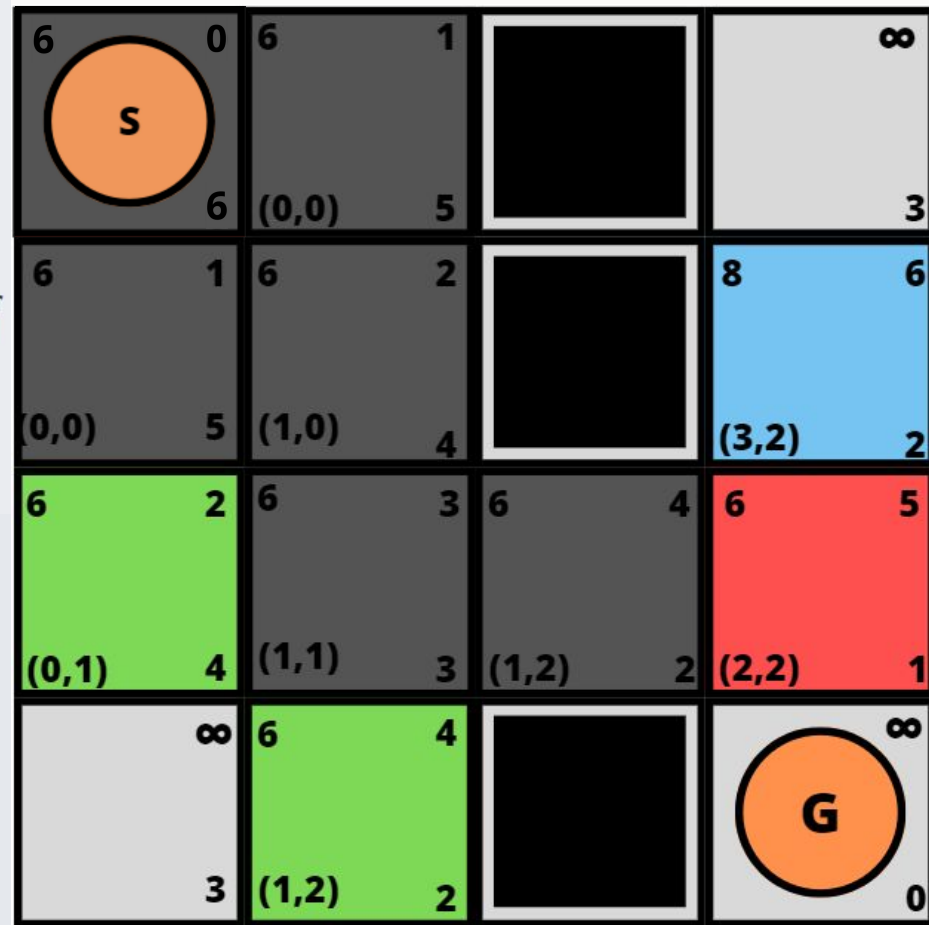
```



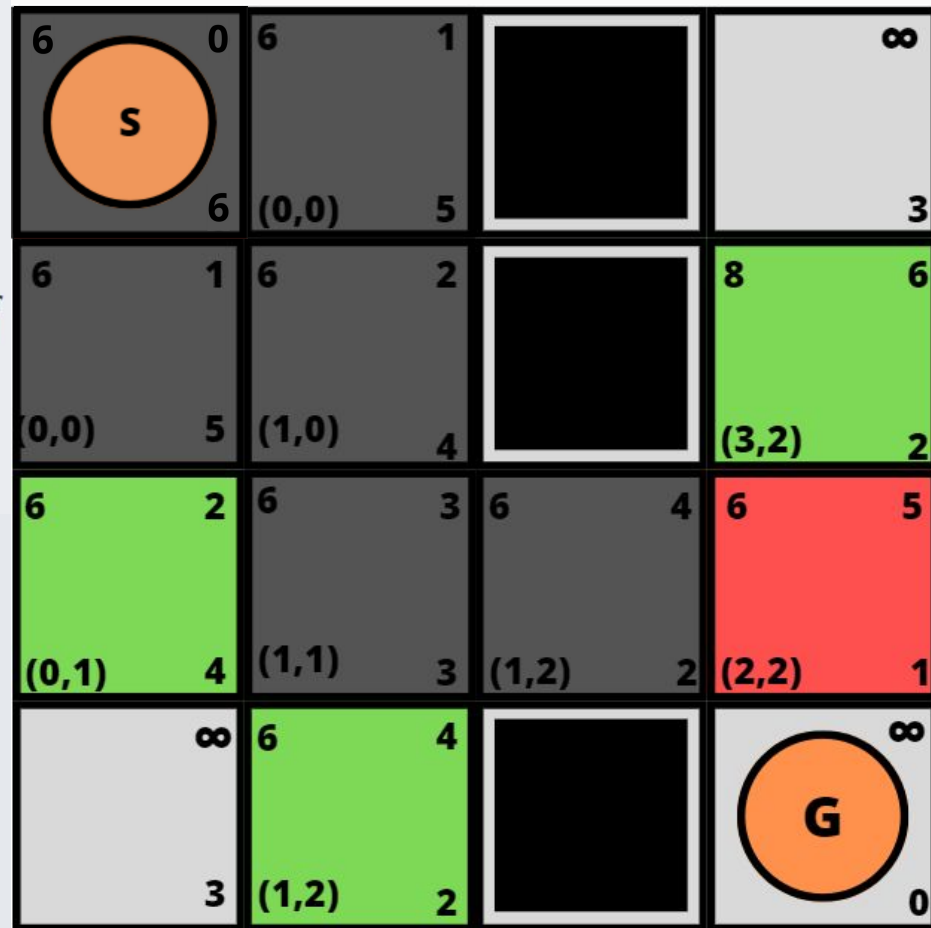

```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
8     1  $cost_v = g(u) + c(u, v)$ 
9     2 if  $cost_v \geq g(v)$  return
10    3  $parent(v) \leftarrow u$ 
11    4  $g(v) \leftarrow cost_v$ 
12    5  $f(v) \leftarrow g(v) + h(v)$ 
13    6 if  $v \in Open$  then Reordenar  $Open$ 
14    7 else Insertar  $v$  en  $Open$ 

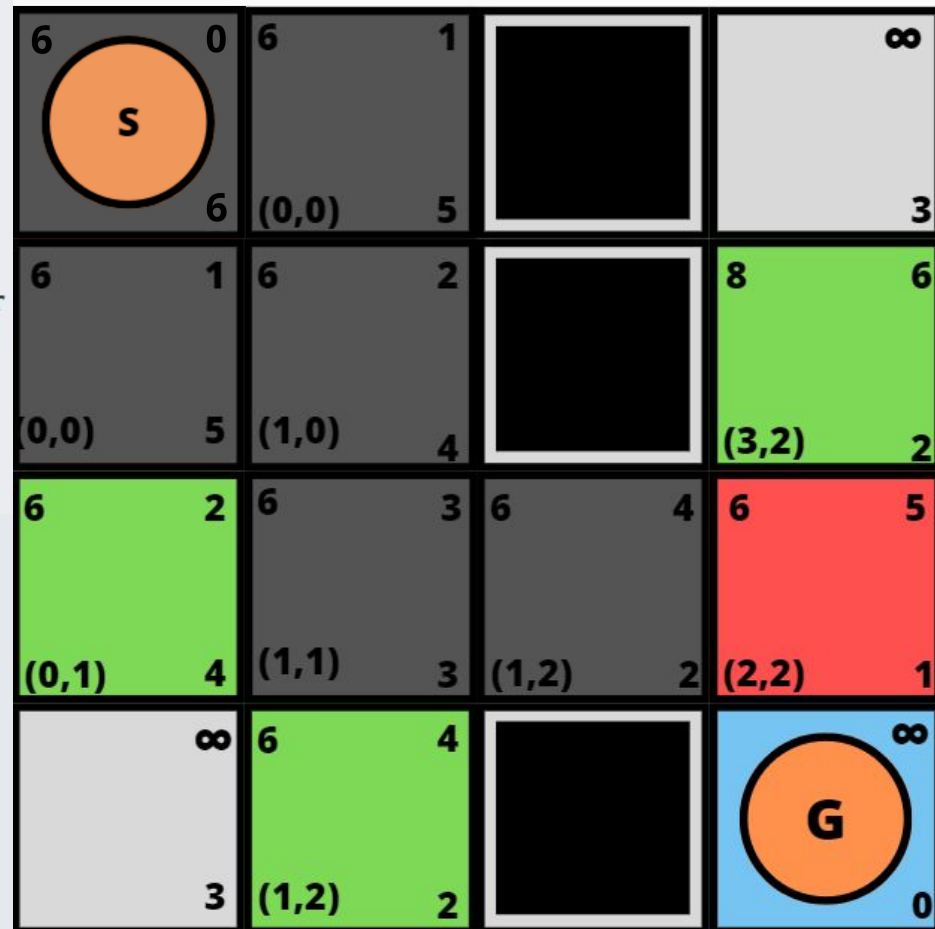
```



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



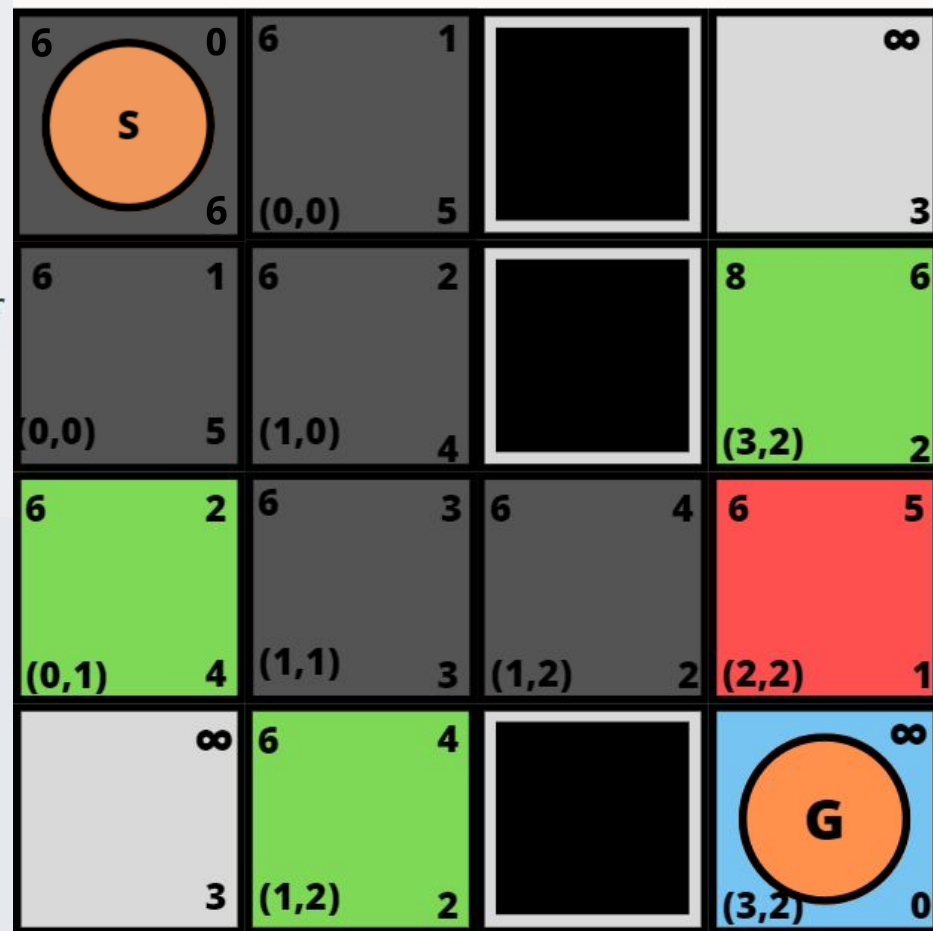
- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$



```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
      1  $cost_v = g(u) + c(u, v)$ 
      2 if  $cost_v \geq g(v)$  return
      3  $parent(v) \leftarrow u$ 
      4  $g(v) \leftarrow cost_v$ 
      5  $f(v) \leftarrow g(v) + h(v)$ 
      6 if  $v \in Open$  then Reordenar  $Open$ 
      7 else Insertar  $v$  en  $Open$ 

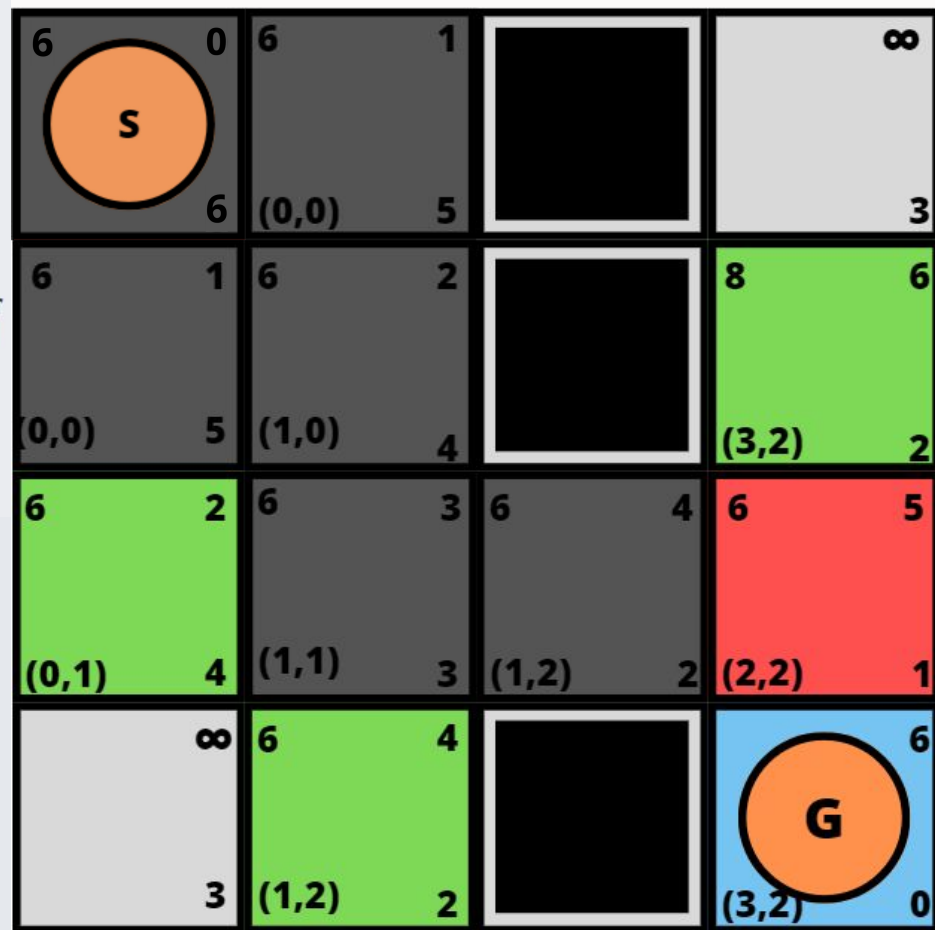
```



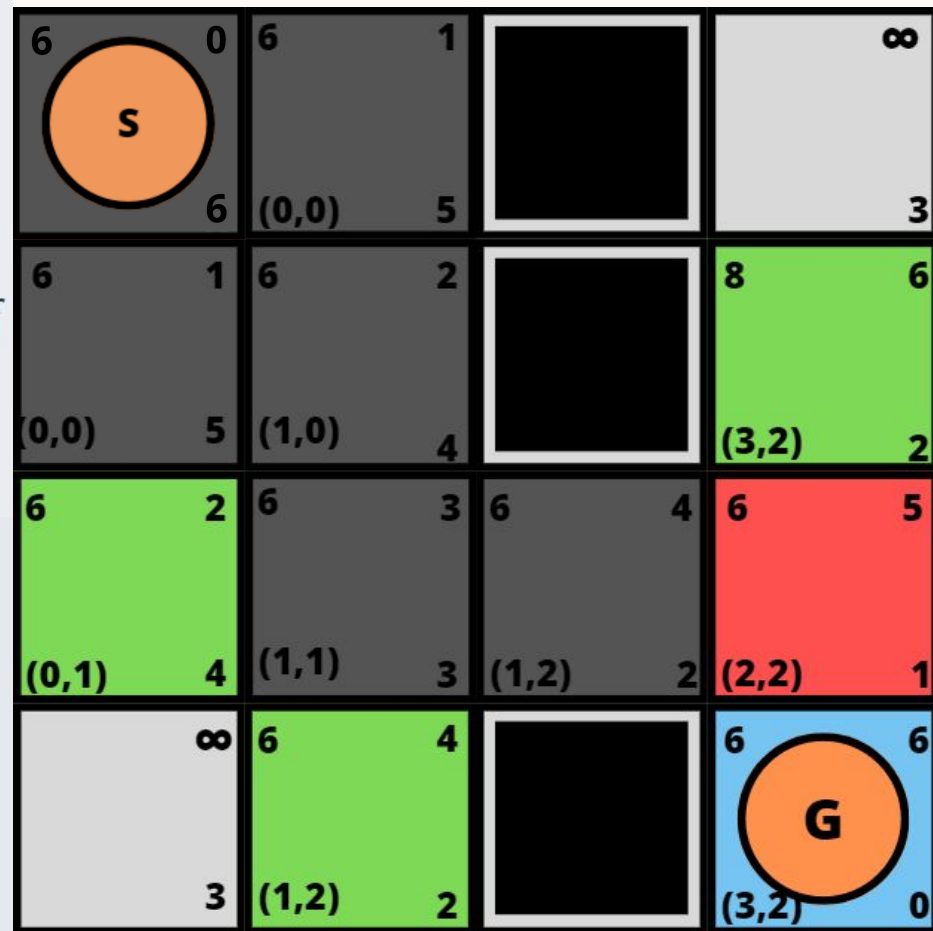
```

1 for each  $s \in \mathcal{S}$  do  $g(s) \leftarrow \infty$ 
2  $Open \leftarrow \{s_0\}$ 
3  $g(s_0) \leftarrow 0$ ;  $f(s_0) \leftarrow h(s_0)$ 
4 while  $Open \neq \emptyset$ 
5   Extrae un  $u$  desde  $Open$  con menor valor- $f$ 
6   if  $u$  es objetivo return  $u$ 
7   for each  $v \in Succ(u)$  do
8     1  $cost_v = g(u) + c(u, v)$ 
9     2 if  $cost_v \geq g(v)$  return
10    3  $parent(v) \leftarrow u$ 
11    4  $g(v) \leftarrow cost_v$ 
12    5  $f(v) \leftarrow g(v) + h(v)$ 
13    6 if  $v \in Open$  then Reordenar  $Open$ 
14    7 else Insertar  $v$  en  $Open$ 

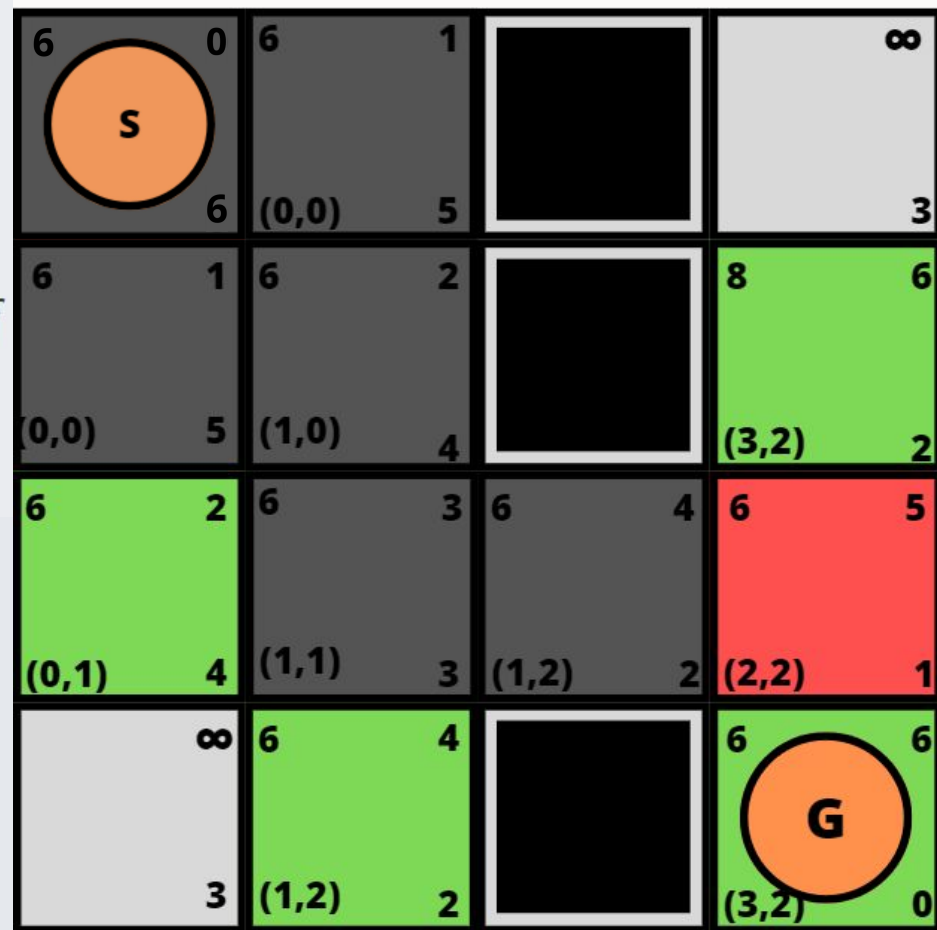
```



- 1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 while $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 if u es objetivo return u
- 7 for each $v \in Succ(u)$ do
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 if $cost_v \geq g(v)$ return
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 if $v \in Open$ then Reordenar $Open$
 - 7 else Insertar v en $Open$



- 1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 while $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 if u es objetivo return u
- 7 for each $v \in Succ(u)$ do
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 if $cost_v \geq g(v)$ return
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 if $v \in Open$ then Reordenar $Open$
 - 7 else Insertar v en $Open$



1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$

2 $Open \leftarrow \{s_0\}$

3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$

4 while $Open \neq \emptyset$

5 Extrae un u desde $Open$ con menor valor- f

6 if u es objetivo return u

7 for each $v \in Succ(u)$ do

1 $cost_v = g(u) + c(u, v)$

2 if $cost_v \geq g(v)$ return

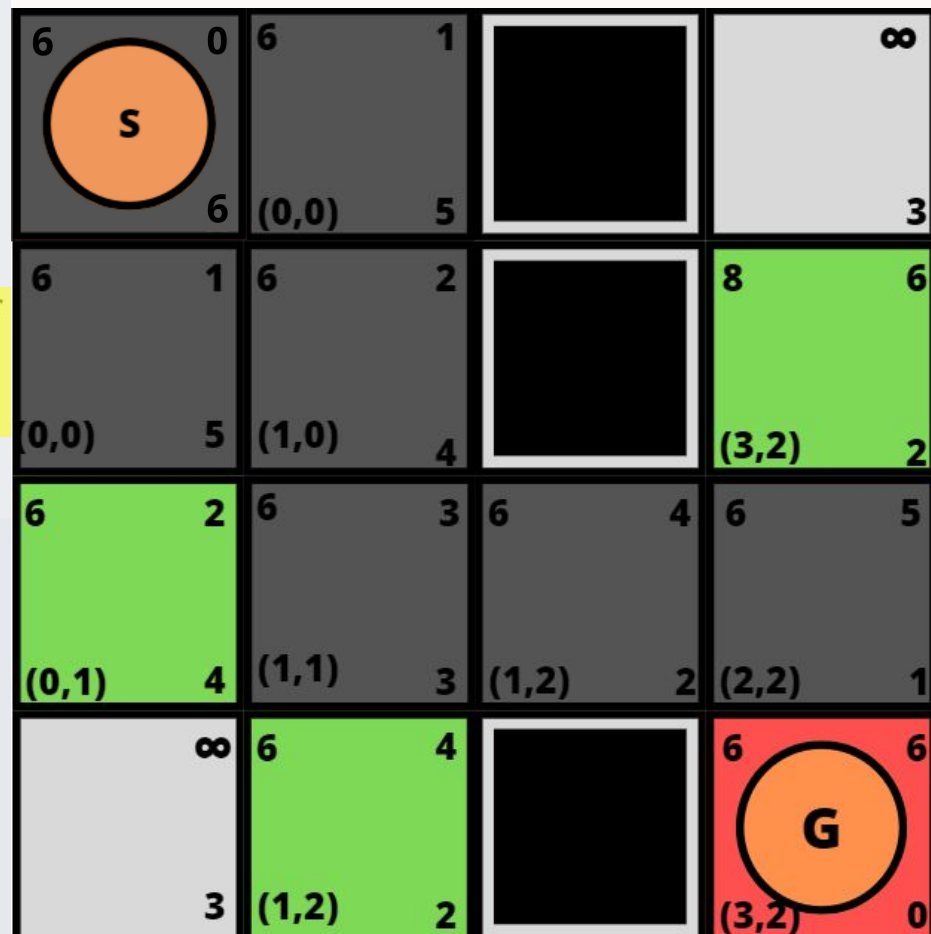
3 $parent(v) \leftarrow u$

4 $g(v) \leftarrow cost_v$

5 $f(v) \leftarrow g(v) + h(v)$

6 if $v \in Open$ then Reordenar $Open$

7 else Insertar v en $Open$



1 for each $s \in \mathcal{S}$ do $g(s) \leftarrow \infty$

2 $Open \leftarrow \{s_0\}$

3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$

4 while $Open \neq \emptyset$

5 Extrae un u desde $Open$ con menor valor- f

6 if u es objetivo return u

7 for each $v \in Succ(u)$ do

1 $cost_v = g(u) + c(u, v)$

2 if $cost_v \geq g(v)$ return

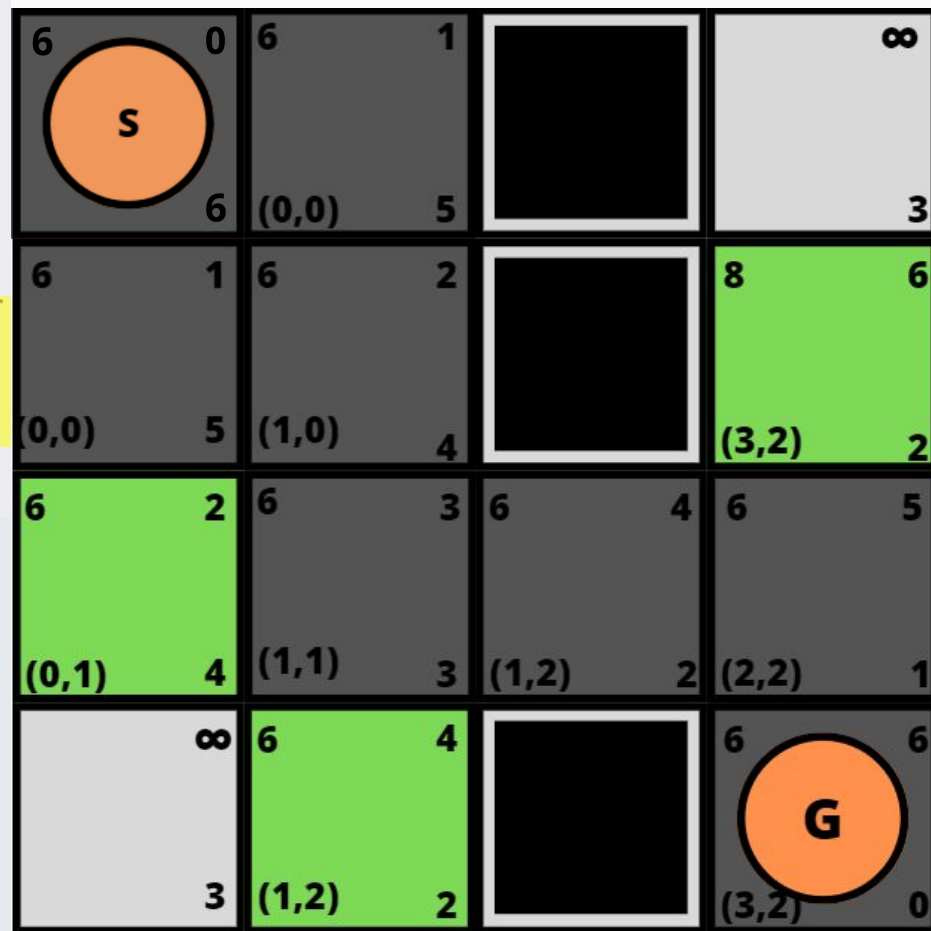
3 $parent(v) \leftarrow u$

4 $g(v) \leftarrow cost_v$

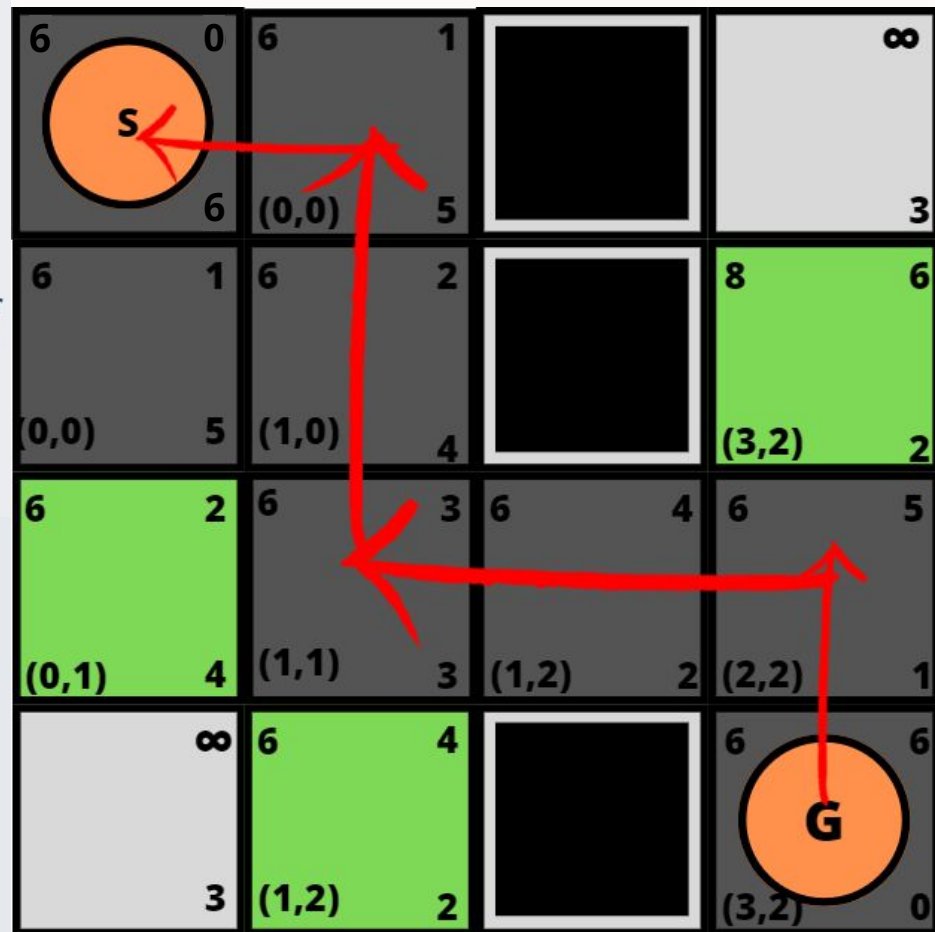
5 $f(v) \leftarrow g(v) + h(v)$

6 if $v \in Open$ then Reordenar $Open$

7 else Insertar v en $Open$



- 1 **for each** $s \in \mathcal{S}$ **do** $g(s) \leftarrow \infty$
- 2 $Open \leftarrow \{s_0\}$
- 3 $g(s_0) \leftarrow 0$; $f(s_0) \leftarrow h(s_0)$
- 4 **while** $Open \neq \emptyset$
- 5 Extrae un u desde $Open$ con menor valor- f
- 6 **if** u es objetivo **return** u
- 7 **for each** $v \in Succ(u)$ **do**
 - 1 $cost_v = g(u) + c(u, v)$
 - 2 **if** $cost_v \geq g(v)$ **return**
 - 3 $parent(v) \leftarrow u$
 - 4 $g(v) \leftarrow cost_v$
 - 5 $f(v) \leftarrow g(v) + h(v)$
 - 6 **if** $v \in Open$ **then** Reordenar $Open$
 - 7 **else** Insertar v en $Open$





¿Por qué A^* es óptimo? - Youtube

¿Por qué A^* es óptimo? - YouTube



Búsqueda Adversaria

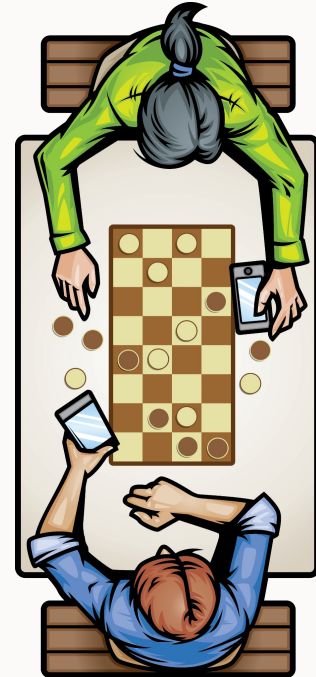


Problemas a desarrollar

Teoría de juegos

Problemas de dos jugadores, de turnos, **información perfecta** y **suma cero**.

- **Información perfecta:** Conocemos toda la información del tablero de juego (estados).
- **Suma cero:** Todo lo que me beneficia a mí, perjudica a mi contrincante en igual medida y viceversa.





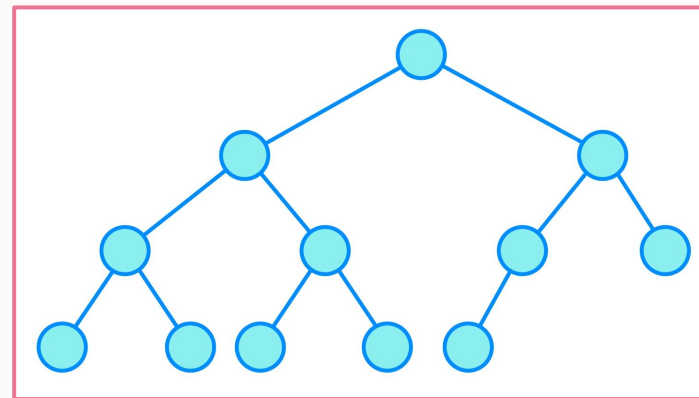
Búsqueda

Podemos considerar un juego como un **espacio de búsqueda**:

- **Estado del juego** = nodo
- **Jugadas/acciones** = aristas

El **objetivo** será un **estado donde ganamos** (estado objetivo):

- Gato: Tres fichas (nuestras) en línea
- Conecta-4: Cuatro fichas (nuestras) en línea
- Ajedrez: Jaque-mate al rey del oponente





Adversario

Yo **no sé** cómo va a jugar mi adversario, pero **asumo que siempre juega lo que es mejor para él**. En consecuencia, considero que **juega lo peor para mí**.

Luego, si mi adversario toma decisiones subóptimas durante el juego, solo es mejor para mí.



Algoritmo Minimax

La búsqueda puede presentarse como una simulación del juego entre dos jugadores:

Max y Min.

Si es mi turno: **Max**

- Elijo la mejor jugada que tengo disponible, para **max**imizar el puntaje.

Si es el turno de mi oponente: **Min**

- Asumo que mi oponente siempre toma las decisiones óptimas (para él) entonces elige la opción que **min**imiza mi puntaje (la mejor jugada para mi oponente es la que me “hace peor”).

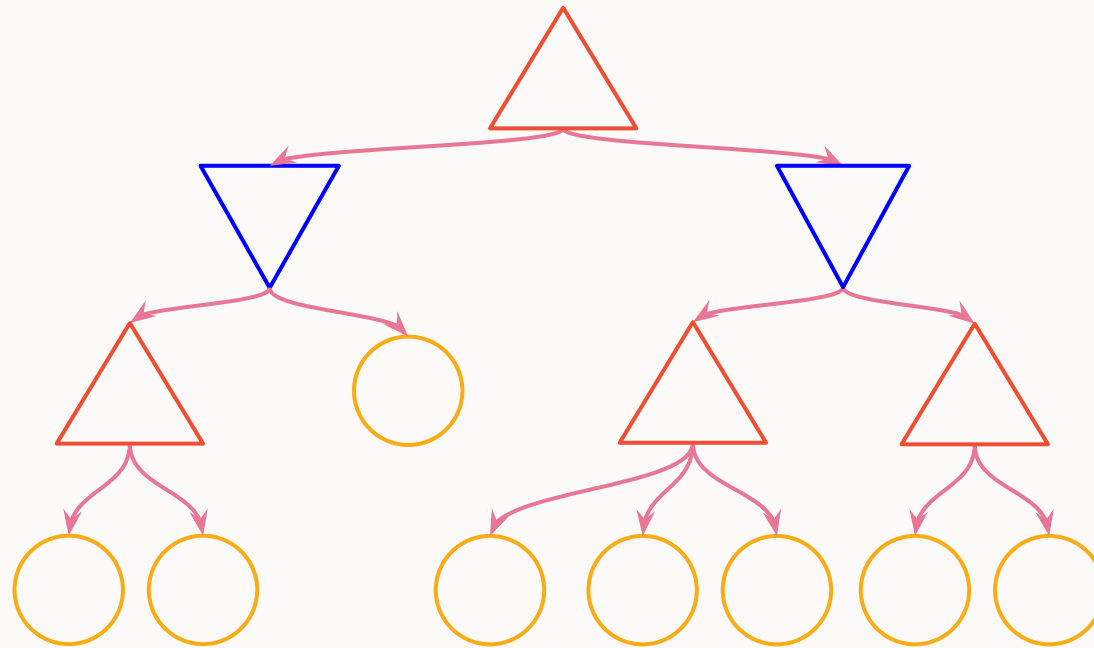


Algoritmo Minimax

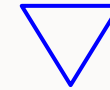
```
function MAX-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v, move  $\leftarrow -\infty$ 
  for each a in game.ACTIONS(state) do
    v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a))
    if v2 > v then
      v, move  $\leftarrow$  v2, a
  return v, move
```

```
function MIN-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v, move  $\leftarrow +\infty$ 
  for each a in game.ACTIONS(state) do
    v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a))
    if v2 < v then
      v, move  $\leftarrow$  v2, a
  return v, move
```

Cálculo del valor Minimax



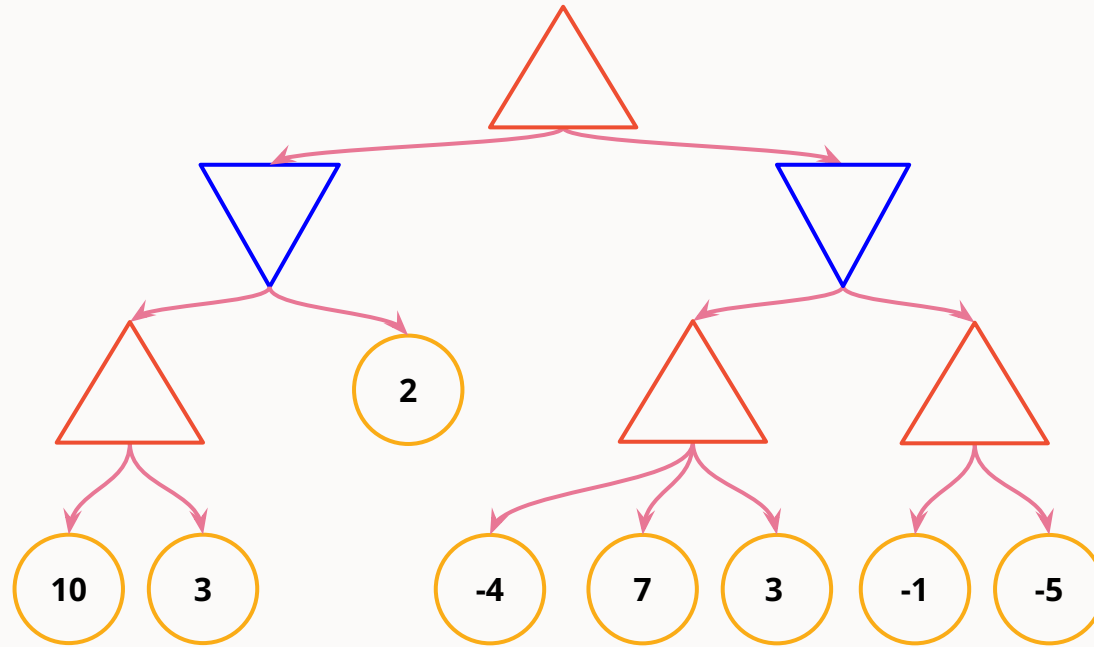
 Jugador Max

 Jugador Min

 Estados Terminales

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$

Cálculo del valor Minimax



 Jugador Max

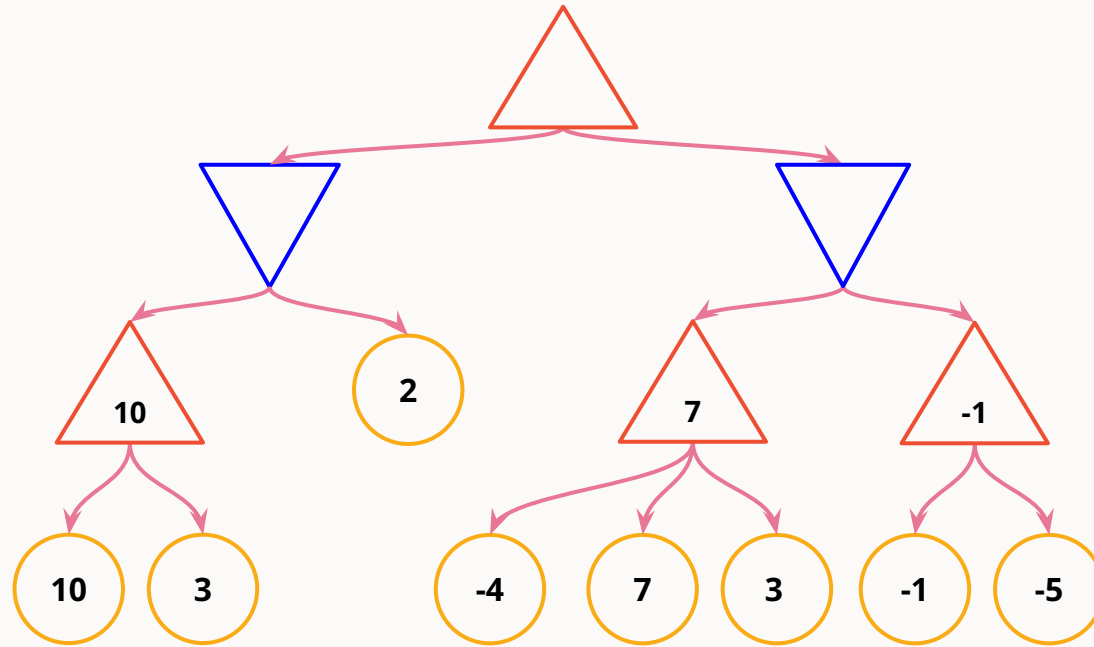
 Jugador Min

 Estados Terminales

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$



Cálculo del valor Minimax



 Jugador Max

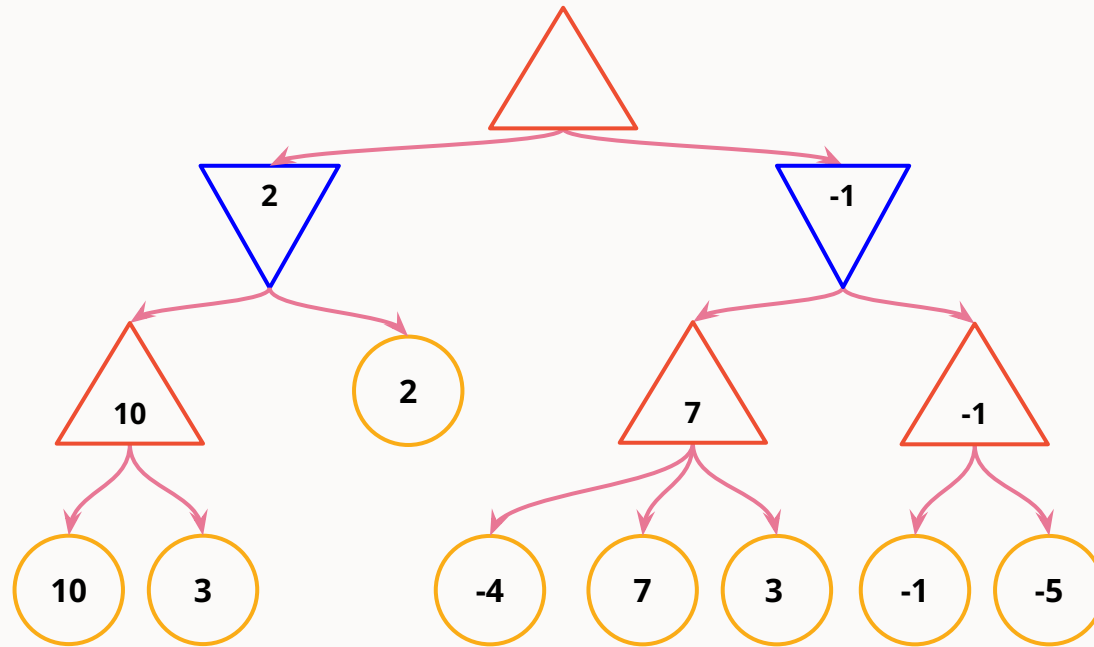
 Jugador Min

 Estados Terminales

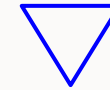
$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$



Cálculo del valor Minimax



 Jugador Max

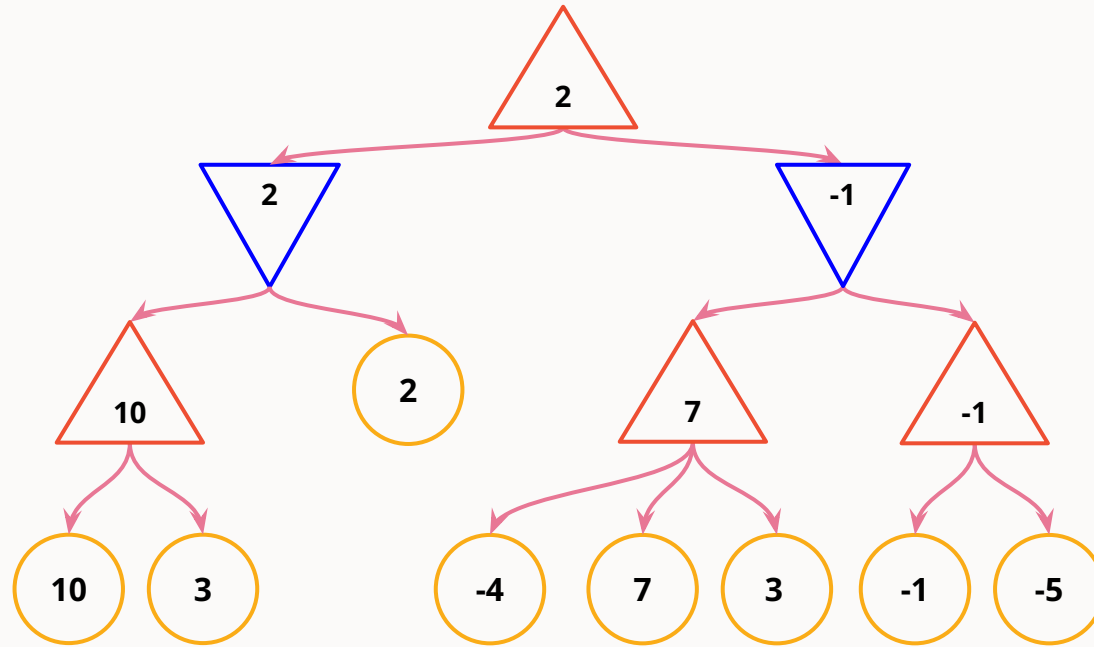
 Jugador Min

 Estados Terminales

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$



Cálculo del valor Minimax



 Jugador Max

 Jugador Min

 Estados Terminales

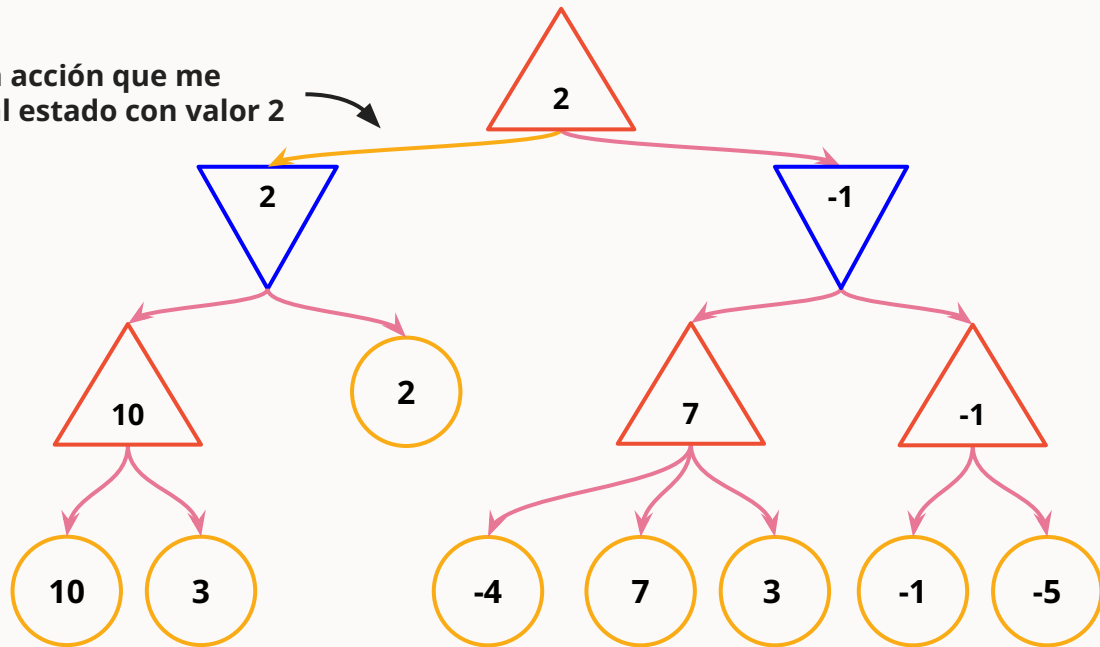
$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$



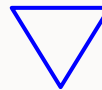
Cálculo del valor Minimax



Elijo la acción que me
lleva al estado con valor 2



Jugador Max



Jugador Min



Estados Terminales

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$





Rendimiento Minimax

La búsqueda Minimax corre DFS **sobre el árbol de estados completo**. Esto genera muchísimos estados para juegos complejos (ej. Ajedrez)

Para mejorar el rendimiento podemos:

- **Acotar** la profundidad del árbol de búsqueda
- **Podar** ramas del árbol de búsqueda



Acotar profundidad del árbol de búsqueda

Función de evaluación

- Es como una **heurística** pero en el contexto de búsqueda adversaria
- **Asigna puntajes a estados no terminales**, lo que permite limitar a un máximo la altura de un árbol

Básicamente, hace una estimación sobre “quién va ganando” llegada una altura máxima definida.



Poda Alpha-Beta

Podemos podar buena parte del árbol si guardamos dos parámetros adicionales en cada llamada a la búsqueda Minimax:

- Alpha: cota **inferior** del valor de un nodo
- Beta: cota **superior** del valor de un nodo

Tener estas cotas nos indica si es necesario calcular un nodo.
Esta decisión **no afecta la optimalidad de la solución.**



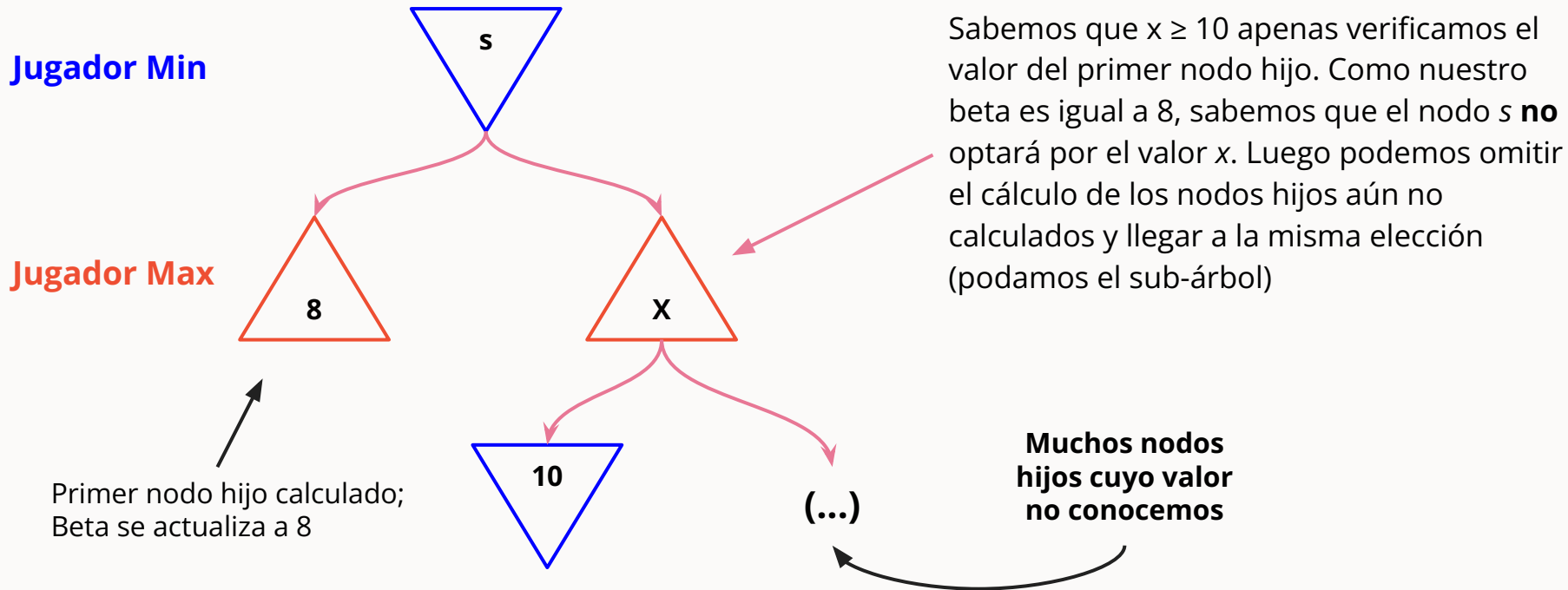
Poda Alpha-Beta

```
function MAX-VALUE(game, state,  $\alpha$ ,  $\beta$ ) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
   $v \leftarrow -\infty$ 
  for each a in game.ACTIONS(state) do
     $v2, a2 \leftarrow$  MIN-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )
    if  $v2 > v$  then
       $v, move \leftarrow v2, a$ 
       $\alpha \leftarrow$  MAX( $\alpha$ ,  $v$ )
    if  $v \geq \beta$  then return  $v, move$ 
  return  $v, move$ 
```

```
function MIN-VALUE(game, state,  $\alpha$ ,  $\beta$ ) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
   $v \leftarrow +\infty$ 
  for each a in game.ACTIONS(state) do
     $v2, a2 \leftarrow$  MAX-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )
    if  $v2 < v$  then
       $v, move \leftarrow v2, a$ 
       $\beta \leftarrow$  MIN( $\beta$ ,  $v$ )
    if  $v \leq \alpha$  then return  $v, move$ 
  return  $v, move$ 
```




Poda Alpha-Beta - Cota superior Beta

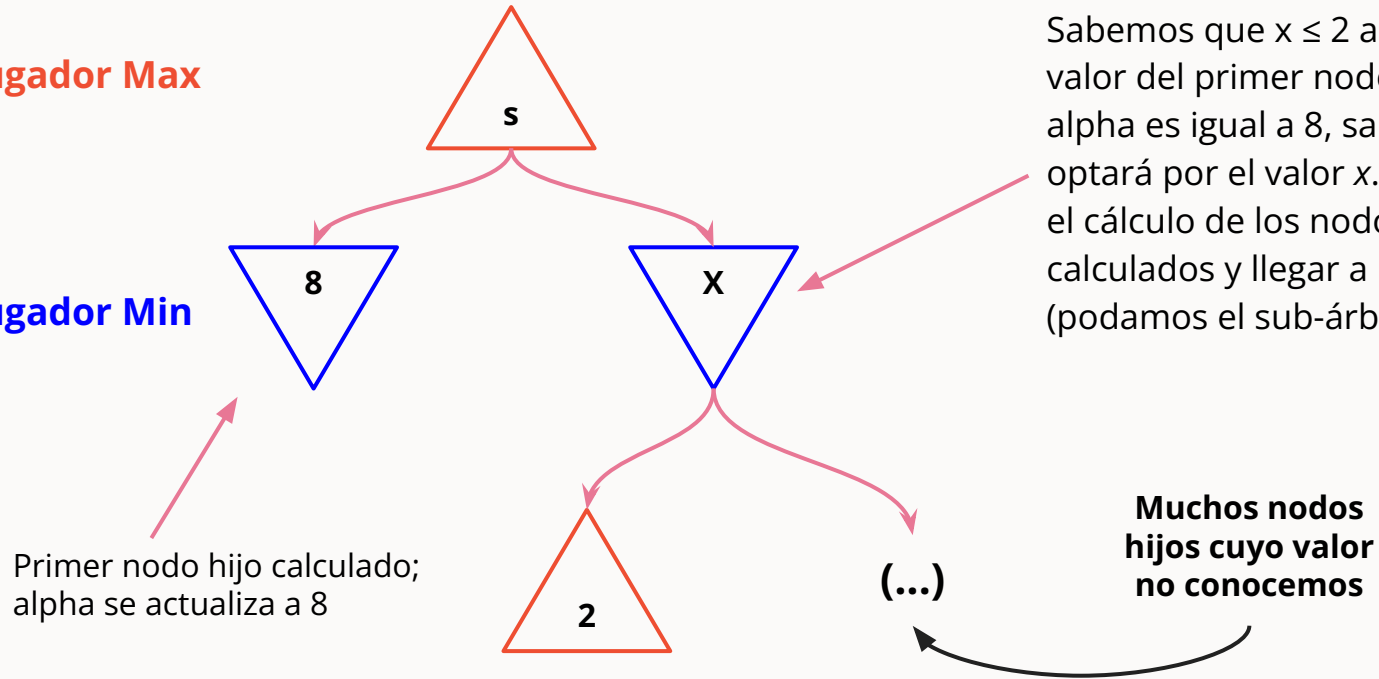




Poda Alpha-Beta - Cota inferior Alpha

Jugador Max

Jugador Min





Apoyo T3



Parte 1: DCC Puzzle 15

Algunos tips

- Utilicen tablas para comparar sus resultados !!!!
- Sean claros con sus análisis de los distintos algoritmos.
- Respondan en un PDF **todas** las preguntas del enunciado.

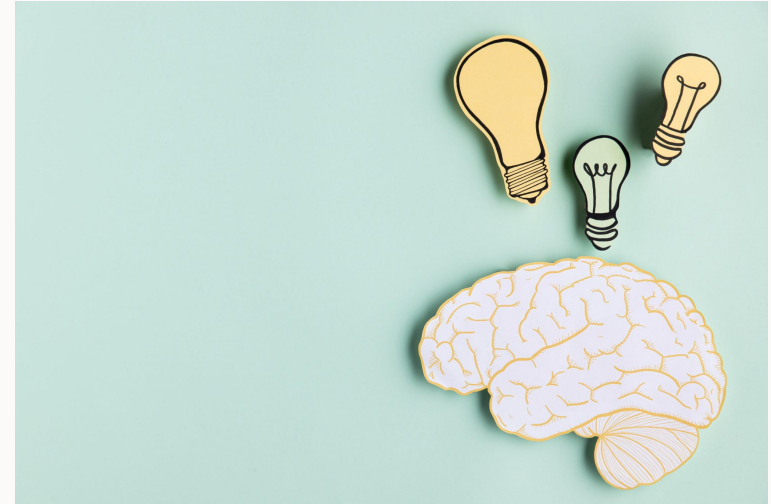
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	



Parte 2: Teoría

Algunos tips

- Entiendan bien el video explicativo
- Sean claros con las demostraciones y explicaciones, no hace falta que sea muy rigurosa mientras se explique bien.

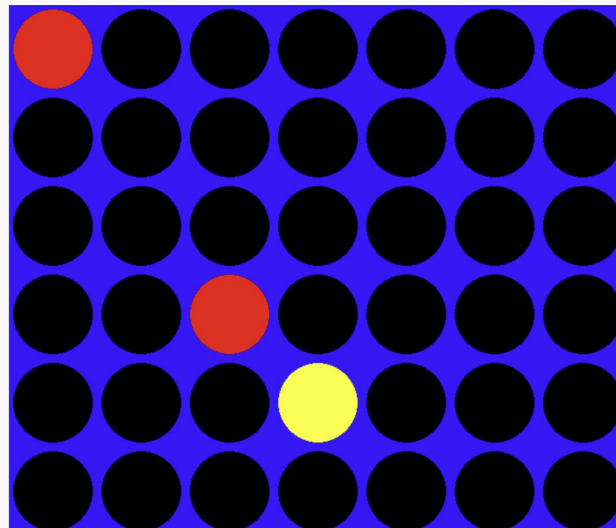




Parte 3: DCConecta4 Spin

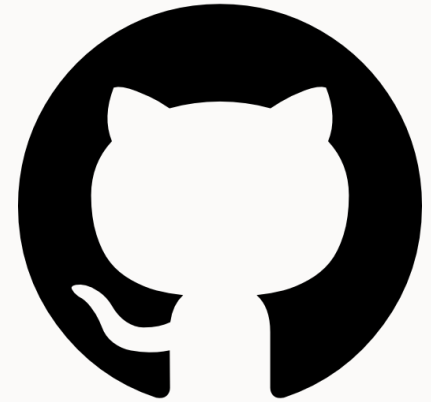
Algunos tips

- Enfóquense en entender el pseudo-código de Minimax tradicional para extenderlo a Max^N
- Recuerden que, para la implementación en código, deben retornar el **mejor valor y la mejor jugada**
- Prueben cómo juegan los agentes de IA para armar sus funciones de evaluación





**Pueden utilizar las
issues para responder
sus dudas :)**





Dudas